

Differentiable force density method for the design of lightweight structures

Rafael Pastrana ^{a, b, *}, Deniz Oktay ^c, Kai-Uwe Bletzinger ^d, Ryan P. Adams ^c,
Sigrid Adriaenssens ^b

^a School of Architecture, Princeton University, Princeton, United States of America

^b Form Finding Lab, Department of Civil and Environmental Engineering, Princeton University, Princeton, United States of America

^c Laboratory for Probabilistic and Intelligent Systems, Department of Computer Science, Princeton University, Princeton, United States of America

^d Chair of Structural Analysis, Technical University of Munich, Munich, Germany

ARTICLE INFO

Keywords:

Force density method
Form-finding
Structural design
Shape optimization
Gradient descent
Automatic differentiation
Differentiable simulation

ABSTRACT

Lightweight roofs, bridges, and towers span long distances with slender cross-sections due to their mechanically efficient shapes. Funicular geometry, spatial configurations where external loads are carried primarily through internal axial forces, can endow these structures with such efficiency. However, converting funicular geometry into a feasible design that satisfies architectural, fabrication, and structural constraints remains challenging due to nonlinear mechanical behaviors and high-dimensional search spaces. This paper presents the differentiable force density method (DFDM), a form-finding method for the automated design of efficient and feasible shapes for lightweight structures modeled as pin-jointed bar systems. The DFDM leverages gradient-based optimization to compute funicular geometries subject to design constraints, while accounting for large-displacement effects and shape-dependent loads, such as self-weight and pressure. By extending automatic differentiation with analytical adjoints, the DFDM solves high-dimensional inverse problems to planarize gridshell panels, simulate pneumatic membranes, and retrofit masonry vaults up to four orders of magnitude faster than gradient-free algorithms and one order of magnitude faster than general-purpose automatic differentiation. Its full differentiability enables load-finding and support-finding. These rich parametrizations allow the optimization of loads and support positions jointly with force densities to discover better-performing solutions than with force density optimization alone. This work unifies structural simulations and differentiable programming, opening up pathways to expedite the development of meter-scale systems that lower material consumption in the built environment.

1. Introduction

Lightweight structures like membrane roofs, cable-stayed bridges, and gridshells are mechanically efficient systems at the architectural scale. Their high stiffness-to-weight ratios allow them to sustain large loads over spans of tens or hundreds of meters with slender cross-sections, and hence with less material volume than structural systems that rely on frame or beam-column action [1,2]. Their mechanical efficiency is often compared to that of systems only found in nature, like a chicken's eggshell, whose thickness is

* Corresponding author.

E-mail addresses: arpastrana@pastrana.edu (R. Pastrana), doktay@princeton.edu (D. Oktay), kub@tum.de (K.-U. Bletzinger), rpa@princeton.edu (R.P. Adams), sadriaen@princeton.edu (S. Adriaenssens).

<https://doi.org/10.1016/j.cma.2026.118783>

Received 30 September 2025; Received in revised form 31 December 2025; Accepted 22 January 2026

Available online 1 May 2026

0045-7825/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

on the order of just two percent of its radius [3,4]. Geometry is at the core of these structures' outstanding performance. Lightweight structures are predominantly *form-active* [5]. They derive their stiffness from their optimal shapes that resist external loads through in-plane axial forces, while dissipating bending and torque, and thereby reducing the structure's strain energy [6,7]. In continuous systems like shells, this type of mechanical response is referred to as *membrane* action, whereas in the discrete case, such as in assemblies of struts and cables, or more generally, bars, it is *truss* action.

1.1. Finding efficient shapes

An efficient shape is central to designing a lightweight structure, but it cannot be prescribed from the outset. Such a shape is a function of the structure's behavior, the applied loads, and the support conditions, so it must be *found* [8].

One recurrent approach to efficient shape computation is with a *form-finding* method that imposes the desired axial-dominant behavior on a pin-jointed bar system, a discrete representation of the structure. The bar system models cable nets and gridshells directly, and approximates membranes and shells by condensing in-plane membrane forces into a network of force resultants akin to a strut-and-tie model [9,10]. In a general, braced or unbraced, pin-jointed bar system, enforcing truss action with prescribed force directions (tension or compression) introduces geometric nonlinearity. Consequently, the bar system must undergo large displacements to reach an equilibrium state. The resulting configuration is a *funicular geometry*: a shape in which every bar carries an axial force with a prescribed direction, and the system is in equilibrium with the applied loads for given boundary conditions, regardless of its topology. A structure embodying a funicular geometry lowers its strain energy and attains mechanical efficiency.

Several form-finding methods exist, with historical precedents that date back to the seminal works of Varignon [11], Rankine [12], and Maxwell [13] on graphic statics, and to fundamental continuum approaches, such as Pucher's formulation of membrane theory and the Airy stress function [14–17]. An established taxonomy [18] classifies numerical form-finding methods for pin-jointed bar systems into three groups: stiffness-based [19,20], dynamic [21–23], and geometric methods [24–27]. The geometric methods are advantageous because they avoid explicit material models, offering more compact problem parametrizations for design without compromising on simulation accuracy. In this taxonomy, the force density method stands out as the foundation of all other geometric approaches for bar systems [18].

The *force density method* (FDM) [28,29] is a form-finding method that computes funicular geometries for structures modeled as pin-jointed bar systems. Since its invention, motivated by the construction of the Munich Olympic Stadium in the early 1970s [30], the FDM has been extended to handle compressive structures like concrete shells and masonry vaults [31–34], and tensile-compressive ones like tensegrities and suspension bridges [35–38]. Although the FDM was at first developed to simulate tensile cable nets, its subsequent extensions to handle compressive force states intuitively follow from Hooke's formalism¹ on the duality of hanging chains and rigid arches [39–41]. Additional studies on the FDM have provided insight into the equivalence between force densities and Piola-Kirchhoff forces [42], serving as an instrumental basis to extend the method to continuous structures like membranes and shells [25,35,43]. A key reason for the FDM's strong presence in the field of form-finding is its numerical tractability for simulation. It reformulates the originally nonlinear equilibrium problem using *force densities*, ratios of internal force to member length, rather than force values as model inputs [28,29]. This reparametrization linearizes the computation of equilibrium states, offering straightforward access to a rich space of funicular geometries.

1.2. Structural design is an inverse problem

Form-finding methods like the FDM support lightweight structure design, but only to a limited extent. The FDM excels at computing funicular geometries by solving a *forward problem*: mapping parameters to geometry, $\theta \rightarrow \mathbf{x}$ (Fig. 1a), and facilitating the exploration of a myriad of such efficient shapes with simple variations of θ (Fig. 1b). However, structural design is fundamentally an *inverse problem* [44–46], where funicular geometry must be steered to satisfy additional constraints prescribed by a design specification. Beyond efficiency, a structure should be buildable with specific materials and fabrication processes [47], comply with construction budget limits [7], and respond to architectural and site considerations [38]. Therefore, for design, one rather looks for the parameters θ that produce a geometry $\hat{\mathbf{x}}$ that fulfills these constraints, while maintaining funicularity for mechanical efficiency. Achieving these goals requires a map from geometry to parameters, $\mathbf{x} \rightarrow \theta$, where $\mathbf{x} = \hat{\mathbf{x}}$ for a given specification. Nevertheless, this relationship runs counter to the forward nature of the FDM and simulations overall, which challenges the solution of inverse problems for structural design utilizing current approaches (Fig. 1a).

To illustrate this challenge, consider the target shape $\hat{\mathbf{x}}$ in Fig. 1, and suppose the task is to compute a funicular geometry that best approximates it. The target shape could express architectural visual intent for a new gridshell roof or represent the geometry of a historical masonry vault that needs to be analyzed for restoration purposes [31,48,49]. Running the FDM under different parameter combinations via an exhaustive search (Fig. 1b) would rarely deliver a close match within practical time and compute budgets. Consequently, such trial-and-error approaches often settle on feasible but inefficient forms that ultimately turn into material-intensive structures that increase embodied carbon and construction costs [50,51]. The challenges of solving inverse problems are compounded by the high number of simulation parameters needed to capture a structure's behavior faithfully, its nonlinearities, and the design constraints, all of which prevent direct inversion of the forward map [46]. These issues stress the need for a systematic approach to

¹ Originally in Latin, *ut pendet continuum flexile, sic stabit contiguum rigidum inversum*. Hooke's formalism on funicular geometry translates to "as hangs the flexible line, so but inverted will stand the rigid arch".

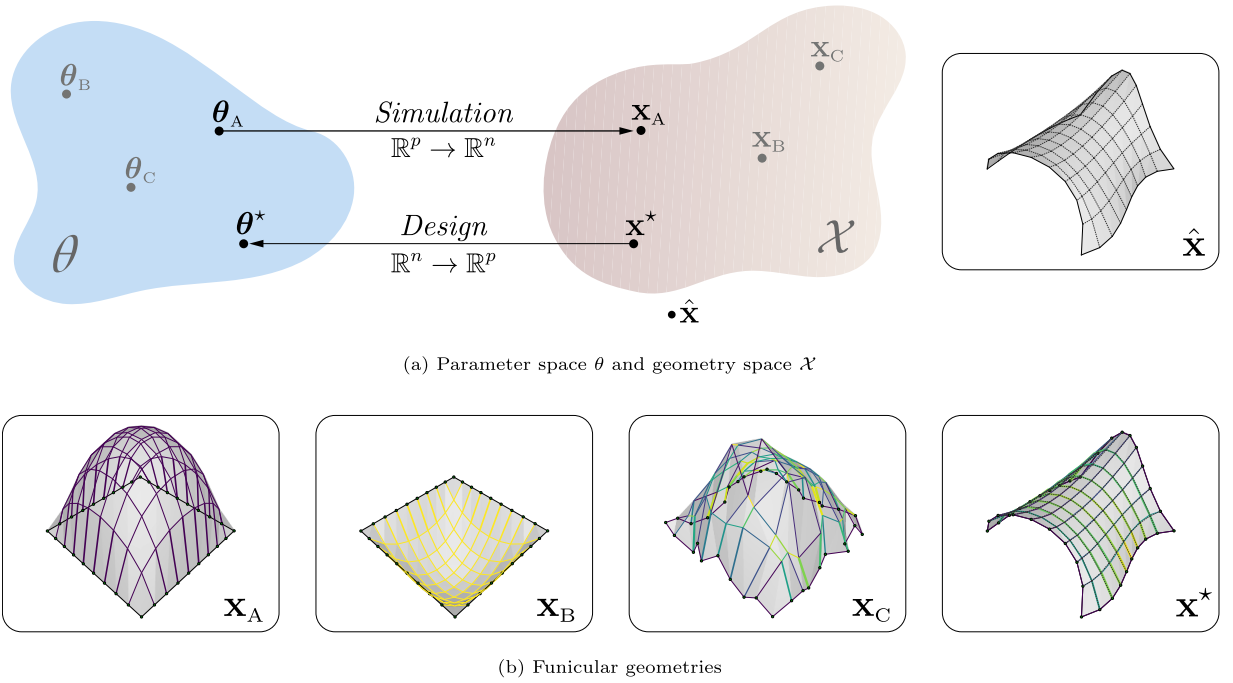


Fig. 1. A structural simulation method like the FDM computes funicular geometry $\mathbf{x} \in \mathcal{X}$ for given parameters $\theta \in \theta$, connecting simulation inputs in $\mathbb{R}^p$ to mechanical behaviors in $\mathbb{R}^n$. The FDM establishes a forward map that generates diverse funicular geometries for variations of θ , like the ones in Fig. 1b. All of them are funicular, including $\mathbf{x}_C$. However, lightweight structural design works in the opposite direction of simulation, looking for parameters θ that produce a funicular geometry $\mathbf{x}$ satisfying a design specification. In this example, that specification is to match a target shape $\hat{\mathbf{x}}$ encoding architectural visual intent, but that is not in the space of funicular geometries, $\hat{\mathbf{x}} \notin \mathcal{X}$. Design hence poses an inverse problem, from behaviors in $\mathbb{R}^n$ to simulation inputs in $\mathbb{R}^p$.

guide the search for efficient and feasible forms to facilitate the design of lightweight structures, while still leveraging the mechanical efficiency guarantees provided by form-finding through the FDM.

1.3. Searching for efficient and feasible shapes with optimization

Optimization offers a principled approach to automate the search for specific funicular geometries in the space induced by the FDM. By converting a design specification into a scalar-valued *loss function*, an optimization algorithm can iteratively adjust the simulation parameters θ to minimize the loss value and converge towards a funicular geometry that satisfies design constraints [46, 52]. Optimization hence establishes an inverse map from funicular geometry to parameters $\mathbf{x} \rightarrow \theta$, but only approximately, where $\mathbf{x} \approx \hat{\mathbf{x}}$. The lower the loss value, the better the discovered geometry matches the specification (Fig. 1).

Gradient-free methods, like genetic algorithms and evolutionary strategies, appear frequently in structural optimization due to their versatility [53–60]. They only require evaluating the loss function for fitness evaluation, making them flexible to tackle general design specifications and loss function formulations [61]. However, gradient-free strategies scale poorly with simulation cost and problem dimensionality [62], and are thus most effective for small-scale problems. Gradient-based optimization offers computational advantages for faster, high-dimensional search. This approach leverages the *gradient* of the loss function, the direction of steepest ascent provided by the derivatives of the loss function with respect to θ , to guide the search towards optimal solutions in fewer computation steps [52,63]. Numerous precedents demonstrate the power of gradient-based optimization to solve complex design tasks for tall buildings [64–66], tensegrities [35,67], cable-stayed bridges [68], and shells [44,69]. Yet, this approach remains underutilized for general structural design due to its rigidity to accommodate change while triaging design options. The central barrier lies in deriving analytical expressions for the gradient of the loss function. This specialized, manual process is labor-intensive and must be repeated whenever a design specification is updated. The high human effort involved discourages adoption, and motivates the quest for automated derivative calculation approaches to make gradient-based design optimization for inverse form-finding not only more adaptable, but also more tenable for a wider group of researchers.

1.4. Computing exact derivatives automatically

Automatic differentiation (AD) is a family of algorithmic techniques that compute the derivatives of functions expressed as computer programs without manual derivation [70,71]. It is a cornerstone of differentiable programming [72]. The main idea behind AD is the representation of a function as the composition of elementary operations assembled into a computation graph [73,74]. Derivative

computation proceeds by walking this graph and applying the chain rule of calculus recursively. When the walk is completed from inputs to the outputs of the function, it is forward-mode AD. When performed in the opposite direction, from outputs to inputs, it is reverse-mode AD or *backpropagation* [75,76].

Compared to other computational differentiation techniques, like numerical or symbolic differentiation, AD is numerically cheaper and more versatile. Numerical differentiation, on the one hand, can only approximate the gradient and requires at least one function call per parameter to be differentiated, making each optimization step prohibitively expensive in large-scale problems [72,77]. Symbolic differentiation, on the other hand, provides a concrete mathematical gradient expression, but is limited to differentiating analytical functions and suffers from expression swell [34,78]. In contrast, AD provides exact gradients up to machine precision with a computational effort that is only a small multiple of the execution cost of the function being differentiated [70,72,79]. Due to its scalability, exactness, and flexibility, AD is the backbone of modern deep learning, where backpropagation feeds the gradients needed to train a wide array of neural networks to solve ambitious visual recognition or natural language processing tasks [78,80,81]. Besides neural network training, differentiable simulation is an additional domain where AD emerges as an instrumental tool to solve complex optimization problems governed by physics laws.

1.5. Differentiable simulation for fast structural design

Differentiable simulation extends the computational benefits of AD from data-driven to physics-based models like the FDM. A *differentiable simulation method* implements a physics-based model in an AD framework, so that the derivatives of the model outputs with respect to its inputs can be obtained automatically to accomplish gradient-based design optimization for a chosen loss function and problem parametrization [82]. Note that while these models are differentiable in a mathematical sense, their numerical implementation must be carefully structured in a computer program to be compatible with AD.

Differentiable simulation methods are propelling design optimization in various areas of engineering and the physical sciences, such as robotics [83–86], fluid and molecular dynamics [85,87,88], and metamaterials [89,90]. Structural design is no exception. Precedents in differentiable structural simulation methods have centered on topology optimization of planar domains [91–93] and 3D solids subject to stress and buckling constraints [94]. Related work has also proposed differentiable structural analysis frameworks for compliance reduction [95,96] and embodied-carbon minimization [97]. However, these approaches are limited to small-displacement, linear elastic models with beam and shell elements. Such models preclude large displacements, and necessitate topological triangulation (e.g., bracing) and explicit cross-section and material parameters to simulate a bar system [98]. Furthermore, linear elastic models fail to enforce target tensile or compressive force states a priori, a necessary feature to design lightweight structures such as cable nets (tensile state) or masonry shells (compressive state).

For the inverse form-finding of bar systems, differentiable variants of the thrust network analysis [99,100] and the combinatorial equilibrium modeling framework [77] exist. Nevertheless, they concentrate on specific structures, such as shells and layered trusses, and require careful preprocessing of a bar system's topology for simulation, which ultimately limits general applications. To improve generalization, prior works have demonstrated the integration of the FDM with AD [34,101,102]. But these efforts have focused on software implementations rather than on the mathematics behind both the FDM and AD. A tighter coupling between them by leveraging the abundant literature on analytical derivatives for the FDM [29,42] and by investigating how to infuse them into AD's backpropagation trace could be beneficial for fast structural optimization. Additionally, these FDM precedents have concentrated on optimizing the input force densities. This restriction narrows the funicular design space compared to that resulting from jointly optimizing loads and support conditions whenever possible. Such expanded design space exploration could enable the discovery of optimal structural shapes under stringent design specifications.

Whether focused on structural analysis or form-finding, a broader issue persists: current differentiable structural simulation methods only consider constant external loads [34,95–97,101,102]. While convenient for computation, as the simplification allows predicting structural behavior in only one simulation step, this assumption precludes accounting for loads that are intrinsically based on a structure's evolving geometry throughout simulation, and for which analytical approaches exist [42,103,104]. Such *shape-dependent loads* include self-weight, forces arising from wind or water action (i.e., fluid-structure interaction), or from the internal pressure needed to stiffen pneumatic systems (e.g., cushions or balloons). Developing differentiable form-finding methods that account for large-displacement effects and shape-dependent loads would close this gap and better support lightweight structural design in real-world settings.

1.6. Contributions

In this paper, we propose the differentiable force density method (DFDM), a differentiable form-finding method that leverages automatic and analytical differentiation to enable the computation of funicular geometries that fulfill architectural, manufacturing, and structural constraints via gradient-based optimization. As a result, the DFDM streamlines shape design for lightweight structures modeled as pin-jointed bar systems subject to shape-dependent loads. This paper makes three specific contributions to that end, extending the capabilities of the canonical FDM [28,29] and building upon preceding studies at the intersection between the FDM and AD [34,101,102]:

- **Differentiable mechanical model for structural design optimization.** We model the mechanical behavior of pin-jointed bar systems in the large-displacement regime. Unlike existing differentiable structural simulation methods, our differentiable model supports shape-dependent loads, such as self-weight and pressure, through an iterative equilibrium solver based on the updated

reference strategy [42] (Sections 2.1–2.4). Importantly, force equilibrium is preserved throughout optimization, not just at convergence, enhancing robustness for structural design (Section 2.5).

- **Accelerated gradient computation with analytical adjoints and automatic differentiation.** AD streamlines derivative calculation, but incurs computational overhead from repeated operations. We identify such bottlenecks in the DFDM and derive analytical vector-Jacobian products via implicit differentiation to resolve them. By injecting these specialized adjoints into general-purpose AD, we leverage the sparsity and fixed-point structure of the proposed mechanical model to compute gradients more rapidly while maintaining the flexibility of AD (Sections 2.6–2.8).
- **Extended parametrizations for optimal shape discovery in constrained tasks.** We introduce *load-finding* and *support-finding*, new parametrizations enabled by the DFDM's full differentiability. We prove their advantages in complex design tasks that feature multiple constraints, like repairing a masonry shell via external post-tensioning and devising a doubly-curved gridshell with flat panels. These tasks, in addition to shape-matching and length-control problems, highlight the DFDM's ability to supply exact gradients for all simulation parameters to minimize general loss functions for the discovery of efficient and feasible structural forms (Section 3).

We validate the DFDM's effectiveness and correctness by reproducing analytical and numerical experiments of past studies [31,105–108]. Together, these contributions establish a unified and extensible framework for gradient-based shape optimization of lightweight structures, addressing limitations of current structural simulation methods and better attuning form-finding for practical design tasks, particularly at the conceptual stage, where rapid iterations are key and design decisions have the highest impact on a structure's performance and cost [51,109].

1.7. Notation

This paper uses lowercase italics a to refer to scalars and functions, lowercase boldface letters $\mathbf{a} \in \mathbb{R}^n$ to indicate column vectors, and uppercase boldface letters $\mathbf{A} \in \mathbb{R}^{n \times m}$ to indicate matrices. Sets and collections are represented by uppercase italics, $\mathcal{A}$. Derivatives are denoted using ∂ , whether total or partial. When a variable depends on one input, the derivative should be interpreted in the total sense. The $\text{vec}(\mathbf{A})$ operator denotes the column-wise vectorization of matrix $\mathbf{A}$. We write the vector resulting from this reshaping operation by lowercasing the matrix, such that $\mathbf{a} = \text{vec}(\mathbf{A}) \in \mathbb{R}^{nm}$. Conversely, we reshape vector $\mathbf{a}$ into matrix $\mathbf{A}$ by filling the matrix columns in order, and represent the output matrix by uppercasing the vector symbol, $\mathbf{A} = \text{matrix}(\mathbf{a}) \in \mathbb{R}^{n \times m}$. When a vector is accompanied by a lowercase italic subscript, like $\mathbf{a}_i$, it indicates the i th row of $\mathbf{A}$. We employ roman subscripts to indicate variable categories, such as $a_{\max}$ or $\mathbf{a}_A$. The subscript $(\cdot)_0$ is reserved to denote the initial value of a variable.

2. Differentiable force density method

In this section, we present the DFDM and the main methodological contributions of this work. First, we introduce the form-finding problem (Section 2.1), and then explain the force density method's mechanical model from an energy perspective to demonstrate its consistency with structural mechanics (Section 2.2). Then, we present the extension of the FDM to general pin-jointed bar systems (Section 2.3 and 2.4), and introduce it as a differentiable model to compute funicular geometry that complies with constraints using gradient-based optimization (Sections 2.5 and 2.6). The formulation of the custom adjoints of a linear system solver and the updated reference strategy solver are developed thereafter (Sections 2.7 and 2.8). The Appendix presents the numerical validation of this work and a succinct DFDM implementation.

2.1. The form-finding problem

Consider a pin-jointed system with four bars described by position $\mathbf{x}_1 \in \mathbb{R}^3$, support positions $\mathbf{s}_j \in \mathbb{R}^3$; and internal forces $f_j \in \mathbb{R}$ (Fig. 2). The bars are connected to a rotation-free joint at $\mathbf{x}_1$, and they are translationally fixed to the ground at positions $\mathbf{s}_1$ to $\mathbf{s}_4$. A downward load $\mathbf{p}_1 \in \mathbb{R}^3$ acts at $\mathbf{x}_1$. For given f_j , the solution of the form-finding problem is to find the position of $\mathbf{x}_1$ that eliminates the residual force $\mathbf{r}_1 \in \mathbb{R}^3$:

$$\mathbf{r}_1(\mathbf{x}_1) = \sum_j \frac{f_j}{l_j} (\mathbf{x}_1 - \mathbf{s}_j) - \mathbf{p}_1 = \mathbf{0} \quad (1)$$

If the norm of $\mathbf{r}_1$ is zero, then the structure is in equilibrium. Note that the bar's length $l_j = \|\mathbf{x}_1 - \mathbf{s}_j\| \in \mathbb{R}$ is a nonlinear function of the unknown $\mathbf{x}_1$ since it involves calculating the norm of the coordinate differences between the endpoints of the bar. Eq. (1) can be solved more tractably by prescribing a ratio between the internal force and the length of the bars as inputs, rather than the internal forces:

$$\mathbf{r}_1(\mathbf{x}_1) = \sum_j q_j (\mathbf{x}_1 - \mathbf{s}_j) - \mathbf{p}_1 = \mathbf{0} \quad (2)$$

where the ratio $q_j \in \mathbb{R}$ is the force density of the j th bar, defined as:

$$q_j = \frac{f_j}{l_j} \quad (3)$$

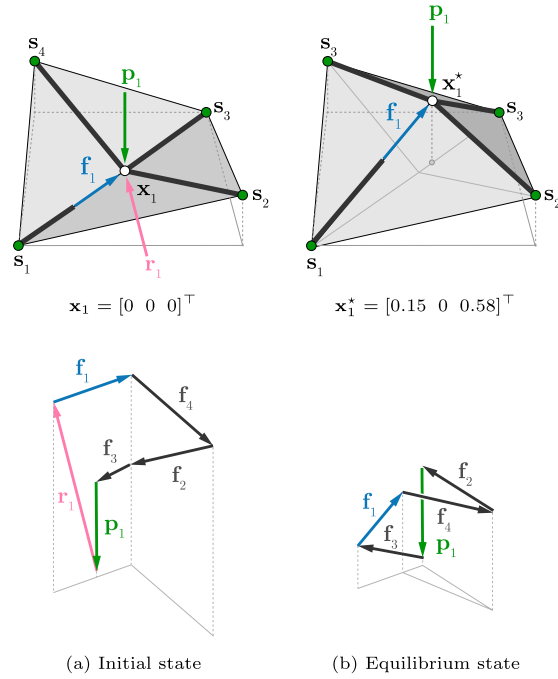


Fig. 2. The objective of solving the form-finding problem is to calculate the free 3D coordinates of a pin-jointed bar system, so that the residual forces vanish. The resulting geometry attains a state of equilibrium. For the 4-bar system subject to a load vector $\mathbf{p}_1$ in this figure, the initial configuration is not in equilibrium because the norm of the residual force vector $\mathbf{r}_1$ at position $\mathbf{x}_1$ is nonzero. Graphic statics [110,111] provides a visual approach to assess the system's lack of equilibrium as its corresponding force diagram (bottom) would be open in the absence of $\mathbf{r}_1$. The bar system achieves equilibrium only after translating $\mathbf{x}_1$ to $\mathbf{x}_1^*$, making the system funicular, as evidenced by the closed force diagram beneath it.

We can solve Eq. (2) with Newton's method. The corresponding Newton step $\Delta \mathbf{x}$ reads:

$$\frac{\partial \mathbf{r}_1}{\partial \mathbf{x}_1} \Delta \mathbf{x} = -\mathbf{r}_1 \quad (4)$$

Due to the linearity of $\mathbf{r}_1$ with respect to $\mathbf{x}_1$, we find a root of Eq. (2) in a single step:

$$\mathbf{x}_1^* = \mathbf{x}_1 - \mathbf{K}^{-1} \mathbf{r}_1 \quad (5)$$

where $\mathbf{x}_1$ denotes the position of the free vertex in the initial configuration shown in Fig. 2a, and $\mathbf{K} \in \mathbb{R}^{3 \times 3}$ is the Jacobian $\partial \mathbf{r}_1 / \partial \mathbf{x}_1$, the stiffness matrix of the system:

$$\mathbf{K} = \begin{bmatrix} \sum_j q_j & 0 & 0 \\ 0 & \sum_j q_j & 0 \\ 0 & 0 & \sum_j q_j \end{bmatrix} \quad (6)$$

If we expand Eq. (5), we arrive at an analytical expression to solve the form-finding problem:

$$\mathbf{x}_1^* = \frac{\sum_j q_j \mathbf{s}_j + \mathbf{p}_1}{\sum_j q_j} \quad (7)$$

This expression provides key insight: the equilibrium solution $\mathbf{x}_1^*$, shown in Fig. 2b, is independent of $\mathbf{x}_1$. This feature avoids potential numerical divergences in calculating a root of the residual due to a poor initial guess of $\mathbf{x}_1$ as long as $\mathbf{K}$ is invertible (e.g., none of its diagonal entries are zero, $\sum_j q_j \neq 0$). The use of force densities as inputs to solve form-finding problems and the extension of Eq. (2) to general pin-jointed bar systems are the foundations of the FDM, which we present in more detail in Section 2.3.

2.2. Mechanical model

Before exposing the general FDM, we take a step back and motivate it from an energy standpoint, complementing the traditional force balance approach presented in Section 2.1. This viewpoint is necessary to explain the FDM's mechanical model for geometrically nonlinear analysis based on structural mechanics, which is one where a cable net, or a pin-jointed bar system in the general case, is subject to prestressing forces and large displacements. Our exposition consolidates ideas from previous studies [42,43,112].

We consider one of the bars in the structure shown in Fig. 2a and apply an axial prestressing force f to it, while disregarding elastic deformations. In its initial configuration, the internal energy π_{int} of the bar is the product of the second Piola-Kirchhoff force

f_{PK2} and the Green-Lagrange strain ϵ_{GL} , integrated over the bar's initial length l_0 :

$$\pi_{\text{int}} = \int_0^{l_0} f_{PK2} \epsilon_{GL} dl = \frac{f_{PK2}}{2l_0} (l^2 - l_0^2) \quad (8)$$

Unlike a first-order strain, the geometrically nonlinear strain ϵ_{GL} accounts for large displacements and ensures mechanical compatibility with the bar's deformed length l :

$$\epsilon_{GL} = \frac{l^2 - l_0^2}{2l_0^2} \quad (9)$$

Following the principle of minimum potential energy, an equilibrium state for the bar system shown in Fig. 2 can be found by minimizing the total energy of the system:

$$\Pi = \Pi_{\text{int}} + \Pi_{\text{ext}} \rightarrow \min \quad (10)$$

where Π_{int} is the sum of the individual bar's energy, and Π_{ext} is the external energy accrued by the applied load $\mathbf{p}_1$. After discretizing the bars, a stationary point of Eq. (10) is obtained by equating to zero the gradient (i.e., the first variation) of the energy for a virtual displacement $\delta \mathbf{x}_1$:

$$\nabla_{\mathbf{x}_1} \Pi = \sum_j \nabla_{\mathbf{x}_1} \pi_{\text{int}} + \nabla_{\mathbf{x}_1} \Pi_{\text{ext}} = \sum_j \left(\frac{f_{PK2} l}{l_0} \right)_j \frac{\partial l_j}{\partial \mathbf{x}_1} \delta \mathbf{x}_1 - \mathbf{p}_1 \delta \mathbf{x}_1 = \mathbf{0} \quad (11)$$

The position $\mathbf{x}_1$ encodes the free vertex coordinates, and $(\cdot)_j$ denotes the quantities associated with the j th bar. The derivative of the length l_j with respect to $\mathbf{x}_1$ is the ratio, post-deformation, of the normalized difference between the bar's end positions to the bar's length:

$$\frac{\partial l_j}{\partial \mathbf{x}_1} = \frac{\mathbf{x}_1 - \mathbf{s}_j}{l_j} \quad (12)$$

where $\mathbf{s}_j$ denotes the position of the fixed end of the j th bar. Once we substitute Eq. (12) into Eq. (11) and impose that the first variation of the energy holds for all compatible virtual displacements $\delta \mathbf{x}_1$, we obtain the equilibrium condition:

$$\nabla_{\mathbf{x}_1} \Pi = \sum_j \left(\frac{f_{PK2}}{l_0} \right)_j (\mathbf{x}_1 - \mathbf{s}_j) - \mathbf{p}_1 = \mathbf{0} \quad (13)$$

Note that the lengths in the deformed configuration l_j , a source of nonlinearity in the energy, cancel out algebraically and thus simplify the above expression. Eq. (13) is equivalent to the nonlinear form-finding problem in Eq. (1), and is singular for co-planar force vectors, or more generally, for tangential displacements (e.g., when three vertices in a structure are co-planar) [42]. If we reparameterize the equation by defining a constant ratio between the second Piola-Kirchhoff force and the undeformed length, such that

$$q = \frac{f_{PK2}}{l_0} \quad (14)$$

we linearize the variation of the energy functional, overcoming singularities that arise in the original formulation, and obtain once again the force density q that underpins the FDM. The solution of Eq. (13) after this reparametrization yields a funicular geometry in equilibrium, as depicted in Fig. 2b.

The structural mechanics treatment of the FDM explains why a pin-jointed bar system is able to reach equilibrium even when large displacements are required, while maintaining mechanical congruency in terms of prestressing forces and nonlinear strains. Viewed from an energy minimization perspective, the gradient of the potential energy $\nabla_{\mathbf{x}_1} \Pi$ (Eq. (11)) corresponds to the residual force $\mathbf{r}_1$ (Eq. (2)) in the bar system shown in Fig. 2, which is the central measure of equilibrium of structures in general. Moreover, the Hessian of this energy, $\nabla_{\mathbf{x}_1}^2 \Pi$, yields the stiffness of the structure, $\mathbf{K}$, which is equivalent to the Jacobian of the residual $\mathbf{r}_1$. Both observations reveal the close relationship between the FDM, other form-finding methods, and the broader field of structural mechanics, as pointed out by previous works [42,112].

2.3. The force density method

Equipped with an understanding of the mechanical model that the FDM builds upon, we now expand it to 3D structures. To that end, we provide a concise overview of how to calculate a funicular geometry $\mathbf{x} \in \mathbb{R}^{3n_u}$ using the FDM for given parameters $\theta \in \mathbb{R}^{n_p}$ defined on a graph G . Fig. 3 summarizes the inputs and outputs. The presentation of the FDM in this work employs vector and matrix operations, i.e., linear algebra, following the original treatment of Schek [29].

2.3.1. Data representation

We represent the topology of a structure modeled as a pin-jointed bar system as a graph G with n_v vertices and n_e edges. The vertices encode the rotation-free joints, and the edges capture the bars linked at these joints. There is only one vertex v_i per joint,

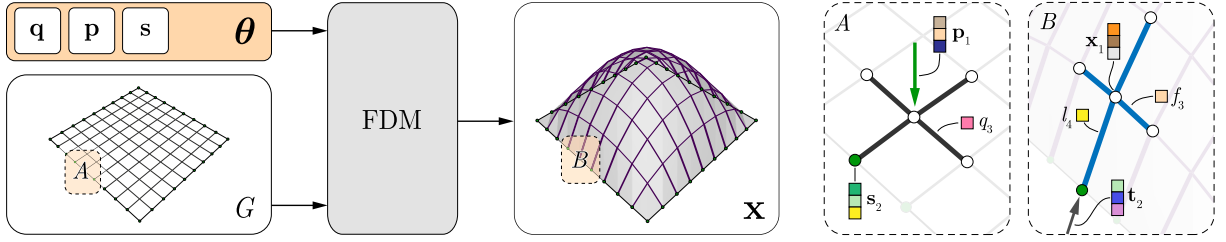


Fig. 3. The force density method (FDM) computes a funicular geometry $\mathbf{x}$ for a pin-jointed bar system parametrized by θ and whose topology is encoded by a graph G . The parameters encompass the force densities $\mathbf{q}$, the applied loads $\mathbf{p}$, and the support positions $\mathbf{s}$. The shaded regions A and B further detail the parameters associated with the edges and vertices of G . In A , we illustrate the FDM inputs: the force density q_3 of edge 3, the position $\mathbf{s}_2$ of support vertex 2, and the load $\mathbf{p}_1$ at vertex 1. In B , the close-up reveals the FDM outputs: the equilibrium position $\mathbf{x}_1$, the internal force f_3 of edge 3, the length l_4 of edge 4, and the reaction $\mathbf{t}_2$ at support vertex 2.

and only one edge e_j per bar. For simulation, the topology of G is embedded in a connectivity matrix $\tilde{\mathbf{C}} \in \{-1, 0, 1\}^{n_e \times n_v}$, where the (ij) th entry c_{ij} is defined as:

$$c_{ij} = \begin{cases} 1 & \text{if } i \text{ is the start vertex of edge } j \\ -1 & \text{if } i \text{ is the end vertex of edge } j \\ 0 & \text{otherwise} \end{cases} \quad (15)$$

While $\tilde{\mathbf{C}}$ presupposes that the edges of G have an orientation, the geometry generated by the FDM is invariant to it. Therefore, the orientation of the edges can be chosen arbitrarily as long as there exists only one edge connecting any two vertices.

For 3D structures, the connectivity matrix is expanded to a block diagonal matrix $\mathbf{C} \in \{-1, 0, 1\}^{3n_e \times 3n_v}$ with as many blocks as the number of Cartesian dimensions [113]:

$$\mathbf{C} = \mathbf{I}_3 \otimes \tilde{\mathbf{C}} \quad (16)$$

where $\mathbf{I}_3 \in \mathbb{R}^{3 \times 3}$ is an identity matrix for the 3D case, and $\otimes$ denotes the Kronecker product [114].

2.3.2. Boundary conditions

To impose boundary conditions, we choose a subset of the graph's vertices of size n_s to be fixed with respect to translations. The *fixed vertices* represent the support locations where the structure is anchored to the ground, and where reaction forces are transferred to the structure's foundations. We assume the fixed vertices are restrained in all three dimensions. The remaining n_u vertices are the *free vertices*. It is at these vertices that a funicular geometry is defined. The effect of imposing boundary conditions is manifested in the connectivity $\mathbf{C}$, which is now split along its columns into submatrices $\mathbf{C}_u$ and $\mathbf{C}_s$ that correspond to the degrees of freedom of the free and fixed vertices, respectively:

$$\mathbf{C} = [\mathbf{C}_u \quad \mathbf{C}_s] \quad (17)$$

In general, each column of $\mathbf{C}$ corresponds to a degree of freedom of a vertex along one Cartesian dimension as per Eq. (16). While we work with fixed vertices restricted in all three dimensions, the partitioning of $\mathbf{C}$ can be performed more granularly to selectively restrain a vertex in one, two, or three dimensions, and thus increase the generalizability of our method. Such partitioning consists of splitting each connectivity block $\tilde{\mathbf{C}}$ into free and fixed submatrices, such that $\tilde{\mathbf{C}} = [\tilde{\mathbf{C}}_u \quad \tilde{\mathbf{C}}_s]$. All resulting submatrices are then assembled in $\mathbf{C}$:

$$\mathbf{C} = \begin{bmatrix} [\tilde{\mathbf{C}}_u \quad \tilde{\mathbf{C}}_s]_x & 0 & 0 \\ 0 & [\tilde{\mathbf{C}}_u \quad \tilde{\mathbf{C}}_s]_y & 0 \\ 0 & 0 & [\tilde{\mathbf{C}}_u \quad \tilde{\mathbf{C}}_s]_z \end{bmatrix} \quad (18)$$

where the subscripts x, y, z denote a separate connectivity block per Cartesian dimension. Funicular geometry computation then proceeds by selecting the columns of the free and fixed degrees of freedom as Eq. (17) indicates for compatibility with Eqs. (20) and (22). Note that when imposing translational fixity along specific dimensions, it would be more precise to distinguish between *free degrees of freedom* and *fixed degrees of freedom* rather than free and fixed vertices, the convention we adopt in this work.

2.3.3. Simulation parameters

The FDM is parameterized by three groups of attributes defined on the vertices and edges of G :

- The force densities, $\mathbf{q} \in \mathbb{R}^{n_e}$: a vector of force densities establishing the ratio between the axial force f and length l per bar, $q \in \mathbb{R}$. The sign of q imposes the direction of the axial force in the bars when the bar system is in equilibrium. A negative q prescribes a compressive axial force, whereas a positive q dictates a tensile axial force.
- The applied loads, $\mathbf{p} \in \mathbb{R}^{3n_u}$: the static external forces exerted on the free vertices. Because the structure is represented as a bar system, the loads cannot be applied on the edges, only at the vertices.

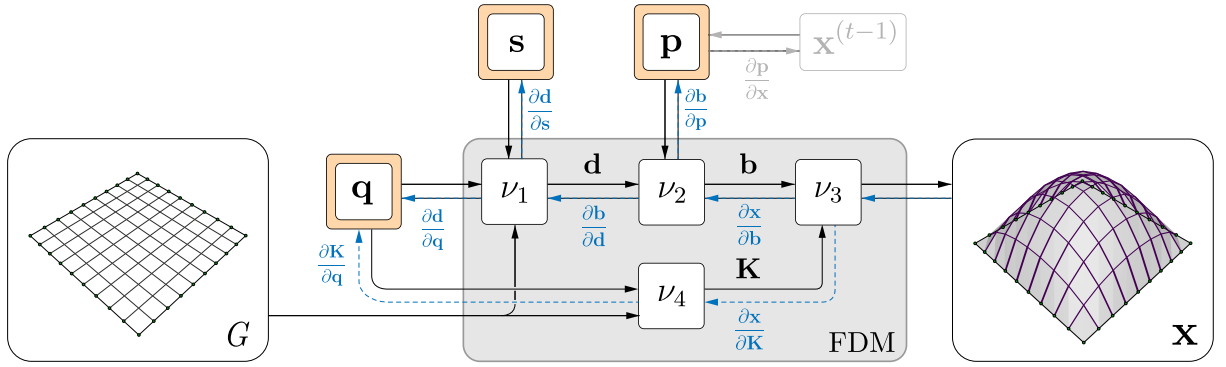


Fig. 4. Computation graph of one FDM step. This graph provides a granular view of the principal numerical operations, or primitives ν_i , at the linear algebra level involved in computing a funicular geometry $\mathbf{x}$ for given θ and G , during both forward simulation and derivative backpropagation. For completeness, we also indicate the dependency of the loads $\mathbf{p}$ on a preceding geometry $\mathbf{x}^{(t-1)}$ when multiple FDM steps are involved in solving for equilibrium under shape-dependent loads. Reverse-mode AD and analytical adjoints provide the derivatives of $\mathbf{x}$ with respect to θ . The backpropagation trace is indicated by the dashed arrows.

- The support positions, $\mathbf{s} \in \mathbb{R}^{3n_s}$: the 3D coordinates of the fixed vertices; the boundary conditions.

For optimization, we can flatten and concatenate the inputs in the parameter vector $\theta^T = [\mathbf{q}^T \ \mathbf{p}^T \ \mathbf{s}^T] \in \mathbb{R}^{n_p}$. The total number of parameters n_p input to the FDM is $n_p = n_e + 3n_u + 3n_s$.

2.3.4. Computation of funicular geometry

A funicular geometry $\mathbf{x}$, defined as $\mathbf{x}^T = [\mathbf{x}_1^T \ \dots \ \mathbf{x}_{n_u}^T] \in \mathbb{R}^{3n_u}$, sustains external loads entirely through axial forces. This property results from finding a state of equilibrium in the system's deformed configuration. As exposed in Section 2.2, equilibrium is reached when the energy Π is at a minimum, which is equivalent to the residual forces $\mathbf{r}^T = [\mathbf{r}_1^T \ \dots \ \mathbf{r}_{n_u}^T] \in \mathbb{R}^{3n_u}$ on the free vertices having zero norm:

$$\mathbf{r}(\mathbf{x}, \theta) = \mathbf{K}(\theta)\mathbf{x} - \mathbf{b}(\theta) = \mathbf{0} \quad (19)$$

Here, $\mathbf{K} \in \mathbb{R}^{3n_u \times 3n_u}$ is the stiffness matrix of the system [18,113]:

$$\mathbf{K}(\theta) = \mathbf{C}_u^T \mathbf{Q} \mathbf{C}_u \quad (20)$$

where $\mathbf{Q} \in \mathbb{R}^{3n_e \times 3n_e}$ is a block diagonal matrix with the force densities, $\mathbf{Q} = \mathbf{I}_3 \otimes \text{diag}(\mathbf{q})$. The effective load vector $\mathbf{b} \in \mathbb{R}^{3n_u}$ deducts the contribution of the edge forces connected to the supports, encoded in $\mathbf{d} \in \mathbb{R}^{3n_u}$, from the loads $\mathbf{p}$:

$$\mathbf{b}(\theta) = \mathbf{p} - \mathbf{d}(\theta) \quad (21)$$

$$\mathbf{d}(\theta) = \mathbf{C}_u^T \mathbf{Q} \mathbf{C}_s \mathbf{s} \quad (22)$$

Since the stiffness and the loads are independent of the geometry, the computation of $\mathbf{x}$ for given θ resolves to the solution of a linear system:

$$\mathbf{x}(\theta) = \mathbf{K}(\theta)^{-1} \mathbf{b}(\theta) \quad (23)$$

The resulting $\mathbf{x}$ is a funicular geometry defined by the 3D coordinates of the free vertices in equilibrium. Eq. 23 can be solved iteratively via conjugate gradient methods [115], or directly via matrix factorization approaches like the Cholesky factorization. A bar system with mixed positive and negative force densities might result in zeroes along the main diagonal of $\mathbf{K}$, rendering it singular and potentially introducing numerical challenges in the form-finding of trusses and tensegrity systems. The solution of Eq. (23) in the presence of such singular systems can be handled by regularizing $\mathbf{K}$ by adding a scaled version of the outer product of its rigid-body eigenvectors (i.e., the eigenvectors spanning the nullspace of $\mathbf{K}$) [112,116]. However, the application of these regularization techniques is beyond the scope of this work.

2.3.5. Completing the equilibrium state

Once a funicular geometry $\mathbf{x}$ is obtained with the FDM, the remaining features that complete the equilibrium state of the system can be calculated. They comprise:

- The edge lengths, $\mathbf{l} \in \mathbb{R}^{n_e}$: the length of the bars after global displacements and local deformations.
- The edge forces, $\mathbf{f} \in \mathbb{R}^{n_e}$: the magnitude of the internal axial forces of the bars, tensile or compressive, based on the signs of $\mathbf{q}$. In structural mechanics, these forces correspond to the Cauchy forces in the elongated bars, considering Poisson effects.
- The support reactions, $\mathbf{t} \in \mathbb{R}^{3n_s}$: the force vectors at the supports, i.e., the forces that the structure will exert on its foundations.

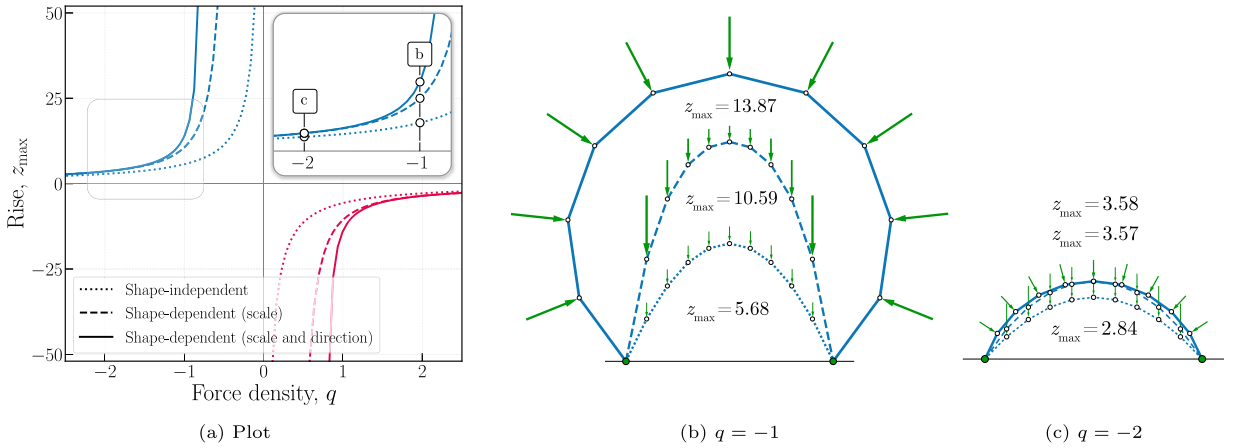


Fig. 5. Changes on the rise $z_{\max}$ of a planar arch versus variations of the force density q , for shape-independent and shape-dependent loads. The two extreme vertices of the structure are fixed, and the rest are free to translate. The magnitude of the starting vertical load applied to the free vertices is $-1/2$. Fig. 5a illustrates that the relationship between q and $z_{\max}$ is nonlinear and shows asymptotic behavior in the vicinity of $q = 0$ for all three loading conditions. Fig. 5b and c demonstrate that a unit change in q , from -1 to -2 , translates into variations in the rise $z_{\max}$ of several units of magnitude. The structures subject to shape-independent loads are shallower than those whose loads are shape-dependent, especially for low q .

The above attributes are computed with:

$$l(\mathbf{x}, \theta) = \sqrt{(\mathbf{U} \odot \mathbf{U}) \mathbf{1}} \quad (24)$$

$$\mathbf{f}(\mathbf{x}, \theta) = \mathbf{q} \odot l \quad (25)$$

$$\mathbf{t}(\mathbf{x}, \theta) = \mathbf{p}_s - \mathbf{C}_s^T \mathbf{Q} \mathbf{u} \quad (26)$$

where $\mathbf{u} = \mathbf{C}_u \mathbf{x} + \mathbf{C}_s \mathbf{s} \in \mathbb{R}^{3n_e}$ and $\mathbf{U} = \text{matrix}(\mathbf{u}) \in \mathbb{R}^{n_e \times 3}$ contain the edge coordinate differences, $\mathbf{1} \in \mathbb{R}^3$ is a vector of ones, $\mathbf{p}_s \in \mathbb{R}^{3n_s}$ are the loads at the supports, if any, and $\odot$ denotes the Hadamard product [114].

2.4. Equilibrium under shape-dependent loads

The FDM generates funicular geometry for constant loads with Eq. (23). The immutability of the loads, however, often fails to capture the mechanical behavior of a structure when the loads themselves depend on its form. That is because, in form-finding, the bar system has to deform under the applied loads to reach equilibrium, and this deformation is, in turn, dependent on the loads. A crucial example of such a *shape-dependent load* is self-weight, generally considered the primary driver of form in the conceptual design of lightweight structures [2]. While an initial estimate of the self-weight can be used to compute a funicular geometry, these loads are seldom updated to reflect changes in shape and mass distribution as the bar system deforms. Fig. 5 illustrates with a simple example the consequences of not updating the loads with the deformation: the height of a funicular arch under constant loads is approximately half of the height of the true equilibrium geometry under its updated weight. Consequently, failing to update the loads can raise questions about the accuracy of the generated geometry, ultimately leading to unrealistic designs.

To overcome these challenges, we extend the FDM to support the computation of geometries under shape-dependent actions. We achieve this by generating a funicular geometry iteratively, in line with other form-finding works [27,117]. The dependency of the loads $\mathbf{p}$ on the geometry $\mathbf{x}$ shows up in $\mathbf{b}$ in Eq. (19):

$$\mathbf{r}(\mathbf{x}, \theta) = \mathbf{K}(\theta) \mathbf{x} - \mathbf{b}(\mathbf{x}, \theta) = \mathbf{0} \quad \text{where} \quad \mathbf{b}(\mathbf{x}, \theta) = \mathbf{p}(\mathbf{x}, \theta) - \mathbf{d}(\theta) \quad (27)$$

Instead of finding a root of Eq. (27) directly, we decompose the task into a sequence of linear equilibrium problems by applying the updated reference strategy (URS) [6]. The URS is a general framework for solving nonlinear form-finding problems by iteratively updating the geometry of a bar system until arriving at a fixed point of the equilibrium constraint. The benefit of this strategy is not only its fast convergence, but that the equilibrium of the system is ensured per iteration (Fig. 8). Unlike the original URS, however, we keep the force densities unchanged and only alter the loads.

At each iteration $t + 1$, we recalculate the applied loads $\mathbf{p}^{(t+1)}$ based on the last geometry $\mathbf{x}^{(t)}$, and update $\mathbf{b}$ using Eq. (21). The relationship between $\mathbf{p}$ and $\mathbf{x}$ is, in principle, arbitrary and might not even be directly defined on the vertices. We focus on loads that exhibit two types of shape dependency: scale, and both scale and direction. The latter type is also known as a *follower load*, which is necessary to simulate inflatable and air-supported structures. We show design examples of both load types in Sections 3.2 and 3.4.

To deal with follower loads, we utilize a discrete load projection from face-based area loads to vertex-based loads [108,118], using tributary areas that depend on the bar system's geometry. A face is a polygonal surface patch whose boundary is a topological cycle with three or more edges. The positions of the face's boundary vertices constitute a closed polygon in $\mathbb{R}^3$ [119]. We compute non-overlapping faces on an instance of G using a wall-follower, graph traversal algorithm [120].

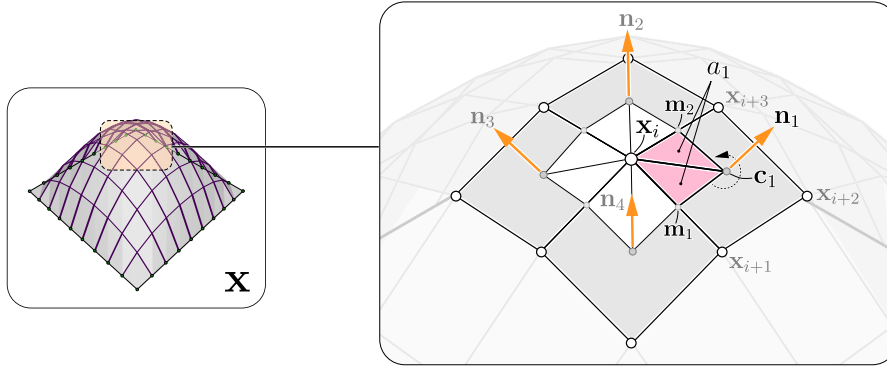


Fig. 6. Tributary area computation to generate shape-dependent loads. At each form-finding iteration t , the area a_k of face k attributed to vertex i is the sum of the areas of the two incident triangles $(\mathbf{x}_i, \mathbf{m}_j, \mathbf{c}_k)$, where $\mathbf{x}_i$ is the vertex position, $\mathbf{m}_j$ are the midpoints of the two edges j shared by k and i , and $\mathbf{c}_k$ is the face centroid. In this example, the four faces connected to vertex i are shaded in gray, and the value of a_1 for the first face equals the sum of the two pink triangle areas. The white triangles complete the tributary areas of the other three faces incident to the vertex. The figure also indicates the face normal vectors $\mathbf{n}_k$, produced after ordering the face vertices counterclockwise with respect to the load direction.

The shape-dependent load $\mathbf{p}_i^{(t+1)}$ of vertex i at iteration $t + 1$ is a function of the geometry of its neighborhood at iteration t :

$$\mathbf{p}_i^{(t+1)} = \sum_{k \in F(i)} a_k^{(t)} \boldsymbol{\psi}_k^{(t)} \quad \text{where} \quad \boldsymbol{\psi}_k^{(t)} = \psi_k \mathbf{n}_k^{(t)} \quad (28)$$

Here, $F(i)$ corresponds to the faces incident to vertex i ; $a_k \in \mathbb{R}$ is the tributary area of the face k ; $\psi_k \in \mathbb{R}$ is the load magnitude on face k , and $\mathbf{n}_k \in \mathbb{R}^3$ is its unit normal vector. We get $\mathbf{n}_k$ with the shoelace formula:

$$\mathbf{n}_k^{(t)} = \frac{\tilde{\mathbf{n}}_k^{(t)}}{\|\tilde{\mathbf{n}}_k^{(t)}\|} \quad \text{where} \quad \tilde{\mathbf{n}}_k^{(t)} = \frac{1}{2} \sum_{i=1}^{n_k} \mathbf{x}_i^{(t)} \times \mathbf{x}_{i+1}^{(t)} \quad (29)$$

with the closure condition that $\mathbf{x}_{n_k+1} = \mathbf{x}_1$. In the above equation, n_k is the number of vertices per face, and $\mathbf{x}_i$ are the 3D coordinates of the face vertices ordered counterclockwise relative to the application direction of the load. When the load is not a follower load, and it is rather of interest to scale the load magnitude based on the tributary face areas of a vertex, we replace $\mathbf{n}_k$ with an arbitrary unit vector in $\mathbb{R}^3$. In the case of self-weight, we set this vector to the vertical downward direction. We estimate the tributary area a_k of face k as the sum of the areas of the triangle pair connecting vertex i to the face centroid $\mathbf{c}_k$ and to the midpoint $\mathbf{m}_j$ of the two boundary edges of k meeting at i . Fig. 6 exemplifies this calculation. The centroid $\mathbf{c}_k$ at iteration t is obtained by averaging the current position $\mathbf{x}_i$ of the face vertices:

$$\mathbf{c}_k^{(t)} = \frac{1}{n_k} \sum_{i=1}^{n_k} \mathbf{x}_i^{(t)} \quad (30)$$

Similarly, the midpoint $\mathbf{m}_j$ of edge j with endpoints at $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$ is the average of their positions:

$$\mathbf{m}_j^{(t)} = \frac{\mathbf{x}_i^{(t)} + \mathbf{x}_{i+1}^{(t)}}{2} \quad (31)$$

Once the loads are regenerated, we apply the FDM to find the next funicular geometry $\mathbf{x}^{(t+1)}$ via Eq. (23):

$$\mathbf{x}^{(t+1)}(\mathbf{x}^{(t)}, \boldsymbol{\theta}) = \mathbf{K}(\boldsymbol{\theta})^{-1} \mathbf{b}(\mathbf{x}^{(t)}, \boldsymbol{\theta}) \quad (32)$$

We repeat the process until convergence. Hence, a funicular geometry in equilibrium with shape-dependent loads results from a sequence of geometries that starts at $\mathbf{x}^{(t)}$ and converges to a fixed point after τ iterations:

$$\mathbf{x} = \mathbf{x}^{(\tau)}(\mathbf{x}^{(\tau-1)}(\dots(\mathbf{x}^{(t+1)}(\mathbf{x}^{(t)}, \boldsymbol{\theta}), \boldsymbol{\theta}), \boldsymbol{\theta}), \boldsymbol{\theta}) \quad (33)$$

The URS converges when the mean pairwise difference $\Delta \mathbf{x}$ between the 3D coordinates $\mathbf{x}_i$ of all the free vertices at two consecutive iterations t and $t + 1$

$$\Delta \mathbf{x} = \sum_{i=1}^{n_u} \|\mathbf{x}_i^{(t+1)} - \mathbf{x}_i^{(t)}\| \quad (34)$$

is smaller than or equal to a convergence threshold η . That is, $\Delta \mathbf{x} \leq \eta$. Fig. 7 shows a schematic of the recursive computation of a funicular geometry in a computation graph, and Fig. 8 illustrates the URS convergence for both scale and scale-and-direction dependency on a cable net.

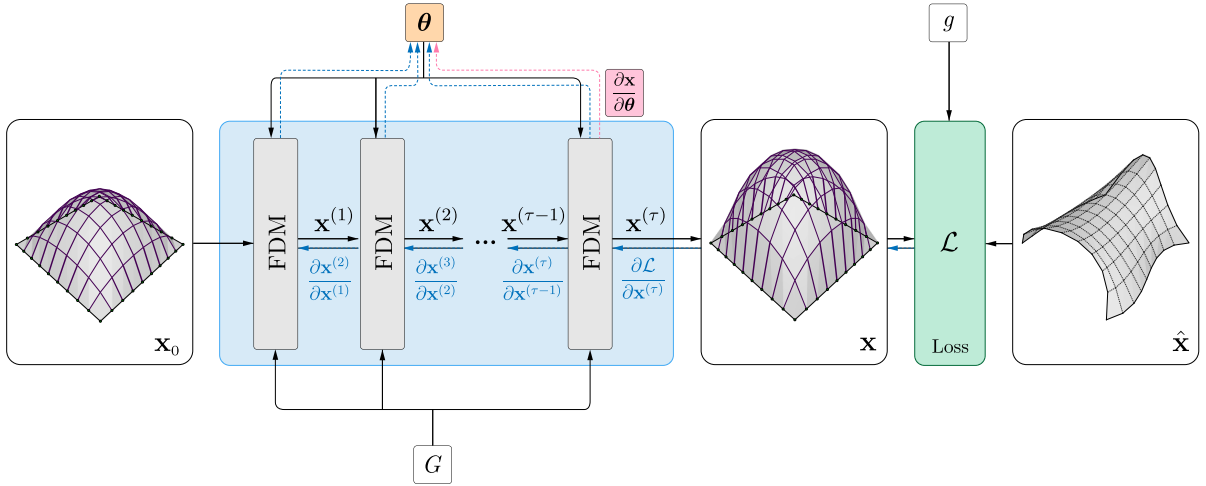


Fig. 7. Computation graph of the FDM under shape-dependent loads. A funicular geometry $\mathbf{x}$ is obtained by iteratively updating both the geometry and loads from an initial guess $\mathbf{x}_0$ until convergence to a fixed point at iteration τ , while θ and G remain constant. Note the changes between the initial and converged geometries. For design, we formulate a loss function $\mathcal{L}$ containing goals g that measures the difference between $\mathbf{x}$ and a target $\hat{\mathbf{x}}$ representing a design specification, and then minimize the loss value $\mathcal{L}(\theta)$ with gradient-based optimization. The gradient value $\nabla_{\theta} \mathcal{L}(\theta)$ is computed with reverse-mode AD, and implicit differentiation bypasses the full iteration history, backpropagating only through the last FDM step via $\partial \mathbf{x} / \partial \theta$ in the diagram.

2.5. Solving inverse form-finding problems with gradient-based optimization

This work aims to automate the generation of efficient shapes for lightweight structures that satisfy architectural, fabrication, and structural constraints. Finding such feasible and efficient shapes requires solving an inverse form-finding problem since the space of funicular geometries $\mathcal{X}$ is inaccessible directly. This space must be explored indirectly through variations of parameters θ , after executing a forward structural simulation, herein provided by the FDM. Navigating the design space spanned by θ to solve such inverse problems is challenging due to a lack of interpretability and the high dimensionality of this space; in addition to the nonlinearities in the mechanical behavior of a pin-jointed bar system (Fig. 5).

To tackle these challenges, we solve inverse form-finding problems with gradient-based optimization. This approach involves repeatedly executing the FDM in the inner loop of an iterative optimization algorithm to minimize a loss function. Our choice is motivated by the proven success of gradient-based methods in navigating high-dimensional, nonlinear performance spaces to optimize physical systems towards target behaviors [90,121–123]. Gradient-based optimization methods are also well suited to under- or over-constrained problems, frequent in lightweight structural design, where the number of target conditions differs from the available degrees of freedom to optimize [113,124–126].

For a given design task, we formulate a scalar-valued loss function $\mathcal{L}$ in terms of the form-finding parameters θ and the funicular geometry $\mathbf{x}$, following a penalty approach. The loss function measures the fit of $\mathbf{x}$ to a design specification:

$$\mathcal{L}(\theta, \mathbf{x}) = \sum_{i=1}^{n_g} \alpha_i g_i(\mathbf{x}(\theta)) \quad (35)$$

where g_i is a differentiable, generally nonlinear, goal function that computes a property of interest on $\mathbf{x}$ that should be minimized. One design specification may contain n_g different goal functions whose relative importance to the task is controlled by a nonnegative weight factor α_i . The general formulation of $\mathcal{L}$ facilitates the composition of diverse g_i to define custom specifications, as we illustrate in Section 3. The specific goals might nonetheless affect the convergence rate of the optimization, particularly for conflicting goals where improvement on one goal deteriorates performance on another, in which case the optimal solution would yield a trade-off between goals. Since $\mathbf{x}(\theta)$ is an equilibrium state variable that depends on θ through the FDM, the loss value $\mathcal{L}$ is effectively only a function of θ . Therefore, we work with the reduced objective and write $\mathcal{L}(\theta) = \mathcal{L}(\theta, \mathbf{x}(\theta))$ hereafter for notational simplicity.

To solve an inverse form-finding problem, we look for an optimal θ^* that minimizes $\mathcal{L}(\theta)$ and thus produces a funicular geometry $\mathbf{x}^*$ sufficiently close to the design specification. We obtain θ^* as the solution of a constrained optimization problem:

$$\begin{aligned} \min_{\theta} \quad & \mathcal{L}(\theta) \\ \text{s.t.} \quad & h_i(\theta) = 0 \quad \forall i = 1, \dots, n_h \\ & m_i(\theta) \leq 0 \quad \forall i = 1, \dots, n_m \\ & \theta_{lb} \leq \theta_i \leq \theta_{ub} \quad \forall i = 1, \dots, n_p \\ \text{with} \quad & \mathbf{r}(\theta) = \mathbf{0} \end{aligned} \quad (36)$$

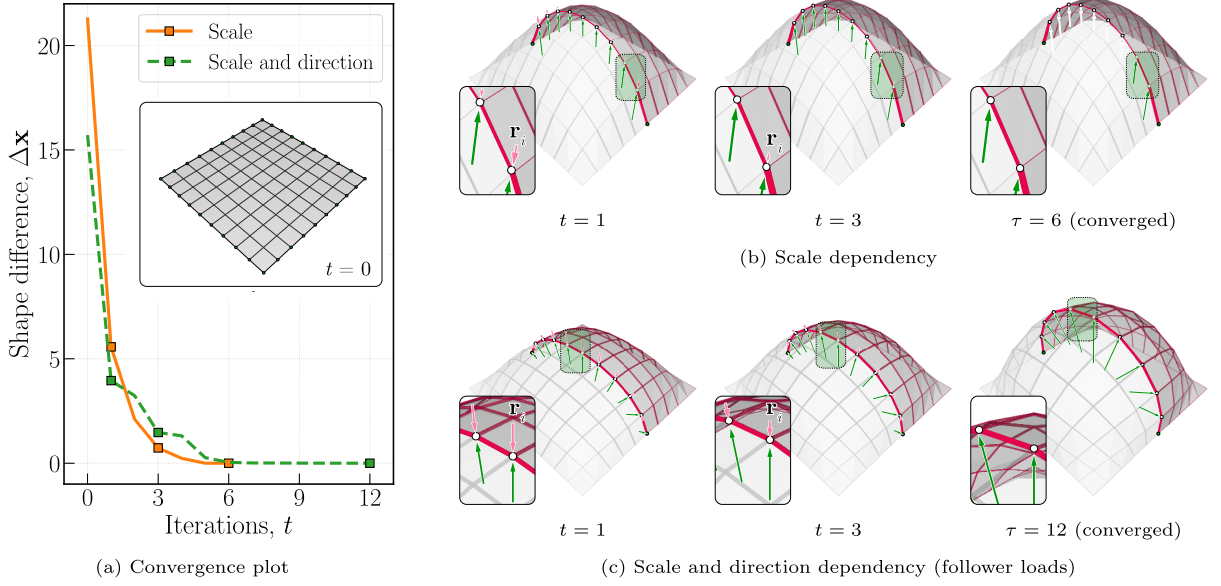


Fig. 8. Equilibrium convergence under shape-dependent loads. We minimize the pairwise difference between two subsequent versions of $\mathbf{x}$ with the URS, repeatedly solving Eq. (32) until we find a fixed point. Here, the process initiates from a flat grid at $t=0$ and we visualize a few iterations to illustrate the progression. At each iteration, the callout shows the load $\mathbf{p}_i$ at vertex i and the residual $\mathbf{r}_i$ computed from the position $\mathbf{x}_i$ at the previous iteration. Equilibrium is reached in $\tau=6$ iterations when only the load scale is shape-dependent, and in $\tau=12$ iterations when both scale and direction vary with the geometry.

This problem can be subject to n_h equality constraints h , n_m inequality constraints m , and n_p lower θ_{lb} and upper θ_{ub} bounds on the parameters; all of which delimit the explorable regions of the design space a priori. We explicitly list the equilibrium constraint $\mathbf{r}(\theta) = \mathbf{0}$ to ensure that the resulting geometry is funicular. This condition is enforced by construction by the FDM (Section 2.1) as long as $\mathbf{K}$ is invertible and the URS converges. Note that while all entries of θ can be optimized simultaneously, it is often convenient to restrict optimization to a subset to impose known biases, such as fixed support positions or prescribed external loads representing self-weight or other external actions.

We can solve Eq. 36 with gradient descent, iteratively updating θ in the direction of the negative gradient value for descent $\nabla_{\theta} \mathcal{L}(\theta) \in \mathbb{R}^{n_p}$, until we find the locally optimal parameters θ^* :

$$\theta^{(s+1)} = \theta^{(s)} - \mu^{(s)} \nabla_{\theta} \mathcal{L}(\theta^{(s)}) \quad (37)$$

The optimization process, from an initial guess until convergence, is illustrated in Fig. 9. Higher-order, gradient-based optimization algorithms like LBFGSB [127], SLSQP [128], and Levenberg-Marquardt [129] can speed up optimization by calculating suitable values for the step size μ or enhanced parameter update rules based on the structure of $\mathcal{L}$. Numerically, optimization converges when the loss value satisfies $\mathcal{L}(\theta) \leq \epsilon$ or the norm of the gradient $\|\nabla_{\theta} \mathcal{L}(\theta)\| \leq \kappa$ vanishes.

A main difference between our strategy for solving an inverse form-finding problem and other works is that we decouple the mechanics and the design, framing the problem as bilevel optimization. The inner loop solves for equilibrium via the FDM, producing a funicular geometry $\mathbf{x}$ that minimizes an energy functional Π , and the outer loop tunes θ so that $\mathbf{x}$ meets the goals in $\mathcal{L}$. The original FDM formulation [28] and subsequent works [113,125,130] combine equilibrium constraints and the specification's goals into the same loss function. Then, they optimize directly over $\mathbf{x}$, effectively creating a single-level optimization problem that could neither arrive at geometry in equilibrium nor satisfy the goals if the task specification is unfeasible and the optimization algorithm fails to converge. In contrast, we establish a hierarchy between mechanics and design by decoupling the two, optimizing θ instead of $\mathbf{x}$. This separation allows the FDM to focus on enforcing equilibrium to generate a valid $\mathbf{x}$ in equilibrium at each iteration, and the optimization algorithm to search for a solution that fulfills the design specification only within the space of funicular geometries. Both approaches are well known in structural optimization as simultaneous analysis and design (SAND), and nested analysis and design (NAND) [131]. Our work follows the NAND approach.

2.6. Derivatives computation with automatic differentiation

The expression of the gradient $\nabla_{\theta} \mathcal{L}$, required to find a minimum of Eq. (35), consists of the partial derivatives of the loss function with respect to the FDM parameters θ :

$$\nabla_{\theta} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta} + \frac{\partial \mathcal{L}}{\partial \theta} \quad (38)$$

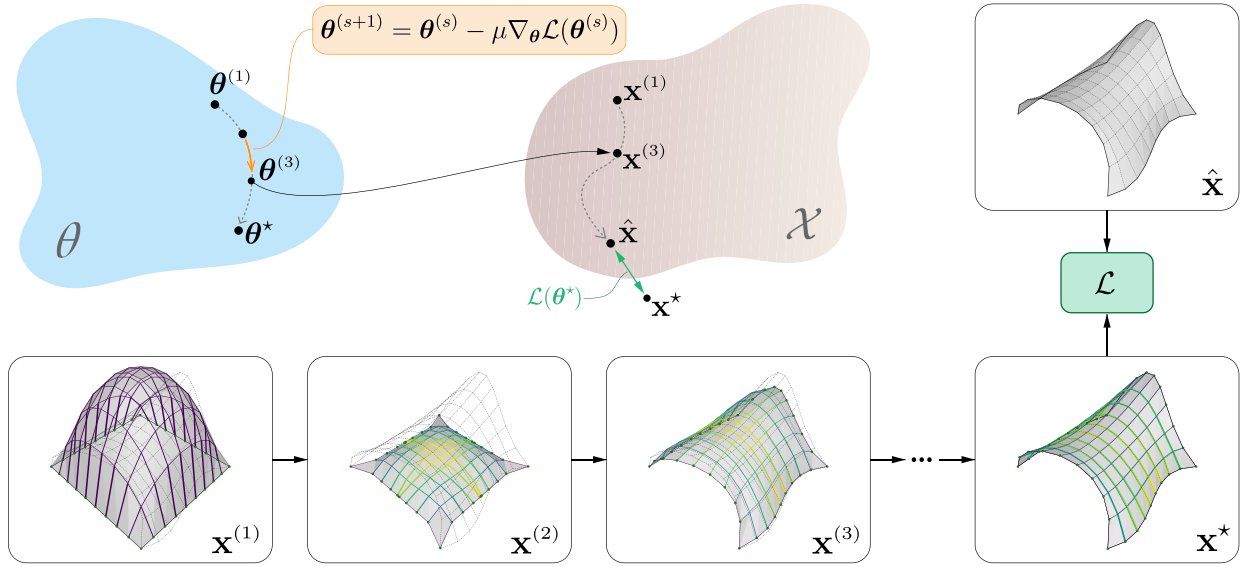


Fig. 9. Gradient descent is a numerically efficient approach to solving inverse form-finding problems in lightweight structural design. This optimization algorithm searches for a funicular geometry $\mathbf{x}$ that is as close as possible to a target $\hat{\mathbf{x}}$ by iteratively updating the parameters θ with the gradient of the loss function $\nabla_{\theta}\mathcal{L}(\theta)$. The loss value $\mathcal{L}(\theta)$ quantifies how well $\mathbf{x}$ satisfies a design specification. The search concludes upon finding optimal parameters θ^* and the optimal geometry $\mathbf{x}^*$ that minimize $\mathcal{L}(\theta)$, providing a solution where generally $\mathbf{x}^* \approx \hat{\mathbf{x}}$.

In particular, the Jacobian $\partial\mathbf{x}/\partial\theta \in \mathbb{R}^{3n_u \times n_p}$, the state sensitivity in structural optimization, concatenates the partial derivatives for each element of θ :

$$\frac{\partial\mathbf{x}}{\partial\theta} = \begin{bmatrix} \frac{\partial\mathbf{x}}{\partial\mathbf{q}} & \frac{\partial\mathbf{x}}{\partial\mathbf{p}} & \frac{\partial\mathbf{x}}{\partial\mathbf{s}} \end{bmatrix} \quad (39)$$

Previous works might offer solutions to obtain the derivatives in the above equations, but only at first glance. The FDM is a known form-finding method for over five decades [28,29], and many analytical derivative expressions have been published since then for various design tasks. Examples include prescribing target edge lengths and forces on cable-nets [29], designing tensegrities [67], matching reaction forces for bridge design [132]; approximating target shapes [37,133]; reducing the compliance of 2D trusses [134]; and minimizing the energy of compression-only shells [107,135]. While informative, reusing these equations to tackle other tasks, different from those they were originally formulated for, would require judicious mathematical manipulation. Moreover, these formulae mainly express derivatives with respect to the force densities (e.g., Schek's original work [29] derived the Jacobian $\partial\mathbf{x}/\partial\mathbf{q}$, the first entry in Eq. (39)), leaving the possibility to optimize the remaining FDM parameters an open research topic.

Rather than generating analytical derivative expressions for every possible design task, we implement the FDM in an AD framework [136] to automate this process. Our full software implementation is available in reference [137]. Here, we focus on the mathematics behind the implementation, which generalizes across different AD frameworks [138,139], and offer a compact implementation in the Appendix. In particular, we employ reverse-mode AD to automatically obtain the gradient value $\nabla_{\theta}\mathcal{L}(\theta)$ needed to minimize Eq. (35), for arbitrary loss functions, and for all the FDM inputs: the force densities $\mathbf{q}$, the applied loads $\mathbf{p}$, and the support positions $\mathbf{s}$. Optimizing force densities concurrently with the applied loads, or the support positions, enables load-finding and support-finding, expanding the funicular design space to arrive at lower loss values, and hence better-performing geometries (Section 3). Meanwhile, AD allows us to remain agnostic to the loss function $\mathcal{L}$, without having to re-derive analytical expressions of $\partial\mathcal{L}/\partial\mathbf{x}$ and $\partial\mathcal{L}/\partial\theta$ for every change in $\mathcal{L}$.

Reverse-mode AD works by first storing intermediary operations, or *primitives*, performed during the evaluation of the loss function (i.e., the forward or primal pass) in a directed acyclic computation graph. Each primitive v_i corresponds to a vertex in the computation graph, connected to the other vertices according to the data flow. AD then propagates derivatives in reverse, from the loss value back to the inputs, applying a *vector-Jacobian product* (VJP) defined for each primitive at each step of the traversal [70,136]. This reverse pass, known as backpropagation, computes the full gradient $\nabla_{\theta}\mathcal{L}(\theta)$ in a single graph traversal whose time complexity is asymptotically the same as one forward pass, offering better scaling with the number of parameters than forward-mode AD for scalar loss minimization, which would require a separate forward pass per parameter [72].

For a function representable as a simple sequence of primitives with a single input and output, the backpropagation algorithm can be expressed as the chain rule:

$$\bar{v}_0 = \bar{v}_n \frac{\partial v_n}{\partial v_{n-1}} \frac{\partial v_{n-1}}{\partial v_{n-2}} \dots \frac{\partial v_1}{\partial v_0} \quad (40)$$

where the upstream *cotangent* $\bar{v}_i = \partial v_n / \partial v_i$ is the accumulated derivative of the chain's output v_n up to the output of the primitive v_i . When the cotangent is a vector $\bar{\mathbf{v}}$ and the input and output of $\mathbf{v}$ are vectors too, each step in the backpropagation sequence is a VJP:

$$\bar{\mathbf{v}}_{i-1}^T = \bar{\mathbf{v}}_i^T \frac{\partial \mathbf{v}_i}{\partial \mathbf{v}_{i-1}} \quad (41)$$

In each step, the cotangent is right-multiplied by the Jacobian of the primitive, which linearizes the primitive around its input. This VJP is the *adjoint* map of this linearization, mapping sensitivities in the output space back to sensitivities in the input space [72]:

$$\left[\frac{\partial \mathbf{v}_i}{\partial \mathbf{v}_{i-1}} \right]^T \bar{\mathbf{v}}_i = \bar{\mathbf{v}}_{i-1} \quad (42)$$

Backpropagation successively applies the adjoints to pull derivatives from the output of the chain back to its inputs (Eq. (40)). When the output primitive v_n is the loss function $\mathcal{L}$, then $\bar{v}_n = 1$, and the backpropagation recursion yields the gradient $\bar{v}_0 = \nabla_{\theta} \mathcal{L}$. The extension of the backpropagation algorithm from computation chains to general graphs, like the ones that arise in the FDM in Figs. 4 and 7, is straightforward. Broadly, it entails handling branching and merging of the primitives and summing over the VJPs via fan-in and fan-out rules, repeating Eq. (40), but now for all paths from the output to the inputs in the graph. We refer the reader to [72,140] for an in-depth explanation.

A numerically efficient feature of reverse-mode AD is that it avoids materializing full Jacobians of the primitives to compute the gradient $\nabla_{\theta} \mathcal{L}(\theta)$. That is because $\mathcal{L}$ is scalar-valued, so one only has to perform a sequence of VJPs to solve Eq. (40), treating differentiation as a chain of matrix-vector products rather than more expensive matrix-matrix multiplications. Backpropagation speed, however, is constrained by the level of abstraction of the primitives: from elementary arithmetic (sums, products, and divisions), to linear algebra kernels (common in AD frameworks), to domain-specific simulation operators [141]. Higher levels of abstraction can lower computational overhead by reducing the number of VJPs to backpropagate through [142]. In the following sections, we develop mechanics-specific adjoints to expedite backpropagation and, by extension, accelerate optimization with the DFDM.

2.7. Linear solver adjoint

A bottleneck in our method is the solution of the linear system in Eqs. (23) and (32). We want to avoid backpropagating through all the operations involved in solving the system since we only require the derivatives at the solution $\mathbf{x}$ for optimization, not at any of its intermediate steps. Moreover, although for small problems we can materialize the full dense stiffness matrix $\mathbf{K}$, for larger problems we want to take advantage of its inherent sparsity to get computational speedup, especially as the linear system is solved many times during optimization.

Therefore, we apply implicit differentiation [52,143] to derive the adjoint of a linear system solver for compatibility with reverse-mode AD. This adjoint conceals the internals of the solver, enabling a more efficient backward pass, and allows us to stay flexible with the choice of solver without altering the differentiation logic. While previous works have developed linear solver adjoints for structural optimization via AD [95,97], they assume that $\mathbf{x}$ only depends on $\mathbf{K}$, the left-hand side of the linear system, while disregarding the dependency of the solution on the boundary conditions on the right-hand side. That assumption amounts to differentiating only with respect to the force densities $\mathbf{q}$ in our case. The adjoint proposed herein is more comprehensive than the precedents and accounts for the dependency of $\mathbf{x}$ on $\mathbf{K}$ and the effective loads $\mathbf{b}$, considering both left- and right-hand sides. This capability allows us to optimize $\mathbf{q}$, in addition to the applied loads $\mathbf{p}$ and the support positions $\mathbf{s}$, all encompassed by θ .

Given a loss function $\mathcal{L}$, AD provides us with a cotangent $\mathbf{v}^T = \partial \mathcal{L} / \partial \mathbf{x}$ with upstream derivatives, and our goal is to implement the VJP of the linear solve to obtain the first term of the gradient in Eq. (38):

$$\nabla_{\theta} \mathcal{L} = \mathbf{v}^T \frac{\partial \mathbf{x}}{\partial \theta} + \frac{\partial \mathcal{L}}{\partial \theta} \quad (43)$$

where $\mathbf{v}^T \partial \mathbf{x} / \partial \theta$ is the term we are interested in for the adjoint. To derive it, we first restate the equilibrium constraint (Eq. (19)), observing that $\mathbf{x}$ is an implicit function of θ at the solution through this constraint:

$$\mathbf{r}(\mathbf{x}, \theta) = \mathbf{K}(\theta) \mathbf{x}(\theta) - \mathbf{b}(\theta) = \mathbf{0}$$

Next, we take the total derivative of the equilibrium constraint to find an expression for $\partial \mathbf{x} / \partial \theta$:

$$\frac{\partial \mathbf{r}}{\partial \theta} + \frac{\partial \mathbf{r}}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta} = \mathbf{0} \implies \frac{\partial \mathbf{x}}{\partial \theta} = - \left[\frac{\partial \mathbf{r}}{\partial \mathbf{x}} \right]^{-1} \frac{\partial \mathbf{r}}{\partial \theta} \quad (44)$$

$$\frac{\partial \mathbf{x}}{\partial \theta} = -\mathbf{K}^{-1} \frac{\partial \mathbf{r}}{\partial \theta} \quad (45)$$

The computation of the full Jacobian would require the solution of a linear system with $\mathbf{K}$ on the left-hand side for each parameter in θ . We only need to know the VJP to evaluate $\nabla_{\theta} \mathcal{L}$ with backpropagation; not the full Jacobian, so we leverage the adjoint trick to simplify the calculation:

$$\mathbf{v}^T \frac{\partial \mathbf{x}}{\partial \theta} = -\mathbf{v}^T \mathbf{K}^{-1} \frac{\partial \mathbf{r}}{\partial \theta} = -\lambda^T \frac{\partial \mathbf{r}}{\partial \theta} \quad (46)$$

where we define $\lambda \in \mathbb{R}^{3n_u}$ as the solution to the adjoint system

$$\mathbf{K}^T \lambda = \mathbf{v} \quad (47)$$

Since $\mathbf{K}$ is symmetric, we can skip the transposition and use the same linear solver we use in the forward pass to solve Eq. (47), just with a different right-hand side. Once we find $\mathbf{x}$ during simulation with Eq. (23) and λ in the backward pass, we apply AD to automatically compute the VJP $\lambda^\top \partial \mathbf{r} / \partial \theta$, and hence the adjoint needed to backpropagate swiftly through the FDM.

2.8. Efficient backpropagation through shape-dependent loads

Computing funicular geometry $\mathbf{x}$ under shape-dependent loads consists of solving Eq. (32) repeatedly with the URS until convergence to a fixed point (Section 2.4). While standard AD can backpropagate through all iterates to evaluate the gradient $\nabla_\theta \mathcal{L}$, unrolling them incurs elevated computational costs and numerical instabilities [144], such as vanishing gradients [145], which may stall optimization. These issues can be mitigated by leveraging again the implicit function theorem [52,143].

We formulate a second analytical adjoint with implicit differentiation to backpropagate through shape-dependent loads. The benefit of this approach is to evaluate $\nabla_\theta \mathcal{L}$ only with information from the last iteration τ of the URS, without knowledge of any of the intermediate iterations involved in finding it (Fig. 7). Let $\mathbf{x}$ be a fixed point of Eq. (32). We rewrite this equation to expose the dependency of $\mathbf{x}$ on θ :

$$\mathbf{x}(\theta) = \mathbf{K}(\theta)^{-1} \mathbf{b}(\mathbf{x}, \theta) = \mathbf{y}(\mathbf{x}, \theta) \quad (48)$$

where $\mathbf{y}$ is a shorthand for the equilibrium solution at the fixed point. We differentiate both sides to obtain the Jacobian $\partial \mathbf{x} / \partial \theta$ at the fixed point:

$$\frac{\partial \mathbf{x}}{\partial \theta} = \frac{\partial \mathbf{y}}{\partial \theta} + \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta} \implies \frac{\partial \mathbf{x}}{\partial \theta} = \left[\mathbf{I} - \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right]^{-1} \frac{\partial \mathbf{y}}{\partial \theta} \quad (49)$$

where $\mathbf{I} \in \mathbb{R}^{3n_u \times 3n_u}$ is the identity matrix. We search for an expression for the product of $\partial \mathbf{x} / \partial \theta$ and the cotangent $\mathbf{v}^\top = \partial \mathcal{L} / \partial \mathbf{x}$ for backpropagation. This VJP is given by

$$\mathbf{v}^\top \frac{\partial \mathbf{x}}{\partial \theta} = \mathbf{v}^\top \left[\mathbf{I} - \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right]^{-1} \frac{\partial \mathbf{y}}{\partial \theta} = \lambda^\top \frac{\partial \mathbf{y}}{\partial \theta} \quad (50)$$

where $\lambda \in \mathbb{R}^{3n_u}$ is the solution of the adjoint linear system

$$\left[\mathbf{I} - \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right]^\top \lambda = \mathbf{v} \quad (51)$$

We leverage the structure of Eq. (48) and specialize the adjoint for a more explicit formulation. The Jacobian $\partial \mathbf{y} / \partial \mathbf{x}$ resolves to

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{K}^{-1} \frac{\partial \mathbf{p}}{\partial \mathbf{x}} \quad (52)$$

because $\mathbf{K}$ and $\mathbf{d}$ are independent of $\mathbf{x}$ (Section 2.3), and where the Jacobian $\partial \mathbf{p} / \partial \mathbf{x}$ encodes the sensitivity of the applied loads to perturbations on the geometry. Finding $\partial \mathbf{y} / \partial \mathbf{x}$ in Eq. (52), however, would necessitate a separate linear solve for each entry in $\mathbf{x}$. We can avoid that using the adjoint trick again since we do not need to materialize this Jacobian, but rather know how to take products with it. If we expand the left-hand side of Eq. (51), the linear system becomes

$$\lambda - \frac{\partial \mathbf{p}}{\partial \mathbf{x}}^\top \alpha = \mathbf{v} \quad (53)$$

where $\alpha \in \mathbb{R}^{3n_u}$ is the solution of:

$$\mathbf{K}^\top \alpha = \lambda \quad (54)$$

From this point on, we compute the target adjoint $\mathbf{v}^\top \partial \mathbf{x} / \partial \theta$ of the URS at the fixed point in two steps. First, we find λ by solving Eq. (51) through Eq. (53) with an iterative solver like conjugate gradient [146]. This approach allows us to solve the linear system by the repeated application of the VJP $\partial \mathbf{p} / \partial \mathbf{x}^\top \alpha$, instead of materializing the Jacobian of the loads in memory, which would be more costly. After we find λ , we leverage AD again to compute $\lambda^\top \partial \mathbf{y} / \partial \theta$ in Eq. (50), and thereby obtain the URS adjoint we seek.

3. Results

After having explained how the DFDM works in tandem with gradient-based optimization, we now showcase it in four design tasks. Each task highlights and benchmarks a different feature of the DFDM under a separate design specification. All of the presented numerical experiments are run on one CPU thread of an M2 silicon chip with 64 GB of RAM. Unless otherwise stated, we use a direct sparse solver to solve all linear systems involving $\mathbf{K}$ in the forward and backward passes. The choice of hardware and solver follows from the computational scaling studies we present in the Appendix. We set $\eta = 10^{-6}$, and utilize the LBFGSB [147], a quasi-Newton gradient-based algorithm, to solve the optimization problems with a convergence threshold of $\epsilon = 10^{-6}$ and $\kappa = 10^{-6}$, except if we specify a different configuration in the task description. For all tasks, we display the solutions generated by the fastest converging optimization approach.

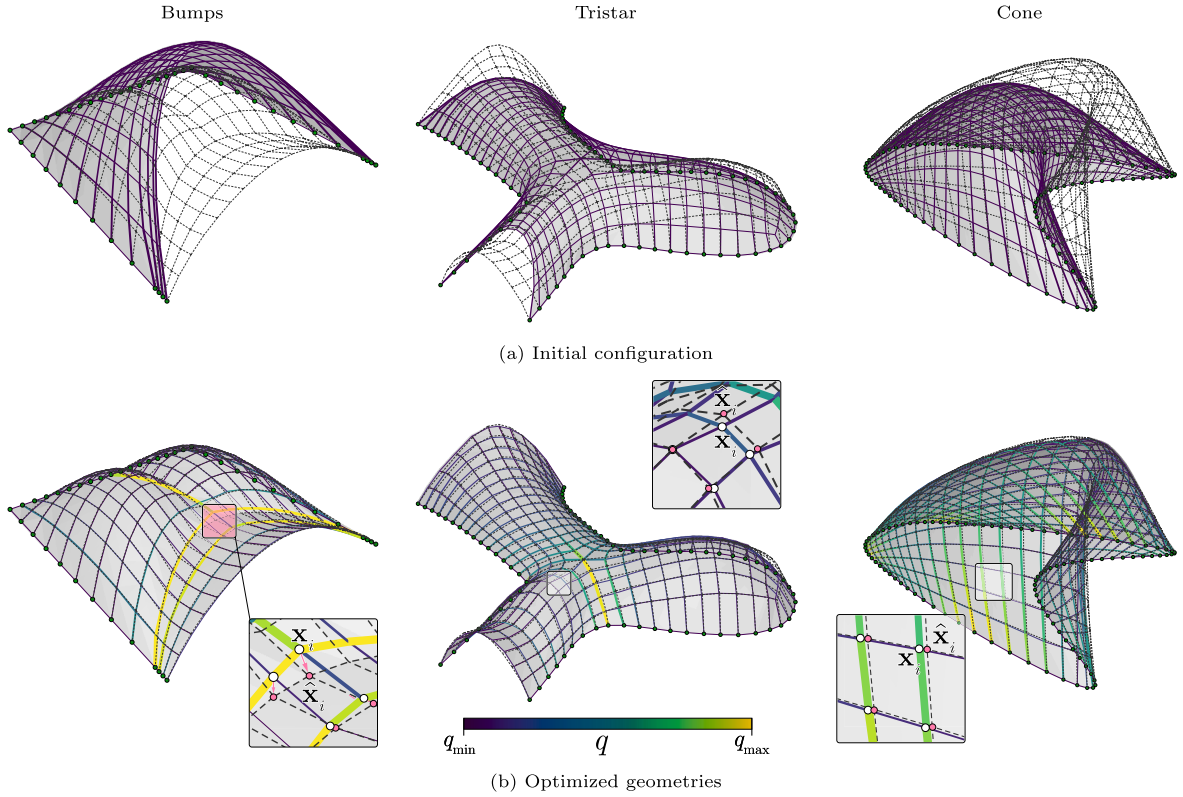


Fig. 10. Shape matching task. We approximate three arbitrary double-curved shapes with compression-only alternatives. The first row shows the funicular geometries before the optimization process, generated with a constant force density q for every edge, and in one FDM step. The second row depicts the optimized geometries. The optimal force densities need to have a spread to solve the task, as visible by the colored edges. As detailed in the callouts and Table 1, the distance between the individual vertex positions in equilibrium $\mathbf{x}_i$ and the targets $\hat{\mathbf{x}}_i$ is small, and the optimized solutions closely match the desired shapes.

3.1. Matching target shapes

Calculating funicular geometries that approximate arbitrary shapes facilitates the design of lightweight structures, such as grid-shells and thin shells, while preserving visual intent [27,37,77,99]. This task can alternatively be framed as a correction problem, where the objective is to find the deformations that an arbitrary shape has to undergo to reduce its total strain energy and thereby enhance its mechanical efficiency [148]. Addressing shape matching problems is also applicable for evaluating the mechanical integrity of heritage construction. One such example is the analysis of unreinforced masonry vaults modeled as thrust networks [31,33,49,149], as they must resist external loads mainly under axial compressive stresses.

Our approach for matching target shapes involves selecting a target surface and defining a graph G to approximate it with. Then, we minimize the mean distance between the 3D coordinates in equilibrium per vertex $\mathbf{x}_i$ and its desired location $\hat{\mathbf{x}}_i$ on the target:

$$g_1(\theta) = \frac{1}{n_u} \sum_{i=1}^{n_u} \|\mathbf{x}_i(\theta) - \hat{\mathbf{x}}_i\|^2 \quad (55)$$

An exact match between the positions in equilibrium and the targets in Eq. (55) evaluates to zero. For arbitrary shapes, however, this is unlikely as such a match would imply that the target is funicular from the outset. Therefore, the objective of this task is to generate a funicular geometry where the free vertices are the closest, in a least-squares sense, to a target shape.

We approximate the three doubly-curved shapes displayed in Fig. 10 to evaluate the DFDM's accuracy and speed. The selected shapes have appeared in previous research [31,106]. They exhibit synclastic and anticlastic curvatures and are supported along their perimeter. The optimization problem formulation is:

$$\begin{aligned} \min_{\mathbf{q}} \quad & \mathcal{L}(\theta) = g_1(\theta) \\ \text{s.t.} \quad & q_{lb} \leq q_j \leq q_{ub} \quad \forall j = 1, \dots, n_e \end{aligned} \quad (56)$$

We specifically search for the force densities $\mathbf{q}$ that render compressive axial forces, and accomplish that by setting negative bounds on them. We use $q_{lb} = -50$ and $q_{ub} = -10^{-3}$, for the lower and upper bounds on each force density, respectively. We apply a unit area load of $\psi = 1 \text{ kN/m}^2$ acting downwards on the target shape as self-weight, distributing it into individual vertex loads using the

Table 1
Statistics of the shape matching experiments.

Shape	n_e	n_u	$-q_0$	$\mathcal{L}_0 \downarrow$	$\mathcal{L} \downarrow$	$d_H \downarrow$	$\hat{\Gamma}_{ax} \uparrow$	$\Gamma_{ax} \uparrow$	$\Gamma/\hat{\Gamma} \downarrow$	Iters $\downarrow$	Time $\downarrow$	Time / Iters $\downarrow$		Boost $\uparrow$
						[%]	[%]	[%]	[%]		[s]	AD [ms]	ADA [ms]	
Bumps	462	204	6.0	21.1	0.03	2.2	43.2	87.4	40.9	277	0.3	2.0	0.9	2.2×
Tristar	799	346	4.5	2.1	0.02	1.0	23.1	96.5	24.1	175	0.3	5.1	1.5	3.4×
Cone	1200	529	3.0	1.0	0.01	0.9	42.6	98.4	35.8	83	0.2	11.3	2.2	5.1×

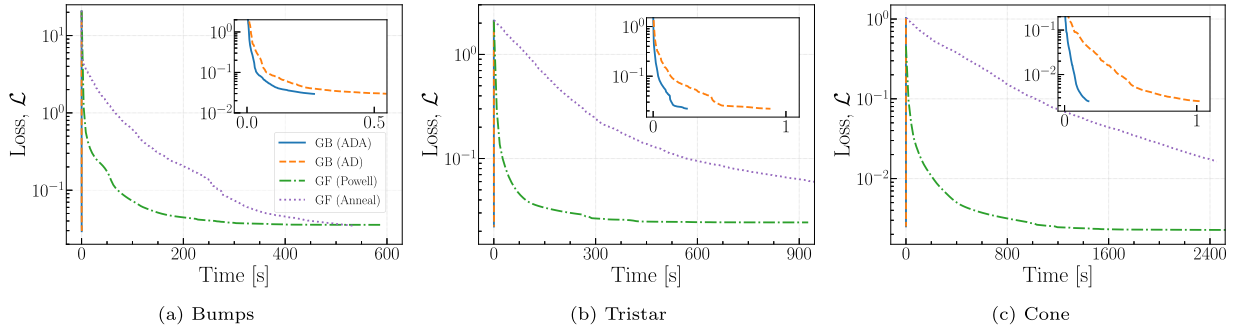


Fig. 11. Convergence history for the shape matching task for the three targets. The callouts zoom in on the loss value progression within the first second of the optimization process. Gradient-based (GB) optimization with LBFGB is three orders of magnitude speedier than gradient-free (GF) alternatives (Powell's method and dual annealing). Within gradient-based optimization, the DFDM's specialized approach leveraging AD with an analytical adjoint (ADA), is consistently faster than general-purpose AD.

vertex's tributary area on the target (Eq. (28)). To emulate the experimental settings of previous studies, we restrict the simulation to one FDM step and maintain the vertex loads constant during optimization using $\epsilon = \kappa = 10^{-5}$ as thresholds.

Table 1 summarizes the results. We report the starting force density q_0 for all the edges, the loss $\mathcal{L}$ values at the start and the optimum, elastic strain energies Γ , and Hausdorff distances d_H between funicular geometries and target shapes. The Hausdorff distance measures the similarity between the two point sets as the largest pairwise distance between them. We normalize it and present it as a percentage of the length of the diagonal of the target's bounding box. Table 1 also reports discretization details, the number of optimization iterations, and the total convergence wall time.

Taking the geometries shown in Fig. 10a as the starting point, we find optimal force densities that reduce the loss values by at least two orders of magnitude. Fig. 10b displays the optimal funicular geometries. The optimal force densities follow a non-trivial distribution in each example to fulfill the design specification, which highlights the importance of optimization to find them automatically. Our approach is capable of producing tight compression-only matches for all three targets in less than half a second. The largest deviation between $\mathbf{x}$ and the target $\hat{\mathbf{x}}$, indicated by d_H , is only 2.2% for *bumps*, and within 1% for *tristar* and *cone*. We reason that the looser fit of the first shape relative to the other two is that the former features sharp creases, while the latter two shapes do not present such curvature discontinuities.

Accuracy is important to find efficient and feasible shapes, but so is speed for quick design iterations. To quantify that, we benchmark the wall time of our gradient-based approach against two representative gradient-free alternatives: Powell's algorithm [150] and simulated annealing [151]. We use standard implementations of both algorithms with default parameters [152], and cap the total number of optimization iterations at 1000. Fig. 11 shows the convergence histories for all optimization approaches and the three target shapes. Powell's algorithm converges to loss values nearly as good as the DFDM's, but takes over a thousand times longer. Simulated annealing is the slowest among all algorithms, and only converges within the iteration limit in the smallest problem (*bumps*). By utilizing derivative information to guide the search, the DFDM with gradient-based optimization offers formidable speedups over gradient-free alternatives.

We further distill the computational advantages of utilizing the developed linear solver adjoint (Section 2.7), by comparing our mechanics-specific approach (ADA) with an implementation provided by a public AD framework [136]. As reported in Table 1, we outperform generic AD, leading by between two and five times in optimization speed per iteration and in total wall time (Fig. 11). In particular, gradient-based optimization with the DFDM becomes faster as the optimization problem grows in size, which is beneficial to model structures at finer discretizations.

Lastly, we pinpoint mechanical efficiency improvements based on the elastic strain energy of the resulting designs Γ [148]. To measure this energy, we perform a linear elastic structural analysis on the target shapes and the optimized geometries, applying the same loads and boundary conditions as during form-finding. We assign pipe cross-sections, made of nominal steel S235, to the edges of each structure. The pipes have a diameter of one-tenth of the average edge length per design, and a wall thickness of one-tenth of the diameter. After optimization, the axial-force share Γ_{ax} of the total energy Γ ascends to over 87% for *bumps* and over 96% for *cone* and *tristar*, doubling the pre-optimization share $\hat{\Gamma}_{ax}$ of the targets for the first two shapes, and quadrupling them for the last. This behavior confirms the axial-force dominance of the geometries generated by the DFDM, even under a different, linear elastic,

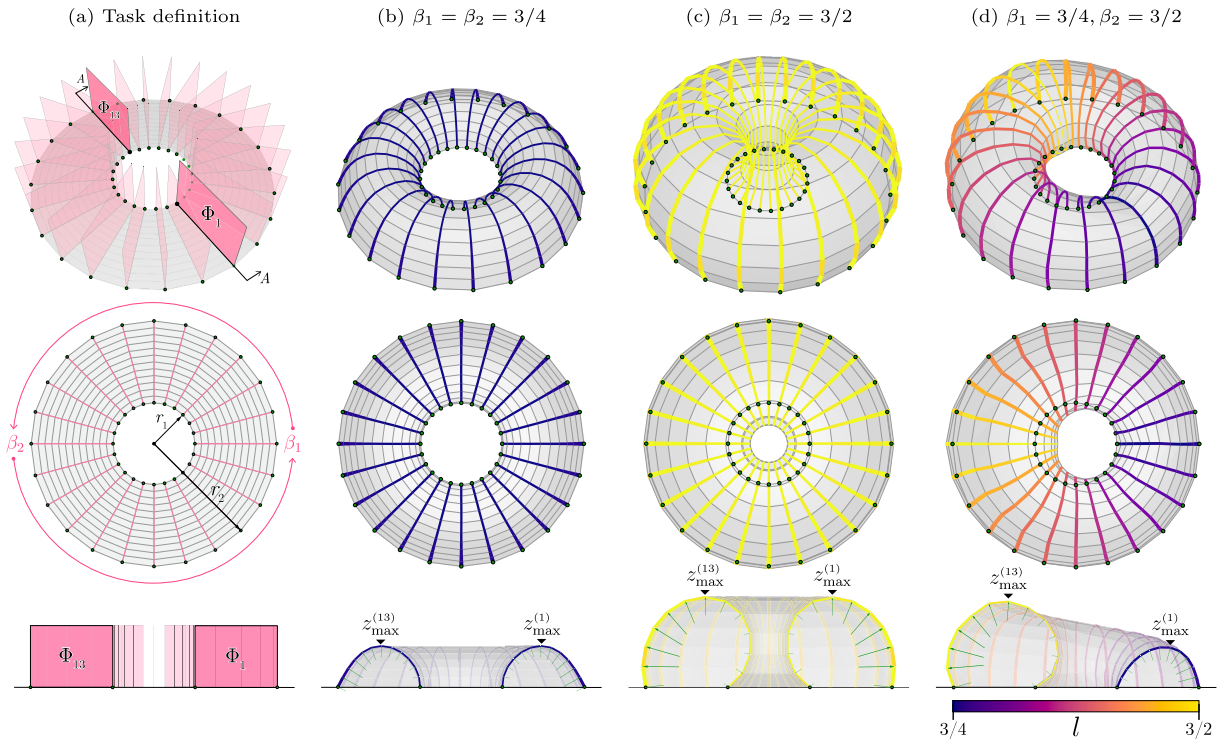


Fig. 12. Design of a pneumatic tensile structure. This structure is a half toroid fixed at its internal and external boundaries of radii r_1 and r_2 , respectively. We compute a geometry for the stiffening cable net of this inflatable system so that the net resists the action of an internal pressure of 0.25 kN/m^2 exclusively under axial tension forces. Additionally, its radial cables are constrained to lie on a plane Φ and to attain a target length l . The target length is interpolated between two boundary values β_1 and β_2 . The geometries computed by the DFDM for three distinct value pairs of β , all of which satisfy the imposed constraints, are shown in the figure. The radial edges are colored according to their length l .

mechanical model. Moreover, all of these geometries cut the total strain energy of the target shapes by at least half. The largest reduction is for *tristar* at 75%, while the smallest is in *bumps* at nearly 60%. The energy diminution across all designs emphasizes the importance of geometry in structural performance, and, crucially, that subtle geometric changes can lead to outsized efficiency gains. We generate such modifications automatically and in nearly real time with the DFDM.

3.2. Designing structures subject to pressure loads

The design of lightweight structures interacting with fluids involves predicting how the fluid's pressure affects both the geometry and the internal forces in the structure. Simulating this behavior is complex due to the dependency of the pressure load on the structure's shape, and vice versa [103]. Finite element simulations can accurately characterize these systems [104,153,154], but require second-order methods and precise material models, adding computational cost and problem dimensionality. Integrating these simulations into optimization schemes increases numerical effort [155,156]. Form-finding methods like the DFDM offer a faster, material-independent alternative for simulating this type of fluid-structure interaction at the conceptual stage of the design process [108,118]. This capability makes the DFDM ideal for exploring plausible preliminary designs, especially for shapes where no closed-form solution exists to represent them [157].

The second task in this work turns to the design of an air-supported structure. An air-supported structure is a single-layer pneumatic system that derives its stiffness from the internal pressurization of a membrane. Over long spans, this type of structure is reinforced by a cable net [157,158]. These tensile structures find applications in temporary construction and in roofs for industrial halls and sports facilities due to their fast erection and their capacity to enclose large areas with membranes only a few millimeters thick.

Here, we consider the design of the reinforcing cable net of a pneumatic structure modeled as a half toroid (Fig. 12). The net consists of 24 radial cables interconnected by 12 tangential cables. Each radial cable is discretized into 12 edges, while the tangential cables are discretized into 13 edges, resulting in a graph G with $n_e = 600$ edges and $n_v = 312$ vertices. The vertices of the cable net are fixed at the internal and external boundaries of the torus with radii $r_1 = 3$ meters and $r_2 = 9$ meters, respectively, amounting to $n_s = 48$ fixed vertices. The remaining vertices are free to move. We apply an internal pressure load of $\psi = 0.25 \text{ kN/m}^2$, normal to the surface of the structure, disregarding self-weight and external loads, as in [158]. We convert internal pressure load on the structure's surface into equivalent follower loads via tributary areas that we apply to the net's vertices [108,118]. During forward simulation, we update the cable net geometry and the loads iteratively following the approach we present in Section 2.4. We set $\eta = 10^{-4}$ for the

URS. In this task, we only utilize the URS adjoint from Section 2.8 during backpropagation to isolate its computational performance for optimization from that of the linear solver adjoint developed in Section 2.7.

The design task consists of computing pneumatic shapes that satisfy three properties: the radial cables attain a specific length $\hat{l}_j$, the radial cables lie on a plane Φ_i , and the entire structure resists the action of internal pressure in an axial tension state. To find a funicular geometry that satisfies these specifications, we minimize three goals at once:

$$\begin{aligned} \min_{\mathbf{q}} \quad & \mathcal{L}(\theta) = \sum_{i=1}^3 \alpha_i g_i(\theta) \\ \text{s.t.} \quad & q_{lb} \leq q_j \leq q_{ub} \quad \forall j = 1, \dots, n_e \end{aligned} \quad (57)$$

The optimization problem is formulated in terms of the force densities $\mathbf{q}$, totaling 600 optimization variables, subject to box constraints $q_{lb} = 10^{-6}$ and $q_{ub} = 20$ to ensure that the cable net axial forces are tensile.

The first goal measures the difference between the length l_j of each of the $n_{rad} = 288$ radial edges and their corresponding target lengths $\hat{l}_j$:

$$g_1(\theta) = \frac{1}{n_{rad}} \sum_{j=1}^{n_{rad}} ||l_j(\theta) - \hat{l}_j||^2 \quad (58)$$

We prescribe each $\hat{l}_j$ through a scalar factor β_j . This factor is linearly interpolated between reference values β_1 and β_2 as we move counterclockwise along the circumference of the torus outer boundary (Fig. 12a). The first and thirteenth radial cables represent the limit cases of the target lengths. For example, $\beta_1 = 1/2$ and $\beta_2 = 3/2$ result in a target length of $\hat{l}_j = 1/2$ and $\hat{l}_j = 3/2$ for the edges in the first and thirteenth radial cables, respectively; and a target length of $\hat{l}_j = 1$ for those in the seventh cable. If $\beta_1 = \beta_2$, the target length is the same for all radial cables.

The second goal quantifies the squared distance between the position $\mathbf{x}_i$ of each free vertex and a constraint plane $\Phi_i = (\mathbf{o}_i, \mathbf{n}_i)$. The plane is defined by an origin point $\mathbf{o}_i$, equidistantly spaced along the circumference of the torus boundary, and a unit normal vector $\mathbf{n}_i$, tangent to the circumference:

$$g_2(\theta) = \frac{1}{n_u} \sum_{i=1}^{n_u} ||(\mathbf{x}_i(\theta) - \mathbf{o}_i)^\top \mathbf{n}_i||^2 \quad (59)$$

In the above equation, the distance between $\mathbf{x}_i$ and Φ_i is the norm of the perpendicular projection of the vector going from $\mathbf{o}_i$ to $\mathbf{x}_i$ onto the plane's normal $\mathbf{n}_i$.

The third goal is an L2 regularization term that promotes solutions with evenly distributed force densities:

$$g_3(\theta) = ||\mathbf{q}||^2 \quad (60)$$

We explore three design options by varying the magnitude of β_1 and β_2 . In the first option, we set $\beta_1 = \beta_2 = 3/4$ while we set $\beta_1 = \beta_2 = 3/2$ in the second option. In the third option, we blend between the coefficients of the first two options, $\beta_1 = 3/4$ and $\beta_2 = 3/2$. In all three variants, we assign $q_0 = 2$ and $q_0 = 3/2$ as starting values for the force densities of the radial and tangential edges, respectively. We start the inflation of the pneumatic structure from a flat state, where the free vertex positions $\mathbf{x}_i$ lie on the same plane as the fixed vertices. The weights are $\alpha_1 = 5$, $\alpha_2 = 1$, and $\alpha_3 = 6 \times 10^{-5}$.

Solving the optimization problem given in Eq. (57) produces the results listed in Table 2 and the funicular geometries shown in Fig. 12. The radial edges in the optimized geometries satisfy the length and planarity goals with a marginal error on the order of 10^{-4} , except for the third experiment, where β_1 and β_2 are different and the planarity error is one order of magnitude higher, but still within three decimal places of zero. The length goal influences the geometry and the internal forces of the pressurized structures. The first two length specifications produce radially symmetric geometries, while the third outputs an asymmetric solution that nevertheless aligns with the target lengths interpolated between β_1 and β_2 . The qualitative good agreement between the desired and the resulting shapes is illustrated in Fig. 12b-12d.

A logical outcome of these experiments is that longer length targets result in expanded structures with a larger surface area a . A somewhat unintuitive result is that the height of the thirteenth cable in the third experiment is 12.5% shorter than that of the second one, despite the target lengths in both experiments being equal for this particular cable (Fig. 12, bottom row). We attribute the discrepancy in height to the lower surface area in the third experiment, which lowers the total applied pressure, and to variations in the profile of the radial cables in elevation. To illustrate this point further, all the radial cables shown in the bottom row of Fig. 12 bulge towards the center of the half torus except for the first radial cable when $\beta_1 = 3/4$ and $\beta_2 = 3/2$. Since the design specification only prescribes target cable lengths, without explicitly constraining their geometry in elevation, the profile of the radial cables can vary while attaining the task desiderata owing to the underspecified nature of the optimization problem. This result also suggests that there might exist multiple funicular solutions that minimize the above loss function, and that the search for a more specific design would require prescribing additional goals on the height or the curvature of the radial cables. The internal forces in the radial cables are less sensitive in absolute terms to the task specification than the geometry of the structure is. For instance, the range of the maximum internal force in the cables shown on the bottom row of Fig. 12—which would be used to dimension the diameter of the cables—varies by only 0.6 kN at most across the three experiments, between $f_{max} = 1.4$ kN and $f_{max} = 2.0$ kN. The reductions in axial forces are commensurate to the changes in a .

Table 2 and Fig. 13 evidence the superior speed of the DFDM with AD and the analytical adjoint for shape-dependent loads (ADA) developed in Section 2.8 over general-purpose AD. By avoiding backpropagation through all the iterations involved in finding

Table 2
Statistics of the pneumatic design task.

β_1	β_2	$\mathcal{L}_0 \downarrow$	$\mathcal{L} \downarrow$	$g_1 \downarrow$	$g_2 \downarrow$	a	$z_{\max}^{(1)}$	$z_{\max}^{(13)}$	$f_{\max}^{(1)}$	$f_{\max}^{(13)}$	Iters $\downarrow$	Time $\downarrow$		Boost $\uparrow$
		$[\times 10^{-2}]$	$[\times 10^{-2}]$	$[\times 10^{-4}]$	$[\times 10^{-4}]$	$[\text{m}^2]$	$[\text{m}]$	$[\text{m}]$	$[\text{kN}]$	$[\text{kN}]$		AD [s]	ADA [s]	
3/4	3/4	134.7	3.5	2.2	0.2	333.4	3.0	3.0	1.6	1.6	49	20.2	2.1	9.6×
3/2	3/2	463.4	2.9	5.1	1.9	566.8	6.6	6.6	2.0	2.0	47	21.6	3.6	6.0×
3/4	3/2	252.5	3.4	2.3	18.8	475.4	3.0	6.3	1.6	1.4	178	71.6	13.9	5.1×

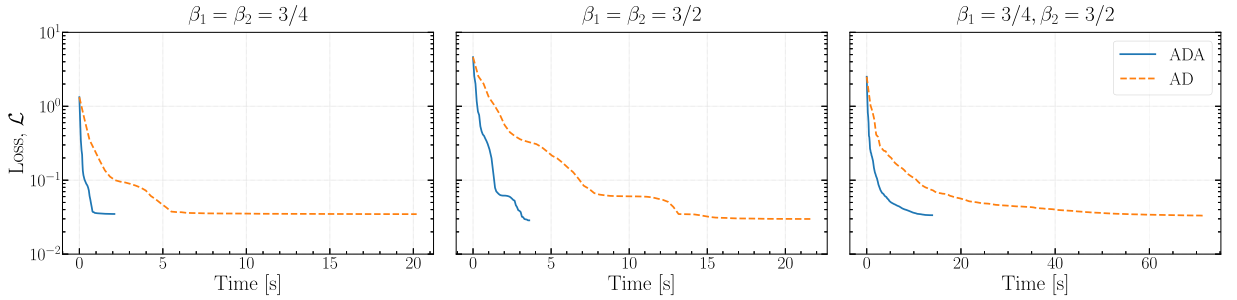


Fig. 13. Convergence plots for the pneumatic design task. The proposed differentiation approach, which augments AD with analytical derivatives for backpropagation (ADA), is consistently faster to converge than off-the-shelf AD as it shortcuts iterations in the backward pass.

a solution in equilibrium with the applied internal pressure, our approach converges 5.1 times faster in the most expensive optimization run, reducing the wall time from 71.6 to 13.9 seconds; and 9.6 times faster in the least expensive case, shortening the wall time from 20.2 to 2.1 seconds. In particular, Fig. 13 demonstrates that while both differentiation approaches eventually reach a solution, AD requires a much extended wall time, producing a convergence curve that is a much stretched version along the time axis of the curve resulting from ADA. This performance boost not only accelerates optimization on a per-design basis but also broadens the scope for design exploration under shape-dependent loads, as one can afford to generate between five and ten different options in the same amount of time that optimization with standard AD takes to find a single solution to this design task.

3.3. Load-finding

In this section, we demonstrate the application of load-finding, the simultaneous computation of optimal external loads and force densities. While this approach is general, here we show its capabilities to repair a cracked masonry dome. This hemispherical dome, presented in [159], has a radius of $r = 5.0$ meters and thickness of 0.5 meters. The dome undergoes a splitting horizontal displacement $\delta_i \in \mathbb{R}^2$ at its supports that induces cracks on the intrados and extrados of the structure (Fig. 14a).

The mechanical behavior of masonry shells is governed by global instabilities, not by local stress failures [160,161]. As a result, the assessment of these structures can be undertaken via limit analysis with a thrust network [32,33], a discrete representation of a compressive stress field in the shell that is described by a funicular geometry. With limit analysis, one can establish a lower bound on the collapse load of a masonry shell [162] insofar as there exists a compressive thrust network in equilibrium with the applied loads that fits within the thickness of the shell, and the no-tension, no-slip, and no-crushing conditions of Heyman's safe theorem are fulfilled [163]. Thrust networks closer to the medial axis of a shell reduce the eccentricity of the internal compression forces and reduce instability. In the reference, the discretization of the dome's thrust network is a radial grid with 20 meridians and 16 hoop rings, comprising $n_v = 321$ vertices, $n_s = 20$ of which are fixed, and $n_e = 640$ edges. We use the same configuration for the input graph G of the pin-jointed bar system representing the thrust network that we use for load-finding. The structure has a representative density of 1 kN/m^3 , and its weight is $\psi = 0.5 \text{ kN/m}^2$. We convert the self-weight into vertex loads by lumping them at the vertices of the network after computing their tributary areas on the medial surface of the hemispherical dome with Eq. (28). The loads remain constant during form-finding (i.e. have no shape dependency during simulation). The total self-weight is 77.4 kN.

Inspired by the reinforced arch method [164,165], a non-invasive and reversible retrofitting system for masonry shells, we aim to repair the dome with an external, post-tensioned cable net attached to the dome. The net would apply forces onto the dome, allowing the formation of a new compressive stress state that halts the supports' settlements and prevents crack propagation [166]. However, instead of optimizing the cable net and the thrust network together, we decouple the task of finding optimal loads applied to the thrust network from that of obtaining the specific cable net that exerts them. This separation enables flexibility in the realization of the retrofitting solution, as different cable net geometries could apply the same load vectors on the dome. Therefore, we seek an optimal set of external retrofitting loads that centers the thrust network within the masonry dome.

To solve this task, we first parametrize each external load with $\lambda_i \in \mathbb{R}$, a scalar that controls its magnitude along the unit normal $\mathbf{n}_i \in \mathbb{R}^3$ on the dome's middle surface at both free and fixed vertices:

$$\mathbf{p}_i = \mathbf{w}_i + \lambda_i \mathbf{n}_i \quad (61)$$

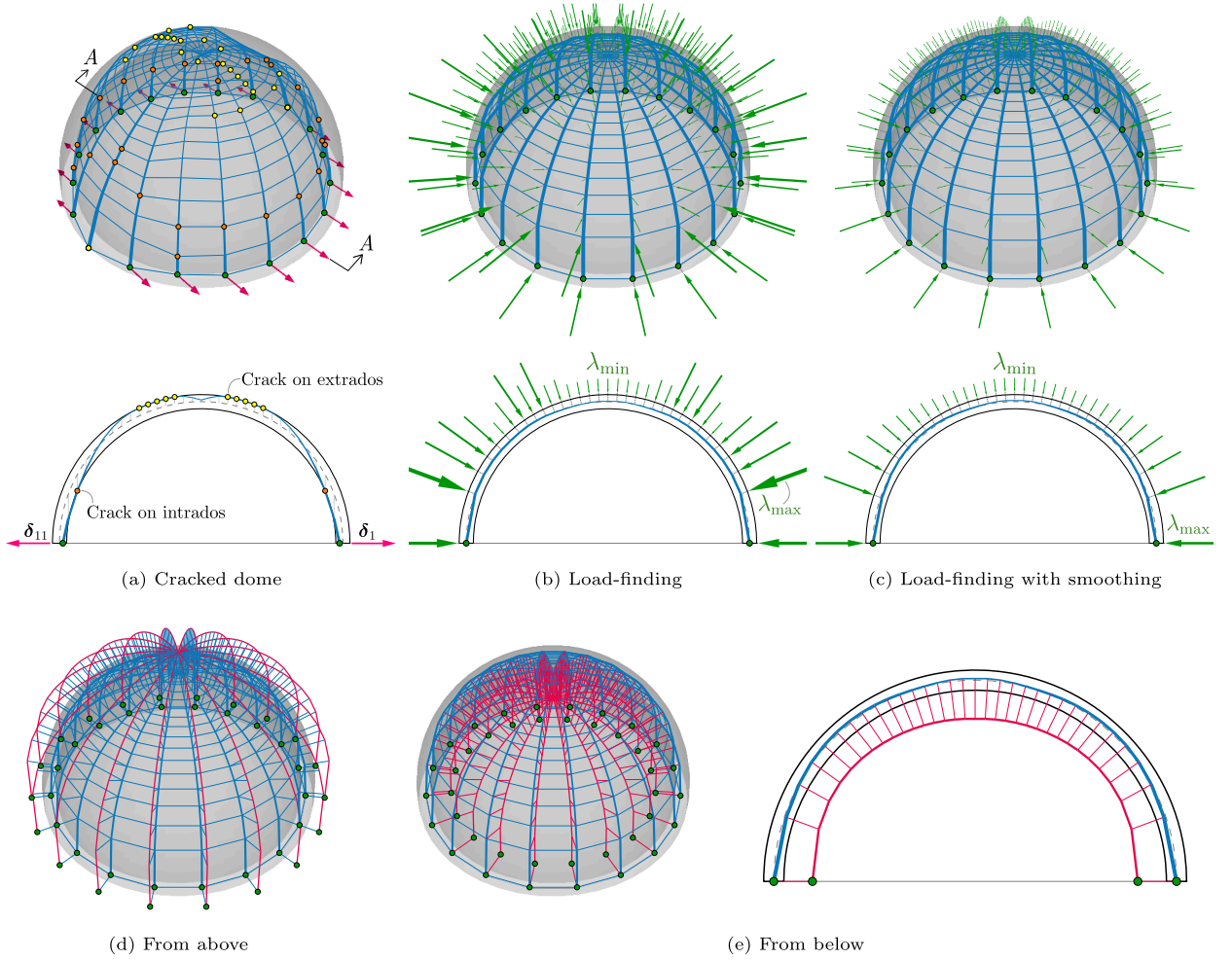


Fig. 14. Retrofitting a hemispherical masonry dome with load-finding. The dome is cracked due to a horizontal settlement at its supports that induces cracks in the masonry, splitting the dome in two in the middle (Fig. 14a). We generate two compressive thrust networks and sets of retrofitting loads, one without and one with smoothing on the individual load factors λ , as shown in Fig. 14b and c, respectively. Load-finding facilitates the design of different post-tensioning cable net geometries that all apply the same retrofitting loads (Fig. 14d and e).

The total load $\mathbf{p}_i$ applied at a vertex i is the self-weight vector $\mathbf{w}_i \in \mathbb{R}^3$ plus the scaled external load $\lambda_i \mathbf{n}_i$.

We bring the thrust network towards the dome's medial surface by matching target heights $\hat{z}$, defined on a half-sphere, on a point-wise basis:

$$g_1(\theta) = \frac{1}{n_u} \sum_{i=1}^{n_u} ||z_i(\theta) - \hat{z}_i||^2 \quad (62)$$

We use the same target field described in [33,159]. Among all possible distributions of external loads that can keep the thrust network within the thickness of the masonry shell, we are interested in one with a uniform spread of load magnitudes to facilitate the construction of the post-tensioning cable net system by reducing the variability of the prestresses and cable diameters. We hence formulate a second objective to control the variance of the load factors λ :

$$g_2(\theta) = \frac{\sum_{i=1}^{n_v} (\lambda_i - \lambda_{\text{avg}})^2}{\lambda_{\text{avg}}} \quad \text{where} \quad \lambda_{\text{avg}} = \frac{1}{n_v} \sum_{i=1}^{n_v} \lambda_i \quad (63)$$

A lower variance leads to a more uniform distribution, with a limit at zero where all loads would be equal.

We minimize both goals to center the thrust network and control the uniformity of the load distribution. The load-finding optimization problem imposes equality, inequality, and box constraints on the solution:

$$\min_{\mathbf{q}, \lambda} \mathcal{L}(\theta) = \sum_{i=1}^2 \alpha_i g_i(\theta) \quad (64a)$$

$$\text{s.t. } t_{x_i}(\theta) = 0 \quad \forall i = 1, \dots, n_s \quad (64b)$$

$$t_{y_i}(\theta) = 0 \quad \forall i = 1, \dots, n_s \quad (64c)$$

$$x_i(\theta) - \hat{x}_i = 0 \quad \forall i = 1, \dots, n_u \quad (64d)$$

$$y_i(\theta) - \hat{y}_i = 0 \quad \forall i = 1, \dots, n_u \quad (64e)$$

$$\hat{z}_i - z_i(\theta) \leq 0 \quad \forall i = 1, \dots, n_u \quad (64f)$$

$$z_i(\theta) - \hat{z}_i \leq 0 \quad \forall i = 1, \dots, n_u \quad (64g)$$

$$q_{lb} \leq q_j \leq q_{ub} \quad \forall j = 1, \dots, n_e \quad (64h)$$

$$\lambda_{lb} \leq \lambda_i \leq \lambda_{ub} \quad \forall i = 1, \dots, n_v \quad (64i)$$

These constraints ensure the feasibility of the funicular geometry computed by the DFDM. Following standard thrust network analysis [26,33], the funicular geometry is restricted to a fixed horizontal projection to maintain the lumped self-weight loads aligned with the direction of gravity and consistent with their distribution on the medial surface of the dome (Eq. (64d) and (64e)). Similarly, the geometry is constrained to fit within the thickness of the masonry shell to comply with the safe theorem [160,163]. We accomplish this requirement by bounding the z coordinates of the free vertices between their vertical projections on the intrados and extrados of the hemispherical dome, z_{lb} and z_{ub} , respectively (Eq. (64f) and (64g)). The analytical description of the intrados, medial, and extrados surfaces of the dome we use to specify these geometric constraints can be found in [33]. Lastly, we set the horizontal components of the reaction forces at the supports $\mathbf{t}_{xy}$ to zero to counteract the thrusts generated by the combined self-weight and retrofitting loads, which might cause additional damaging displacements at the dome supports (Eq. (64b) and (64c)).

The variables of the optimization problem are the vector of force densities $\mathbf{q}$ and the vector of load factors λ . We only optimize for the variables of one of the radial slices of the network to preserve symmetry and speed up convergence. This selection amounts to the edges and vertices in one of the meridians, and one edge per dome hoop, totaling 48 degrees of freedom: 32 force densities and 16 load factors. The other edges and vertices are constrained to have the same force densities and load factors as their counterparts in the optimized radial slice. We prescribe box constraints on the force densities to ensure a compression-only thrust network (Eq. (64h) and (64i)). The force density bounds are $q_{lb} = -50$ and $q_{ub} = -10^{-3}$. In addition, we prescribe a minimum load factor of $\lambda_{lb} = 0.25$, and cap the maximum load factor of $\lambda_{ub} = 2.5$ to control the amount of post-tensioning on the retrofitting intervention.

We solve the optimization problem defined in Eq. (64a) with and without the objective of a uniform distribution of retrofitting loads, with weights $\alpha_2 = 0$ and $\alpha_2 = 1/2$, respectively, to capture the effect of the smoothing term on the solution space. For benchmarking, we solve the same optimization problem, but only with the force densities and without the retrofitting loads, and set this approach as the baseline. Since the thrust network is subject to constant external loads, we only feature the linear solver adjoint developed in Section 2.7 to expedite gradient computations in this task. Then, a gradient-based, sequential quadratic programming algorithm with support for equality and inequality constraints [167] solves Eq. 64a. The optimization starts from the force densities of the cracked dome reported in [159] and $\lambda_0 = 1$ for all loads. The weight of the shape goal is $\alpha_1 = 1$. The optimized thrust networks for load-finding are displayed in Fig. 14b and c, and the convergence history for the loss function and both goals is presented in Fig. 15. Table 3 reports key statistics. Optimization for all three approaches converges in less than a hundred iterations and five seconds of wall time with general-purpose AD. AD with the linear solver adjoint (ADA) cuts the wall time in almost half to under three seconds.

The three approaches bring the thrust network closer to the dome's medial surface, but the improvement depends on the variables of the optimization problem. The reductions in the loss values are more pronounced in load-finding, which brings the loss value down by six orders of magnitude. The baseline, in contrast, decreases the loss value by only two (Fig. 15). Performance on the shape goal g_1 is decisive to reduce the loss value and to align the thrust network with the dome's medial surface. The largest distance to the medial surface d_M in the baseline solution is 39.1% of the shell's thickness. Load-finding shrinks this distance to 3.8%, a tenfold enhancement, offering substantial improvement on the medial surface fit. Nevertheless, the loads obtained with load-finding to minimize g_1 exhibit a ragged distribution, with peaks of $\lambda_{\max} = 2.5$ alternating with $\lambda_{\min} = 0.25$ along the dome's circumference (Fig. 14b). The largest loads, for example, appear at the vertices next to the dome's supports, while the smallest ones arise at the dome's crown and the dome's thirds. This erratic pattern could ultimately result in a cable net that is more expensive to build due to the variability of the prestresses and cable diameters.

Adding the smoothing term g_2 to the loss function mitigates this issue. The resulting load distribution halves the variance of the load factors λ from 0.49 to 0.22 and lowers the average load from 0.6 to 0.3, enabling a more rational post-tensioning design. Fig. 14c corroborates the uniformity of the load distribution. The loads decrease gradually from $\lambda_{\min} = 0.25$ at the dome's crown to $\lambda_{\max} = 1.4$

Table 3

Statistics of the retrofitting task for the baseline (FD), load-finding (LF), and load-finding with smoothing (LFS).

	$\mathcal{L}_0 \downarrow$	$\mathcal{L} \downarrow$	$g_1 \downarrow$	$g_2 \downarrow$	$\lambda_{\text{avg}} \downarrow$	$\lambda_{\text{max}} \downarrow$	$d_M \downarrow$	$\ t_{xy}\ /w \downarrow$	$-f_{\text{avg}} \downarrow$	Iters $\downarrow$	Time $\downarrow$		Boost $\uparrow$
	$[\times 10^0]$	$[\times 10^{-3}]$	$[\times 10^{-3}]$	$[\times 10^{-1}]$	[kN]	[kN]	[%]	[%]	[kN]		AD [s]	ADA [s]	
FD	1.2	44.7	44.7	–	–	–	39.1	22.2	0.7	26	1.1	0.6	1.8×
LF	1620.7	0.1	0.1	4.9	0.6	2.5	3.8	0.0	3.7	75	3.8	2.0	1.9×
LFS	1620.7	11.5	3.4	2.2	0.3	1.4	11.5	0.0	2.6	97	4.9	2.7	1.8×

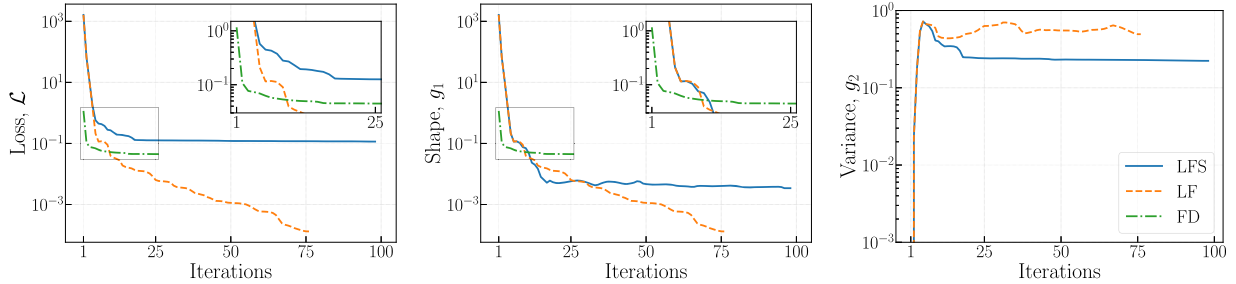


Fig. 15. Convergence history for the masonry retrofitting task using load-finding, with (LFS) and without (LF) smoothing on the load factors. The baseline approach (FD) only optimizes the force densities. We minimize the loss value (left), the geometry fit (middle), and the variance of the loads (right). In the last plot, the variance of LF oscillates since the weight of the variance term α_2 in the loss function $\mathcal{L}$ is zero and is thus not penalized. We display it only for comparison with LFS.

at the supports. It is worth noting that smoothing the retrofitting loads deteriorates the match accuracy, and comes at the expense of a looser fit of the thrust network to the middle surface. But this approach still results in a threefold improvement on d_M over the baseline (39.1% vs. 11.5%) while attuning the solution to manufacturing considerations. The smoothing term ultimately works as a knob that one can dial up or down depending on how uniform the loads should be. The trade-offs between post-tensioning complexity and the thrust network fit depend on specific project conditions, and the DFDM provides, once again, a framework to explore them.

In the updated compressive stress state of the masonry dome, both retrofitting load sets increase the average internal force f_{avg} in the network edges by a factor of five compared to the baseline. But both solutions only thrust vertically at the supports since they satisfy the horizontal reaction constraints we imposed (Eq. (64b) and (64c)). The retrofitting loads counteract the horizontal thrusts. Without them, the dome's supports would still experience at least a fifth of the total self-weight as horizontal thrusts ($\|t_{xy}\|/w$ in Table 3), preserving the tendency of the dome to displace horizontally and aggravating crack propagation.

Fig. 14d and e present two different cable net designs for retrofitting the dome to underscore the flexibility of load-finding. The nets apply the optimized loads onto the masonry dome. One of them repairs the dome from above, while the other does so from below, beneath the intrados. The choice of one cable net over the other would depend on site conditions; for example, on whether the dome's extrados is accessible or not for construction works. The design of retrofitting interventions with load-finding proves useful for repairing masonry shells. It discovers funicular geometries that center the thrust network in a masonry structure while aligning with fabrication considerations for the post-tensioning system. Load-finding generates better designs due to the additional degrees of freedom that expand the search space for achieving lower loss values. While this task has focused on the reinforced arch method [164,165], load-finding could support other retrofitting strategies based on thrust line and thrust network analysis, such as those that strengthen masonry structures using composite materials [168,169].

3.4. Support-finding

In the last design task, we showcase support-finding, the joint optimization of force densities and support positions, in tasks with multiple desiderata. We planarize the geometry of a rigid gridshell of an arbitrary, quad-dominant layout to that effect. A gridshell is a beam network able to span large distances with small cross-section sizes by emulating the bending-minimizing behavior of a continuous membrane [170]. Glazed gridshells with steel members combine transparency with material efficiency to create long-span and naturally lit roofs over public buildings like museums [22,171], malls [172–174], and transport hubs [175–177]; reducing new construction for sustainability and promoting urban densification by repurposing existing structures.

We revisit the iconic gridshell over the courtyard of the British Museum as a case study to showcase our approach. The shape of this gridshell was generated analytically to interpolate smoothly between the inner and outer boundaries [171], and discretized with a triangulated layout that yields planar faces, but without funicularity restrictions [178,179]. Due to its support conditions that limit thrusts by letting the structure slide horizontally on the outer perimeter, the existing gridshell carries its weight through a mix of bending and truss action. Therefore, the task's objective is to propose an alternative design that supports its self-weight in a bending-free state, while reducing horizontal thrusts at the supports and remaining compatible with flat panel cladding. The task setup is shown in Fig. 16a.

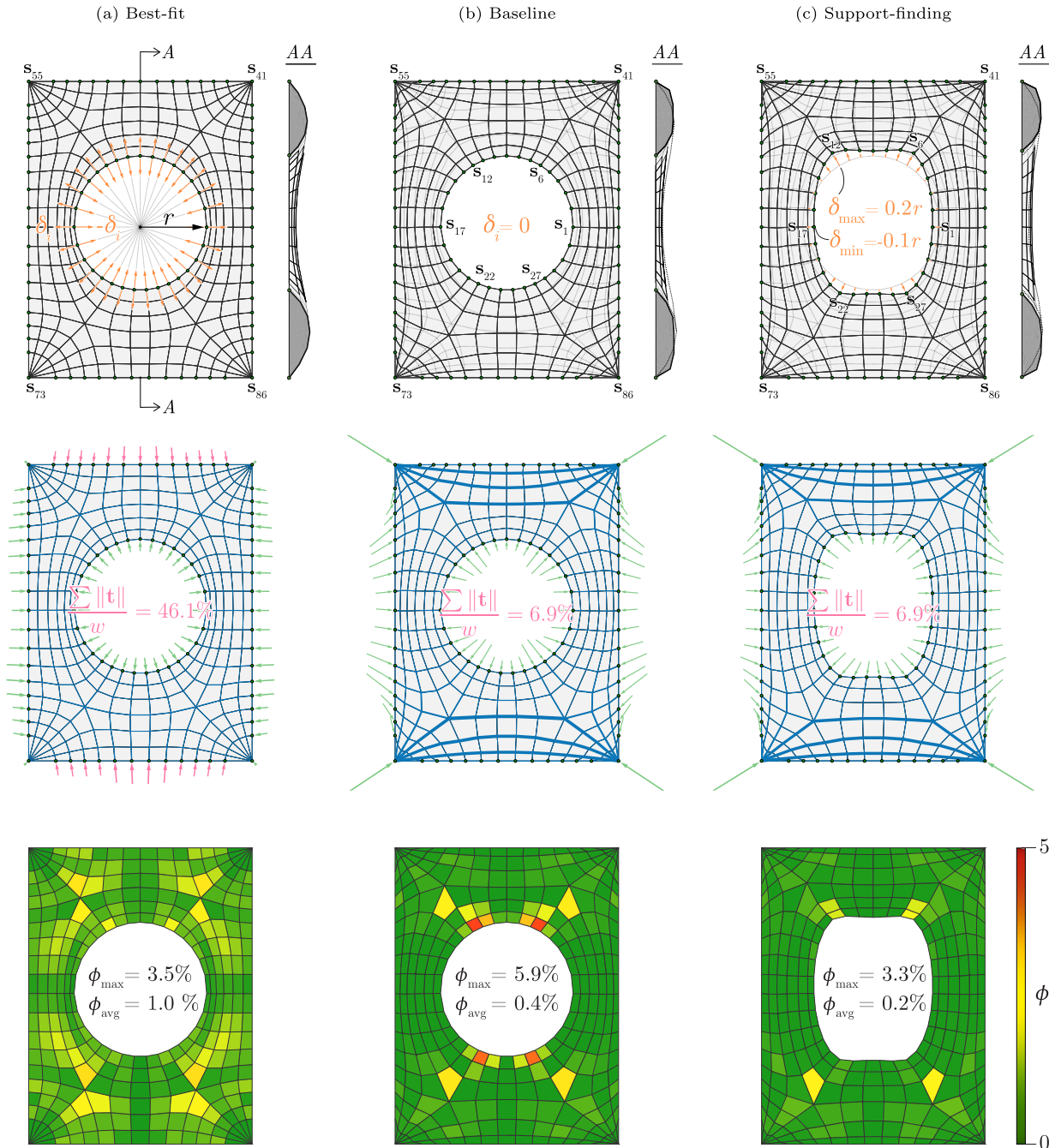


Fig. 16. Planarizing a funicular gridshell with support-finding. The rows show the structure's funicular geometry, the reactions on the supports, and the planarity of the glazing panels. The first column presents the initial geometry, which is a best-fit to the as-built gridshell's shape. The other two columns display optimized designs that minimize thrusts on the north and south sides of the outer boundary, and maximize the planarity of the panels. Support-finding arrives at a compression-only geometry that reduces the thrusts by over 90% and has more flat panels (mean skewness of only $\phi_{\text{avg}} = 0.2\%$ per panel) than the best-fit and the baseline options, offering an optimal solution to reduce manufacturing cost and simplify construction while preserving mechanical efficiency through funicularity.

The British Museum's rectangular courtyard is 97 m long in the north-south direction and 73 m wide in the east-west direction [171]. The gridshell is supported on two planar boundaries located at equal height in elevation. The outer boundary rests on the buildings flanking the courtyard's perimeter, whereas the inner circular boundary, of radius $r = 22$ meters, sits 3 m to the north of the courtyard's center and introduces asymmetry along the north-south axis. In contrast to the existing roof, we let the geometry of the inner boundary vary during optimization. Our hypothetical setup also assumes the east and west buildings can resist horizontal thrust, while those on the north and south sides must be as thrust-free as possible. The quad-dominant gridshell layout we employ is generated via medial-axis meshing [180]. We introduce eight high-valency vertices into the layout to facilitate load transfer towards the corners of the structure, as the existing gridshell does [178,179]. The resulting topology for G comprises $n_e = 599$ edges, $n_f = 282$ faces, and $n_v = 317$ vertices. 30% of the vertices are supported. Every face maps to a glazing panel.

We articulate this design task as the combination of four goals. The first goal quantifies the magnitude of the individual reaction vectors $\mathbf{t}_i$ to suppress them on the $n_{\text{out}} = 25$ support vertices on the north and south sides of the fixed rectangular boundary:

$$g_1(\theta) = \frac{1}{n_{\text{out}}} \sum_{i=1}^{n_{\text{out}}} \|\mathbf{t}_i(\theta)\|^2 \quad (65)$$

The second goal encodes the planarity constraint of a panel by measuring how orthogonal the edge vectors of the face that represents it are to the face's normal. For a given face k , we compute the absolute value of the dot product between the unit face normal $\mathbf{n}_k$ and each normalized edge vector $\mathbf{e}_j$, summing over all edges of the face, $E(k)$. Minimizing this sum encourages the edges to lie within the face plane:

$$g_2(\theta) = \frac{1}{n_f} \sum_{k=1}^{n_f} \sum_{j=1}^{|E(k)|} \|\mathbf{n}_k(\theta)^\top \mathbf{e}_j(\theta)\| \quad (66)$$

where $|E(k)|$ denotes the size of the edge set $E(k)$. We add a third goal as a regularizer that lowers the fairness energy of the vertices to bias the solutions towards smooth geometries with even faces of uniform inner angles:

$$g_3(\theta) = \frac{1}{n_v} \sum_{i=1}^{n_v} \chi_i(\theta) \quad (67)$$

The fairness energy χ_i of vertex i accounts for varying valencies, like in the quad-dominant layout we utilize, following the treatment of Tang et al. [106]. Denoting the ordered cycle of the one-ring neighbors of a vertex by $V(i)$ and its size by $|V(i)|$, the energy is a function of the distance between its position $\mathbf{x}_i$ and that of its neighbors:

$$\chi_i(\theta) = \begin{cases} \left\| \mathbf{x}_i(\theta) - \frac{\mathbf{x}_{i+1}(\theta) + \mathbf{x}_{i+3}(\theta)}{2} \right\|^2 + \left\| \mathbf{x}_i(\theta) - \frac{\mathbf{x}_{i+2}(\theta) + \mathbf{x}_{i+4}(\theta)}{2} \right\|^2 & \text{if } |V(i)| = 4 \\ \left\| \mathbf{x}_i(\theta) - \frac{1}{|V(i)|} \sum_{j=1}^{|V(i)|} \mathbf{x}_{i+j}(\theta) \right\|^2 & \text{otherwise} \end{cases} \quad (68)$$

A fourth and last goal restricts the area of the gridshell's geometry to match the area of the existing roof

$$g_4(\theta) = \|\hat{a} - \sum_{k=1}^{n_f} a_k(\theta)\|^2 \quad (69)$$

by measuring the difference between the as-built target area $\hat{a} = 6275 \text{ m}^2$ and the sum of the face areas a_k of the new geometry. This ensures an adequate comparison, isolating the effect of geometric changes on performance rather than differences in area.

After defining the goals, we set up the optimization problem with the force densities and the support positions on the inner boundary as the optimization variables:

$$\begin{aligned} \min_{\mathbf{q}, \delta} \quad & \mathcal{L}(\theta) = \sum_{i=1}^4 \alpha_i g_i(\theta) \\ \text{s.t.} \quad & q_{\text{lb}} \leq q_j \leq q_{\text{ub}} \quad \forall j = 1, \dots, n_e \\ & \delta_{\text{lb}} \leq \delta_i \leq \delta_{\text{ub}} \quad \forall i = 1, \dots, n_{\text{in}} \end{aligned} \quad (70)$$

We parametrize the position of the $n_{\text{in}} = 31$ inner supports by scaling a unit translation vector $\mathbf{n}_i$ by a displacement factor δ_i :

$$\mathbf{s}_i(\delta_i) = \mathbf{s}_{0_i} + \delta_i \mathbf{n}_i \quad (71)$$

The translation vector points from the centroid of the circular boundary at each support and is contained in the boundary's plane. The scaled translation vector is then added to the initial support position $\mathbf{s}_{0_i}$, as illustrated in Fig. 16a.

The optimization variables are $\mathbf{q}$ and δ , resulting in a total of 630 variables, all subject to box constraints. We apply $q_{\text{lb}} = -100$ and $q_{\text{ub}} = -10^{-3}$ to ensure a compression-only internal force state (and thus a shape that bulges upwards, instead of downwards with a tension-only state), and $\delta_{\text{lb}} = -0.3r$ and $\delta_{\text{ub}} = 0.3r$ to keep the inner boundary relatively close to its initial shape. The bending-free property for the gridshell's geometry is guaranteed by construction upon solving Eq. 27, as in the other tasks we present in Section 3. We apply an area load of $\psi = 1 \text{ kN/m}^2$ as the self-weight of the structure, which is within tolerance of the weight of the existing

Table 4

Statistics of the gridshell design task for the best-fit (BF) solution, the baseline (FD), and support-finding (SF).

	$\mathcal{L}_0 \downarrow$	$\mathcal{L} \downarrow$	$g_1 \downarrow$	$g_2 \downarrow$	$a/\hat{a} \downarrow$	$d_H \downarrow$	$z_{\max} \downarrow$	$-f_{\text{avg}} \downarrow$	$-f_{\text{max}} \downarrow$	$ \mathbf{t} /w \downarrow$	% Flat $\uparrow$	Time $\downarrow$		
			$[\times 10^2]$	$[\times 10^{-2}]$	[%]	[%]	[m]	[kN]	[kN]	[%]	$[\leq 1\%]$	AD [s]	ADA-L [s]	ADA-LU [s]
BF	15.8	15.8	134.2	3.9	97.9	0.5	6.8	63.7	167.2	46.1	59.9	-	-	-
FD	15.8	2.2	3.2	1.5	97.7	3.6	6.0	147.4	686.9	6.9	92.2	66.1	55.3	16.6
SF	15.8	2.1	3.2	0.9	97.6	3.9	6.4	129.8	603.5	6.9	96.8	64.4	55.8	17.4

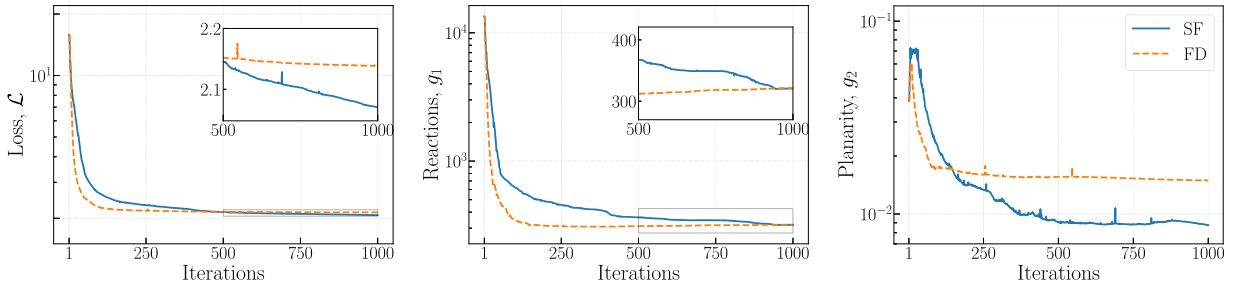


Fig. 17. Convergence history of the gridshell task with support-finding. We optimize the loss value (left), the reaction magnitudes (middle), and panel planarity (right) over a fixed compute budget of 1000 iterations. We contrast the optimization of force densities (FD) with support-finding (SF), where we modify both force densities and support positions jointly. Support-finding converges to a solution with reactions equal to those of the baseline, and outperforms it in planarity and the final loss value.

roof [178], and solve Eq. (27) using the URS (Section 2.4). We note that our work differs from others that simultaneously optimize for bending and planarity [106,181–184], in that they produce statics-aware shapes that might be planarized, but not bending-free.

We compare the performance of support-finding with that of a best-fit approach and a baseline where we only optimize the force densities. The three approaches apply the DFDM and gradient-based optimization, but with a different loss function structure and optimization variables. In the best-fit approach, we compute a funicular geometry that simply approximates the as-built gridshell's shape following the procedure we describe in Section 3.1. The resulting force densities are used as the initial guesses for support-finding and the force density baseline, and both solve Eq. (70) with the goal weights set to $\alpha_1 = 10^{-3}$, $\alpha_2 = 12$, $\alpha_3 = 1$, and $\alpha_4 = 10^{-5}$, for exactly a thousand iterations. We fix the iteration count to distill the benefits of setting the support positions as additional optimization variables under the same computational budget.

Fig. 16 depicts the optimized gridshells, their load-bearing behavior and planarity. Table 4 reports several statistics, and Fig. 17 shows the convergence history through the optimization for support-finding and the baseline. Integrating the linear solver and URS adjoints (ADA-LU) into backpropagation reduces computational cost by nearly four times compared to standard AD. In both support-finding and the baseline, however, the majority of the savings come from the URS adjoint. The linear solver adjoint (ADA-L) only reduces the wall time by 16% at most relative to AD as Table 4 indicates. The URS dominates the computation cost since it calls the linear solver multiple times during both forward and backward passes. The area of the optimized geometries varies within a small relative error of 2.5% from the target, and the Hausdorff distance d_H between the analytical, ground-truth shape and the three optimized geometries is under 4% of the length of the diagonal of the bounding box of the ground truth. These metrics show that the optimized geometries emulate the form of the as-built gridshell with high fidelity, both globally and locally. The best-fit geometry has the tallest profile $z_{\max}$, followed by the support-finding and baseline variants, which are 0.4 and 0.8 m shallower, respectively.

After support-finding, the inner boundary of the gridshell morphs from a circle to a more nuanced, oval-like shape flattened along the north and south sides of the courtyard (Fig. 16c). This deformation is anisotropic and amounts to only a fifth of the inner boundary's radius at most: the support vertices on the north and south are pushed outward with a maximum displacement of $\delta = 0.2r$ at vertices 6 and 12, while those on the east and west are pulled inward by $\delta = -0.1r$ at vertices 1 and 17. These alterations respond to the task specification, reducing spans in directions where thrusts are minimized and enlarging the spans where thrusts are permissible to fulfill the target area constraint.

Despite the different inner boundary shapes, both geometries optimized with support-finding and the force density baseline reduce the sum of the reactions on the north and south supports, expressed as a percentage of the total weight of the structure $||\mathbf{t}||/w$, by nearly 40% from the best-fit alternative, from 46.1% to 6.9%. This reduction is achieved by redirecting force transfer along bars that span between the gridshell's corners, activating internal arching action. A fourfold surge in the largest bar forces $f_{\max}$, a twofold increase in the average bar forces f_{avg} (Table 4), and the larger thrusts at the corners (Fig. 16, second row), evidence this new load-bearing behavior. The arching mechanism is accompanied by shape changes: the longitudinal profile of both optimized geometries bulges upwards, forming a kink just before reaching the supports (Fig. 16b and c).

As much as support-finding and the force density baseline accomplish similar loss values and reaction forces within a second of wall time difference, their outputs vary considerably in panel planarity. If we measure the skewness ϕ_k of panel k —the ratio of the distance between opposing face vertices to the average length of the face edges—and cap it at a stringent limit of 1%, the support-found geometry has the flattest panels, at 96.8%, improving by 36.9% and 4.6% over the best-fit and the baseline, respectively.

Similarly, the panels of the geometry generated by support-finding are, on average, five times flatter than those of the best-fit geometry ($\phi_{\text{avg}} = 0.2\%$), keeping the maximum distortion on par with it ($\phi_{\text{max}} = 3.5\%$), but localized to just half a dozen panels. The baseline offers considerable improvement in panel planarity per se, but every panel is two times more skewed than with support-finding, on average and in the worst case, incurring higher production costs than the support-finding solution if the latter design is chosen for manufacturing. Fig. 17 provides insight into this discrepancy. Sole force density optimization plateaus for the planarity goal g_2 early in the optimization process, locking into a suboptimal basin of the loss landscape. Meanwhile, treating support positions as additional optimization variables enables a steady loss value reduction that eventually surpasses the baseline.

In sum, support-finding produces a bending-free geometry with more flat panels than the two other approaches, while satisfying thrust, fairness, and area constraints. It renders a gridshell design that decreases manufacturing effort, and that is compatible with existing site conditions. By calibrating force densities and support positions concurrently, the DFDM uncovers geometries that balance structural performance with fabrication ease.

4. Conclusion

Lightweight structures enable long-span systems that support many aspects of human activity, from bridges crossing rivers to roofs that shelter public spaces. However, designing their shapes so that they carry loads efficiently while fulfilling additional technical requirements is a longstanding challenge.

In response, we developed the DFDM, a geometrically nonlinear, fully differentiable method to solve inverse form-finding problems on structures modeled as pin-jointed bar systems and subject to shape-dependent loads. By leveraging AD and mechanics-specific adjoints, the DFDM consolidates into a general approach for the computation of funicular geometries that comply with architectural, manufacturing, and structural objectives, accelerating the design of efficient yet feasible shapes for lightweight structures in the large-displacement regime. In combination with gradient-based optimization, the DFDM delivers efficient yet viable structural forms three to four orders of magnitude faster than gradient-free algorithms. The developed adjoints, blending analytical and automatic derivatives, further enhance the DFDM's speed by reducing optimization wall times between two and ten times with respect to standard AD. They illustrate the value of exploiting the computation graph of the DFDM's mechanical model to develop domain-specific AD implementations for faster design. The end-to-end differentiability of the DFDM has supported the design of a wide array of structures, including pneumatic tensile systems, compressive masonry shells, and unstrained gridshells, each with a distinct task specification, loss function, and high-dimensional parametrization. With load-finding and support-finding, we have discovered innovative funicular solutions to repair a damaged hemispherical dome, and to design a bending-free lattice roof compatible with planar glass cladding and that lowers the thrust it exerts on the buildings supporting it.

The work presented in this paper extends the capabilities of the classic FDM for differentiable programming. More broadly, the DFDM contributes to the development of differentiable form-finding methods that merge robust mechanical models and modern deep learning tools to expedite the design of material-minimizing structures built to serve human needs.

4.1. Limitations

A limitation of this work is that the presented design tasks only consider a single load case, rather than multiple load scenarios. This simplification is common in form-finding at the conceptual stage, although selecting the most representative load case for initial design remains a fundamental issue in the field [185]. Here, we focus on the load case we have the most certainty about, self-weight, except for the pneumatic design task, where we only accounted for the inflation pressure. The DFDM nonetheless provides a solid substrate for downstream analysis and refinement for additional load cases. A second challenge is that gradient-based optimization typically converges to a local optimum in non-convex loss functions, and not to a globally optimal solution. This optimum further depends on the initial choice of parameter values. As all our experiments demonstrate, a local optimum is sufficient to solve a design task accurately if the parameters are sensibly initialized. Another drawback is that the calibration of the goal weights in the loss function is ad hoc and requires some experimentation. This issue could be resolved by utilizing meta-learning, exploiting the differentiability of the DFDM to optimize the weights at the same time as the loss function value [186,187], or with new gradient-based optimization algorithms that tune simulation parameters in a conflict-free way towards a local optimum shared by all the terms in the loss function [188].

4.2. Future work

Looking ahead, the DFDM could be extended from discrete to continuous structures to allow for direct design of membranes and shells, as other works have done for the standard FDM [25,35,42,43]. The structural mechanics foundation of the DFDM lays the blueprint for this extension. While we have focused on solving problems with multiple goals and constraints, we aggregate all of them in a single loss function, so extending the developed shape optimization framework to a proper multi-objective optimization setting that offers a Pareto front of solutions would be a next step. The convergence rate of our approach could be further improved by using second-order methods that extract curvature information of the loss landscape, although obtaining Hessians can be more computationally expensive per step than computing gradients. Delineating trade-offs between the two approaches would be important to determine which one is more suitable for a particular task. A last promising direction is the integration of differentiable structural simulation methods like the DFDM into neural networks to further accelerate design optimization. In the deep learning era, differen-

tiable simulation promises to imbue physics knowledge directly into artificial intelligence systems [93,96,189] to create trustworthy surrogate models for even faster design.

CRedit authorship contribution statement

Rafael Pastrana: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Conceptualization; **Deniz Oktay:** Writing – original draft, Software, Methodology, Investigation; **Kai-Uwe Bletzinger:** Writing – review & editing, Validation, Methodology; **Ryan P. Adams:** Writing – review & editing, Methodology, Funding acquisition; **Sigrid Adriaenssens:** Writing – review & editing, Supervision, Funding acquisition.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work has been supported by the U.S. National Science Foundation under grant OAC-2118201 and the Institute for Data-Driven Dynamical Design (ID4). We thank Prof. Robin Oval from the National School of Bridges and Roads in France for providing the geometry and the layout of the British Museum's gridshell, and Prof. Francesco Marmo from the University of Naples for pointing us to the reinforced arch method for masonry retrofitting.

Appendix A. Numerical validation

A.1. Analytical benchmark

We first validate the DFDM with an inverse problem with a known analytical solution [105,107]. As illustrated by Fig. A.1a, the problem consists of finding the rise $z_{\max}$ of a planar arch of fixed horizontal projection, modeled as a pin-jointed bar system, and subject to a distributed load ρ , such that it minimizes its load path energy Ω :

$$\Omega = \sum_{j=1}^{n_e} |f_j| l_j \quad (\text{A.1})$$

The energy Ω corresponds to the sum of the product of the absolute value of the individual edge forces $|f_j|$ and their lengths l_j . A minimum-energy solution corresponds to a minimum-volume structure under a fully stressed design [159,190–192]. The two vertices at the extremes of the arch are supported and separated by the span length d . The other vertices are free to move vertically.

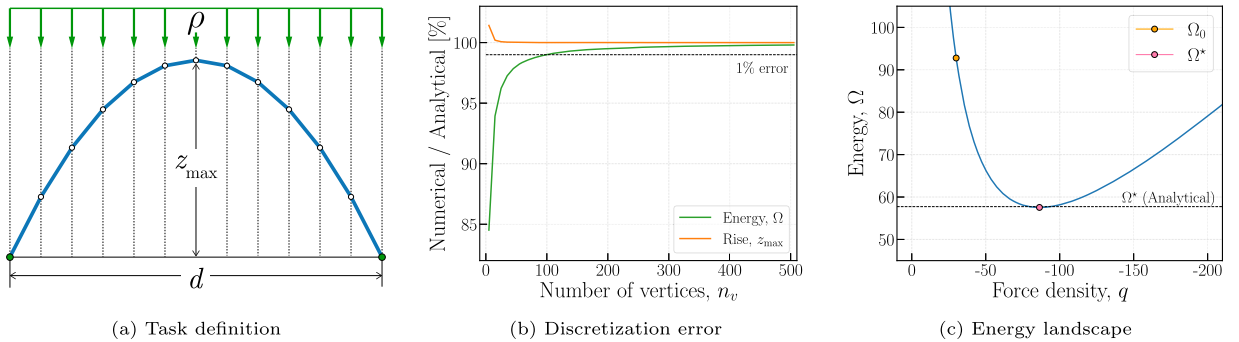


Fig. A.1. Energy minimization of an arch. **Fig. A.1a:** We look for the optimal rise $z_{\max}$ of a compression-only arch with span length d that minimizes the energy Ω needed to resist a uniformly distributed vertical load ρ . The position of the arch vertices projected to the ground plane is constrained to remain evenly spaced on the horizontal axis. **Fig. A.1b:** The DFDM meets the analytical solution of this inverse problem within a 1% error once the arch discretization reaches $n_v = 100$. **Fig. A.1c:** Plotting the energy landscape confirms the agreement between the analytical reference and the numerical solution at the optimal point.

The analytical equations for the optimal load path Ω^* and optimal rise $z_{\max}^*$ of a continuous arch under the above-mentioned boundary conditions are:

$$\Omega^* = \frac{\rho d^2}{\sqrt{3}} \quad (\text{A.2})$$

$$z_{\max}^* = \frac{\sqrt{3} d}{4} \quad (\text{A.3})$$

To solve this problem, we take Eq. A.1 as the sole goal function, plug it into Eq. 35, and minimize it numerically. The structure ought to withstand the applied loads under compressive internal forces to be a valid arch. We thus search for a set of negative force densities that minimize Eq. A.1. Due to the horizontal projection constraint, the optimization problem can be reduced to one variable [135,193,194]. Hence, we look for a single, optimal force density value q_0 that we then apply to all the arch edges. Then, we can obtain the edge forces and lengths with Eqs. 25 and 24, respectively.

We evaluate the sensitivity of the optimal values of Ω^* and $z_{\max}^*$ to the discretization of the arch. To this end, we vary the total number of vertices n_v in the arch, increasing them linearly from 5 to 500. For each variation of n_v , we set the applied load $\rho = 1$, the span length $d = 10$ meters, and initialize the force density to $q_0 = -10$. We convert ρ into individual vertex loads $\mathbf{p}_i$ for each vertex i with the expression $\mathbf{p}_i = \rho d/n_v$. Fig. A.1b shows the ratio between the output and the benchmark values. We match the analytical version of Ω^* within a 1% error after $n_v = 100$. The error gap continuously decreases as the arch discretization grows, reaching a minimal deviation of 0.02% at $n_v = 500$. We also meet the analytical value of $z_{\max}^*$ within a 0.1% difference with fewer vertices than required for the energy, and maintain this accuracy at finer discretizations. These results are in excellent agreement with the values reported for a similar numerical experiment in [107]. Fig. A.1c further confirms the proximity of the numerically optimized value of Ω^* to the analytical reference. The DFDM is able to descend from an initial energy Ω_0 and find an optimal value Ω^* at the basin of the one-dimensional energy landscape of Eq. A.1.

A.2. Accurate optimization under pressure loads

The validation of the DFDM continues by assessing the accuracy of the specialized adjoint for optimizing funicular geometry subject to shape-dependent loads developed in Section 2.8. We reproduce the simulation of a pneumatic creased dome presented in [108] for that purpose. They simulated the pneumatic structure with the combinatorial equilibrium modeling (CEM) framework [27], a form-finding method that makes similar assumptions as the FDM, and with the nonlinear finite element method (FEM) on Abaqus [195]. In the FEM approach, the reference's authors employ Timoshenko beams with six degrees of freedom on the compressive elements, assuming they have bending stiffness; and, like us, they use truss elements with only one degree of freedom to simulate the tensile elements. We compare the DFDM to both approaches and demonstrate the validity of the formulation exposed in our work.

Fig. A.2 presents the setup. The dome is represented as a bar system with 241 vertices and 480 edges. The edges are aggregated into three groups: 10 radial struts, 10 radial cables, and 11 hoop cables. Each radial group contains 12 edges. Both struts and cables alternate radially along the polygonal base of the dome, displacing in and out of the base with a variation of 0.1 m in elevation. Two different spans between opposite ends of the dome result from this alternation: $d_1 = 49.72$ m between struts and $d_2 = 45.81$ m between cables. The dome is anchored to the ground at its base, and inflated with a pressure load of $\psi = 0.5$ kN/m² normal to the dome's surface. We discretize the pressure load as a follower load on the vertices of the dome via Eq. 28. Self-weight is disregarded. The inflated geometry of the dome is conditioned on two constraints: a constant axial force on the hoop cables of $\hat{f} = 17$ kN, and fixed lengths on the radial elements of $\hat{l} = 3$ m. We recast these constraints as goals and minimize the sum of the squared differences over the edges of the radial struts and cables, for the force and length goals, respectively. The force densities are the optimization variables.

We match the axial forces of all the edges of the structure within a 1.0% relative error on average for both simulation types (Table A.1). Specifically, the relative error is 0.87% for the CEM and 1% for the FEM, and the largest error is 1.43 kN in absolute terms. The DFDM is most accurate in capturing the forces of the radial and the hoop cables in tension. The errors on the forces of the radial struts are larger, but still only 2% on average, ostensibly because the compressive elements have no bending stiffness in our model, but they do in the FEM. The dome's geometry computed by the DFDM attains a maximum height of $z_{\max} = 21.48$ m (Fig. A.2), and the largest pairwise distance between the DFDM and the CEM geometry presented in [108] is 1.16%, relative to the dome's longest span. The length of the radial elements is within 0.04% of the target $\hat{l}$ in expectation, i.e., within a centimeter of the target length on average. The close alignment of the forces and the geometry of the creased dome with those produced by a more detailed structural simulation proves that the DFDM accurately captures the behavior of pneumatic structures under pressure loads, validating its accuracy for design optimization.

A.3. Fast sparse optimization

We benchmark linear solvers on CPU and GPU to pick a device that accelerates design optimization with the DFDM. Specifically, we run a comparative analysis where we solve Eq. 23 with direct sparse and direct dense methods on both devices, as the linear solver is called multiple times during the forward and backward passes (e.g., Eq. 47). We then quantify how decreasing the sparsity and increasing the size of the stiffness matrix $\mathbf{K} = \mathbf{C}_u^T \mathbf{Q} \mathbf{C}_u$ impact wall time.

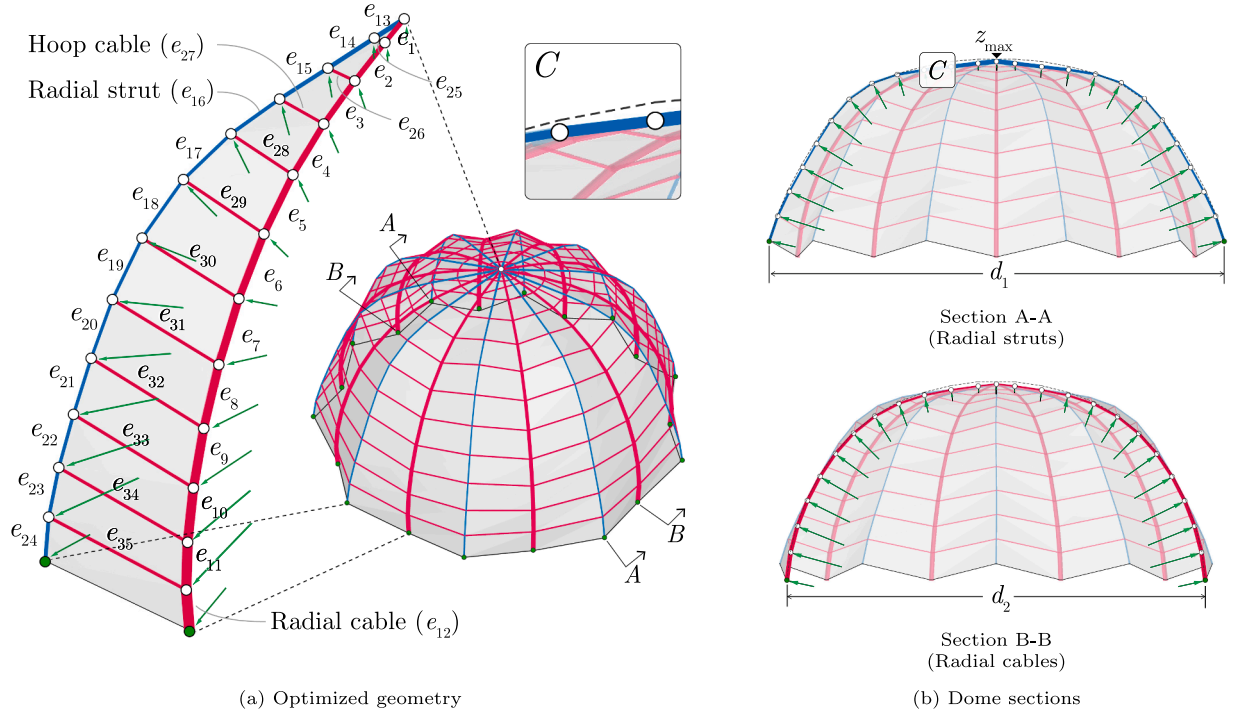


Fig. A.2. Pneumatic creased dome with mixed tension-compression axial forces. Compressive radial struts coexist with tensile radial cables and hoop cables. The inflated dome has a polygonal base of 16 sides, and a maximum height of $z_{\max} = 21.48$ m. The vertex loads representing the internal pressure are perpendicular to the discrete inflated geometry. They are automatically updated during simulation and optimization. The callout shows the section of the dome with the largest difference between the geometry produced by the DFDM and the one output by the CEM reference.

We study three representative structures: an arch (Fig. A.3a), a saddle (Fig. A.3b), and a high-point net (Fig. A.3c). The valence of a free vertex, the number of edges connected to a vertex, is different per structure. In the arch, the valence is two. In the saddle, the valence is four, and in the high-point net, six. As the valence of a free vertex increases, the sparsity of $\mathbf{K}$ decreases. Furthermore, we recursively subdivide the graphs representing the three structures to monotonically increase the order of $\mathbf{K}$, i.e., the number of columns and rows, from $n_u = 10^0$ to $n_u = 10^4$. We run these numerical experiments on an Intel Xeon CPU with eight cores clocked at 2.2 GHz. On GPU, we use an NVIDIA Tesla T4. We run every configuration ten times in sequence using double-precision, floating-point operations, and report averages of the results.

A particular subtlety of solving form-finding problems with the FDM is that the right-hand side of the linear system in Eq. 23 lives in three independent spatial dimensions with $\mathbb{R}^{n_u}$ degrees of freedom each. Dense and sparse solvers on CPU and dense solvers on GPU all support batched linear systems, solving multiple right-hand sides in $\mathbb{R}^{n_u}$ with a single factorization on the left-hand side for computational speedup. Therefore, rather than constructing the full block-diagonal connectivity matrix for three spatial dimensions $\mathbf{C} = \mathbf{I}_3 \otimes \tilde{\mathbf{C}} \in \mathbb{R}^{3n_u \times 3n_u}$ and then building a large $\mathbf{K} \in \mathbb{R}^{3n_u \times 3n_u}$, we only use one of the diagonal blocks of the connectivity matrix so that $\mathbf{C} \in \mathbb{R}^{n_u \times n_u}$ and arrive at a more compact stiffness matrix $\mathbf{K} \in \mathbb{R}^{n_u \times n_u}$. Then, we solve Eq. 23 in batched form via $\mathbf{X} = \mathbf{K}^{-1}\mathbf{B}$, where $\mathbf{X} = \text{matrix}(\mathbf{x}) \in \mathbb{R}^{n_u \times 3}$ and $\mathbf{B} = \text{matrix}(\mathbf{b}) \in \mathbb{R}^{n_u \times 3}$. However, the GPU sparse solver we have access to (from cuSOLVER [196]) does not support batching, so we have to construct the full block-diagonal $\mathbf{C} \in \mathbb{R}^{3n_u \times 3n_u}$ for our experiments on this device. To account for this discrepancy, we report sparse GPU wall times as measured values and as hypothetical values. The latter assumes a batched implementation that is three times faster than the measured values. The computation time needed to assemble both left-hand and right-hand sides into the right format is considered in the plots in Fig. A.3.

Contrary to what one might expect in our modern day of massive GPU acceleration, the sparse CPU solver is the fastest in all problem instances with matrix orders greater than $n_u = 10^2$. GPUs excel at highly parallel tasks, such as matrix multiplication or similar vectorized operations. In contrast, direct sparse linear solvers are typically serial due to their iterative nature. The sparser the matrix $\mathbf{K}$, the less parallelism available for the GPU to exploit. This is one of the reasons why the gap between the sparse CPU and sparse GPU solvers becomes larger as the sparsity of $\mathbf{K}$ increases from the high-net (Fig. A.3c) to the arch (Fig. A.3a). Conversely, the trends in Fig. A.3 suggest that solving Eq. 23 with a sparse solver on GPU could eventually be faster than on CPU for large structures ($n_u > 10^4$) if the valence of the unsupported vertices grows to values larger than six (as in the high-point net), and hence $\mathbf{K}$ becomes less sparse. However, such high valences are rare and often impractical in structural design for manufacturing reasons.

Another useful note is that the dense CPU and dense GPU solvers have comparable performance characteristics in all three problem instances. This is because in the dense solver case, the sparsity pattern of the problem makes no difference in the solving speed, and the solve time is purely governed by the matrix size. Because of the computational overhead required for sparse operations (bookkeeping

Table A.1

Axial forces in a pneumatic creased dome and errors relative to the CEM and the FEM.

Group	Edge, e_j	Force [kN]			Relative error [%]	
		CEM [108]	FEM [108]	DFDM	To CEM	To FEM
Radial cables	1	40.00	39.97	40.05	0.13	0.21
	2	45.59	45.56	44.90	1.52	1.46
	3	51.27	51.21	50.46	1.59	1.47
	4	57.13	57.05	56.35	1.36	1.22
	5	63.40	63.30	62.82	0.92	0.76
	6	70.09	69.99	70.02	0.11	0.04
	7	76.97	76.84	77.02	0.07	0.23
	8	83.85	83.70	83.73	0.15	0.03
	9	90.76	90.62	90.50	0.29	0.13
	10	97.79	97.67	97.32	0.48	0.36
	11	105.06	104.93	104.28	0.74	0.62
	12	112.65	112.49	111.62	0.91	0.77
Radial struts	13	-40.00	-39.75	-41.18	2.96	3.61
	14	-35.10	-34.86	-35.47	1.07	1.76
	15	-30.26	-30.06	-30.45	0.64	1.31
	16	-25.63	-25.45	-25.55	0.30	0.41
	17	-21.57	-21.41	-21.08	2.28	1.55
	18	-18.75	-18.60	-18.44	1.65	0.85
	19	-17.64	-17.50	-17.80	0.90	1.71
	20	-17.99	-17.85	-18.48	2.72	3.52
	21	-19.23	-19.09	-19.89	3.43	4.18
	22	-21.07	-20.94	-21.55	2.26	2.90
	23	-23.61	-23.47	-23.51	0.41	0.19
	24	-27.17	-27.02	-26.45	2.66	2.12
Hoop cables	25	17.00	16.97	17.03	0.17	0.35
	26	17.00	16.90	17.01	0.07	0.66
	27	17.00	16.94	17.01	0.03	0.39
	28	17.00	16.96	17.00	0.02	0.21
	29	17.00	16.95	16.98	0.12	0.17
	30	17.00	16.93	16.97	0.16	0.25
	31	17.00	16.95	16.99	0.07	0.23
	32	17.00	17.05	16.98	0.09	0.38
	33	17.00	17.04	17.00	0.01	0.22
	34	17.00	16.97	17.00	0.03	0.15
	35	17.00	16.93	17.00	0.01	0.42
Average					0.87	1.00

of indices and data arrays), for the smallest sizes in each problem instance, using the dense CPU solver is faster than the sparse CPU solver up to $n_u = 10^2$, where we see a transition point and sparse methods begin dominating.

A.4. Correctness of the analytical adjoints

This section validates the accuracy of the proposed analytical adjoints (ADA) of the linear solver (Section 2.7) and the URS (Section 2.8). To that end, we apply the Taylor remainder convergence test, a rigorous technique for gradient verification [197]. The test assesses the convergence rate of the Taylor remainder of the loss $\mathcal{L}$ at given parameters θ as the size of a perturbation decreases.

The second-order Taylor remainder ρ is defined as:

$$\rho = |\mathcal{L}(\theta + \mu \mathbf{v}) - \mathcal{L}(\theta) - \mu \mathbf{v}^\top \nabla_\theta \mathcal{L}(\theta)| \quad (\text{A.4})$$

where $\nabla_\theta \mathcal{L}(\theta)$ is the evaluated gradient, μ is the step size, $\mathbf{v}$ is a perturbation vector, and $|\cdot|$ denotes the absolute value. For a gradient formulation to be mathematically correct, the remainder ρ must converge quadratically towards zero as μ is halved successively; i.e., $\mathcal{O}(\mu^2)$.

We test the gradients generated via ADA in the pneumatic design (Section 3.2), load-finding (Section 3.3), and support-finding (Section 3.4) tasks. Every task pairs a different loss with either one or both proposed adjoints, offering a broad assessment of the suitability of our method for design optimization. We evaluate the gradient at the initial values of θ (the gradient corresponding to the first optimization step) and perform Taylor tests along ten random vectors $\mathbf{v}$ per task, starting at $\mu = 10^{-2}$. The vectors are sampled

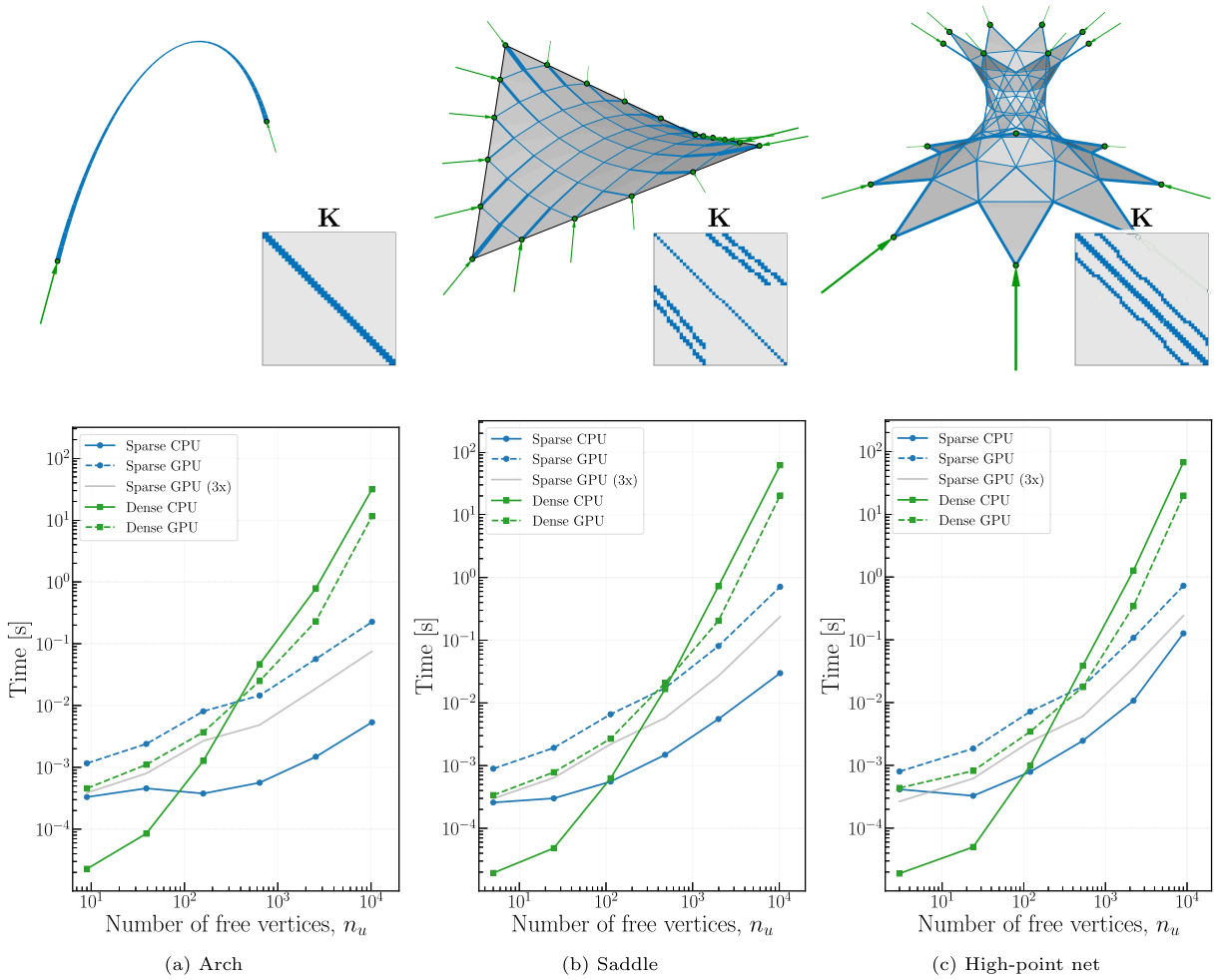


Fig. A.3. Computational performance of linear solvers. We quantify the time elapsed to solve Eq. 23 on two hardware devices: CPU and GPU. We compare dense and sparse solvers (bottom row) on three structures (top row) with stiffness matrices $\mathbf{K}$ of varying sparsity levels. The x-axis shows the order of the matrix, which is equal to the number of free vertices in the structure n_u , and the y-axis displays the wall time in seconds elapsed to solve a linear system.

from a uniform distribution bounded between 0 and 1, and then normalized to be unit-length. We report average values of ρ for each task and compare with a generic AD baseline.

For all tasks, the Taylor remainder ρ of the ADA-generated gradients exhibits consistent second-order decay, on par with the AD baseline, until round-off dominates (Fig. A.4). Over step sizes between 10^{-2} and 10^{-4} , ρ decreases with an average slope equal to 2 within three decimal places for load-finding and support-finding, and within two decimal places for the pneumatic case. Further deviations from the theoretical slope arise for small remainders on the order of $\rho < 10^{-11}$ in all cases. The larger deviation observed for the URS adjoint in the pneumatic task with respect to the theoretical convergence slope and the AD data can be reduced by tightening the URS convergence tolerance η [198]. In sum, the quadratic convergence of the Taylor remainders offers strong evidence of the correctness and accuracy of our method for gradient computation.

A.5. Contribution of the analytical adjoints to optimization speedup

In this section, we quantify how the linear solver and the URS adjoints (ADA) contribute to computational performance gains over general-purpose AD with a parametric study. The test case is a tensile pneumatic structure with a fixed square base subject to an internal pressure load of $\psi = 1 \text{ kN/m}^2$ (Fig. A.5a). The optimization problem consists of finding a low-energy funicular geometry for the structure by minimizing Eq. A.1, with the force densities $\mathbf{q}$ as the optimization parameters. For the study, we refine the discretization of the structure by progressively increasing the number of edges n_e in the graph G modeling the pneumatic system from 60 to 3280, where the number of optimization parameters equals the number of edges, $n_p = n_e$. Additionally, we vary the number of URS iterations from 10 to 100. This iteration budget $t_{\max}$ is equivalent to the number of times the FDM is executed to arrive at a fixed point of Eq. 34. Next, we measure the wall time required to compute the gradient with three ADA modalities: (i) AD with the

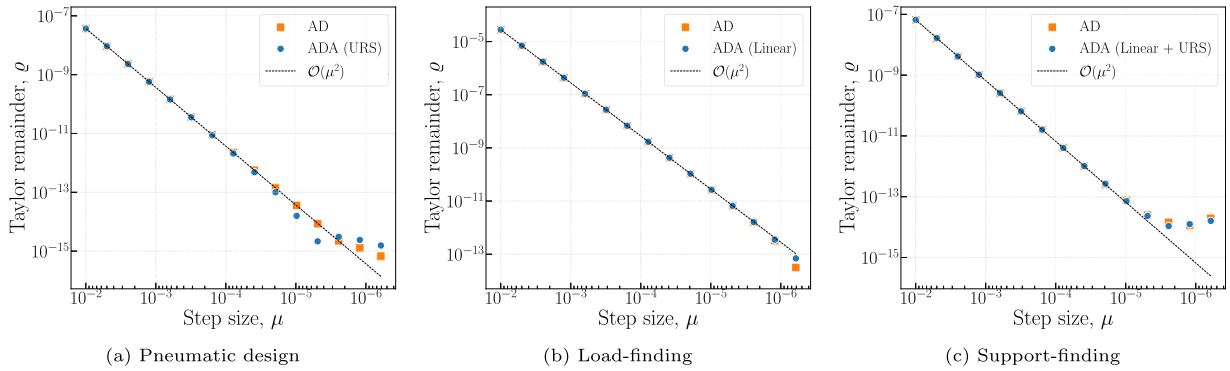


Fig. A.4. Taylor remainder convergence test to verify the correctness of the proposed gradient formulation combining AD with analytical derivatives (ADA) for the linear solver and the URS. The test is performed on three design tasks presented in Section 3. The convergence rate of the Taylor remainders proves the accuracy of the gradients supplied by our method.

linear adjoint, (ii) AD with the URS adjoint, and (iii) AD with both. We pair a sparse linear solver to resolve Eqs. 23, 47, and 53 in cases (i) and (iii). The wall times are averaged over ten runs, considering the cost of both forward and backward passes in each run. The heatmaps in Fig. A.5b report the resulting computational boosts relative to generic AD:

$$\text{Boost} = \frac{\text{Time}(\nabla_{\theta} \mathcal{L}(\theta))_{\text{AD}}}{\text{Time}(\nabla_{\theta} \mathcal{L}(\theta))_{\text{ADA}}} \quad (\text{A.5})$$

Computing gradients via ADA with the linear solver adjoint offers boosts per gradient evaluation of up to 1.8 times in this task. Performance is effectively insensitive to the number of URS iterations since the average speedup is 1.2 times across $t_{\max}$, but improves proportionally with larger n_p . However, on small problems with only a few hundred parameters, AD is faster. This behavior is consistent with the sparsity experiments presented in A.3, where solving Eqs. 23 and 47 with a sparse stiffness matrix $\mathbf{K}$ pays off only in problems of moderate size. Another potential limitation is our implementation: in the backward pass we reuse a single factorization of $\mathbf{K}$ to solve multiple right-hand sides in Eq. 53, but in the forward URS loop we cannot cache the matrix, so we refactorize it at every iteration. Better caching could narrow the gap.

In contrast, applying ADA with the URS adjoint is consistently faster than AD regardless of problem size and iteration count. The computational performance gains range from 1.4 times to 12.2 times. While both $t_{\max}$ and n_p condition these acceleration factors, the speedups are more attractive as the number of URS iterations increases: the average speedup across discretizations rises from 4.6 at 20 iterations, to 7.8 at 60 iterations, and 9.5 at 100 iterations. The URS adjoint is therefore the primary driver for optimization acceleration since it collapses an unrolled fixed-point trace where the linear solver is invoked multiple times. And the longer this solver has to iterate to find a fixed point, the greater the performance boost during backpropagation. Pairing both adjoints in ADA can underperform AD when evaluating gradients if discretizations are coarse (small n_p) or the URS iterates minimally (low $t_{\max}$). This behavior is inherited from the numerical overhead accrued by the linear adjoint in small optimization problems. Nonetheless, activating both adjoints is justified when n_p is on the order of $n_p > 10^3$ in longer URS runs. Not only that, but utilizing both adjoints simultaneously produces the greatest computational boosts in excess of 13 times compared to plain AD and improves over the URS adjoint best-case scenario. This trend suggests that the computational gains over AD yielded by combining both adjoints are greater at finer discretizations.

In summary, employing the URS adjoint by default offers the most significant computational speedups for gradient computation in inverse form-finding tasks involving shape-dependent loads. Adding the linear solver adjoint boosts backpropagation performance when the parameter count is high and the URS necessitates several iterations to converge in the forward pass. The developed analytical adjoints proved beneficial in all the tasks presented in Section 3 because the optimization setups surpass the minimum URS iteration count and discretization thresholds in their respective configurations.

A.6. Local robustness to goal weight perturbations

This work casts funicular design as the minimization of a weighted sum of goals g_i (Eq. 35) using gradient-based optimization. The weights α_i determine the priority of each goal, and the computed funicular geometry $\mathbf{x}$ is conditioned by the scale of these weights. Therefore, judicious weight tuning is oftentimes necessary, especially when several objectives conflict or quantify metrics expressed at different magnitude scales. Once reasonable weights are chosen, a natural question to ask is how sensitive the optimization solution is to small weight changes. This information could be valuable, too, to calibrate how finely to search in weight space.

In this section, we evaluate the local sensitivity of the loss and the optimized geometry to the choice of goal weights in the gridshell design task featuring support-finding in Section 3.4. This multi-objective task consists of computing funicular geometry for a compression-only gridshell with flat panels that exerts zero thrust on the north and south sides of its outer perimeter. The task specification contemplates four goals. Here, we focus on systematically varying the weights of the principal design drivers, reaction force minimization g_1 and panel planarity g_2 , by $\pm 20\%$ to examine optimization robustness.

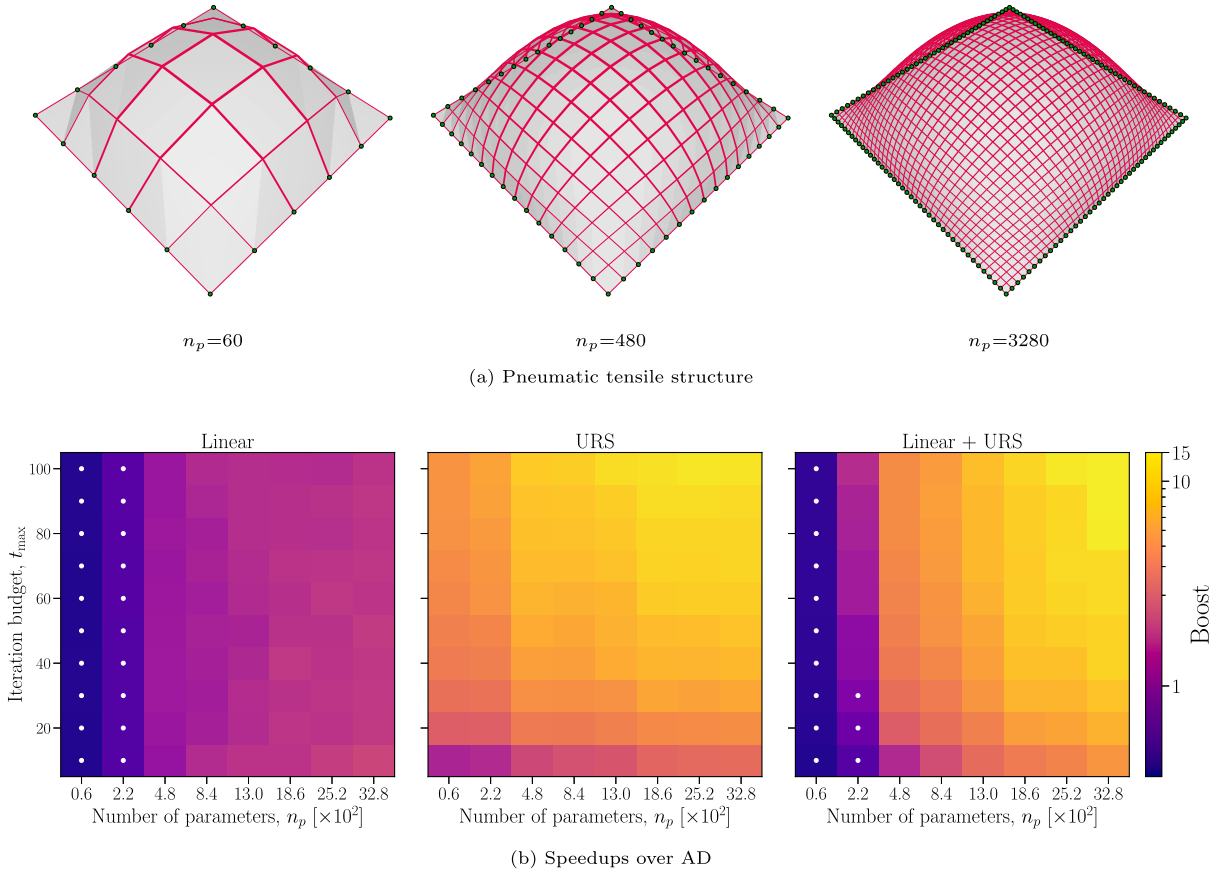


Fig. A.5. Computational boost provided by the linear solver and the URS adjoints relative to generic AD for varying parameter count n_p and URS iteration budget $t_{\max}$ in a pneumatic design task illustrated in Fig. A.5a. The tiles marked with a dot in Fig. A.5b indicate configurations where computing a gradient with general-purpose AD is faster than with the proposed mixed differentiation approach. Coupling AD with the developed adjoints is most effective for large optimization problems or high URS iteration count.

Starting from the reference optimal geometry $\mathbf{x}^*(\alpha_i)$ in Fig. 16c, we perturb one weight at a time:

$$\alpha'_i = \alpha_i(1 + \delta_i), \quad \delta_i \in \{+0.2, +0.1, -0.1, -0.2\} \quad (\text{A.6})$$

For each perturbed weight α'_i , we re-run the full optimization from the same initial parameters θ and identical optimization settings, and compare the generated geometry $\mathbf{x}^*(\alpha'_i)$ to the reference solution.

We report relative changes $\Delta\mathcal{L}$ in the loss:

$$\Delta\mathcal{L}(\alpha'_i) = \frac{\mathcal{L}(\mathbf{x}^*(\alpha'_i)) - \mathcal{L}(\mathbf{x}^*(\alpha_i))}{\mathcal{L}(\mathbf{x}^*(\alpha_i))} \quad (\text{A.7})$$

and apply an equivalent equation to monitor other metrics, such as fluctuations in goal values Δg_i or average panel skewness $\Delta\phi_{\text{avg}}$. To measure geometric variations, we compute the Hausdorff distance d_H between the new designs and the reference, normalized by the diagonal length of the reference geometry's bounding box.

Geometric and loss changes are minor across all perturbations. The loss varies by at most 3% and the geometry by at most 1%, even if we shift the goal weights by 20% (Table A.2). Most variations appear in the goals g_i . They respond inversely to their own weight; for example, increasing α_1 lowers g_1 and vice versa. But such an enhancement comes generally at the expense of deteriorating performance on the other goal, suggesting the reaction minimization and planarity maximization objectives are in conflict. The magnitude of these trade-offs remains unclear due to the nonlinear goal formulations (Eqs. 65 and 66) and the interactions with the fairness and total area goals in this task, and warrants dedicated future analysis.

Nevertheless, variations in local goal values translate to small changes in practical global metrics in this task. For instance, g_1 can rise by 20% but the reactions only experience half of that change. Similarly, even if g_2 and ϕ_{avg} deteriorate by 30% and 35%, respectively, panels already near zero skewness ($\leq \phi_{\text{avg}} = 0.2\%$) stay flat and the share of flat panels with $\phi \leq 1\%$ in the new funicular geometries shifts by 2.6% at most. Fig. A.6 confirms this trend by displaying the reference geometry and the solutions for the perturbation cases $\delta_1 = +20\%$ and $\delta_2 = +20\%$. The main visible changes planarity-wise concentrate on the north side of the structure

Table A.2

Optimization sensitivity to perturbations δ_i in the weights α_i scaling the reaction reduction goal g_1 and the panel planarity goal g_2 in the support-finding task for gridshell planarization.

δ_1	δ_2	$\Delta \mathcal{L} \downarrow$	$\Delta g_1 \downarrow$	$\Delta g_2 \downarrow$	$\Delta t /w \downarrow$	$\Delta \phi_{\text{avg}} \downarrow$	$\Delta \text{Flat} \leq 1\% \downarrow$	$d_H \downarrow$
[%]	[%]	[%]	[%]	[%]	[%]	[%]	[%]	[%]
+20	–	2.5	–11.9	1.7	–6.0	5.0	0.1	0.5
+10	–	0.7	–8.6	0.5	–4.1	0.0	0.4	0.5
–10	–	–0.8	9.8	–4.6	0.4	4.7	–5.0	–1.1
–20	–	–3.0	19.7	–11.6	9.7	–10.0	0.8	0.6
–	+20	2.1	–5.9	–17.9	3.3	–20.0	1.8	0.9
–	+10	2.0	8.2	–7.7	4.2	–5.0	–0.7	1.0
–	–10	–0.8	–0.4	16.8	–0.2	20.0	–1.1	0.6
–	–20	–2.7	–6.7	29.9	–2.8	35.0	–2.6	0.9

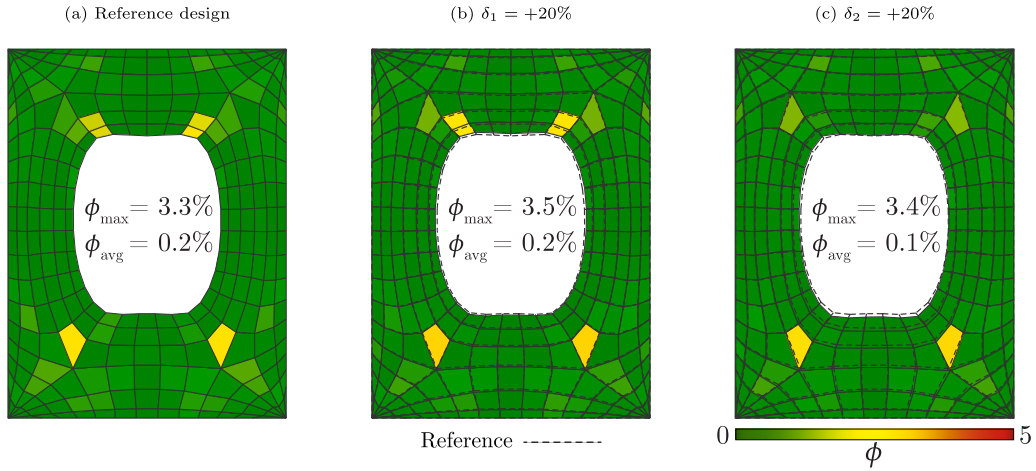


Fig. A.6. Funicular gridshell designs with flat panels generated via support-finding. The gridshell geometries are computed after perturbing the weight α_i of the reaction minimization goal g_1 and the planarity goal g_2 in this task by $\delta_i = +20\%$. The geometric changes arising from such perturbations are minor.

in the vicinity of the inner boundary. The overlay between designs provides further evidence that the resulting geometric alterations are indeed subtle. While balancing multiple goals can make weight selection challenging for structural design, once the weights are in an adequate range the resulting funicular geometries appear numerically stable and only mildly sensitive to small weight perturbations.

```

1 import jax
2 import jax.numpy as jnp
3
4
5 graph = create_graph()
6 c_free, c_fixed = connectivities(graph)
7 theta, x0 = parameters_init(graph)
8 x_hat = target_shape(graph)
9
10 def fdm(theta, x0=None):
11     """The linear force density method"""
12     q, p, s = parameters_split(theta)
13     # stiffness matrix
14     k = c_free.T @ q @ c_free
15     # load matrix
16     d = c_free.T @ q @ c_fixed @ s
17     if x0 is not None:
18         p = loads_shape(p, x0, graph)
19     b = p - d
20     # compute funicular geometry
21     return jnp.linalg.solve(k, b)
22
23 def fdm_iterative(theta, x0):
24     """Iterative force density method using the URS algorithm"""
25     for _ in range(100):
26         x = fdm(theta, x0)
27         if is_fixedpoint(x, x0):
28             break
29         x0 = x
30     return x
31
32 def fdm_inverse(theta, mu, loss):
33     """Inverse force density method via gradient descent"""
34     gradient = jax.grad(loss)
35     for _ in range(500):
36         loss_val = loss(theta)
37         gradient_val = gradient(theta)
38         theta = theta - mu * gradient_val
39         if is_optimal(loss_val, gradient_val):
40             break
41     return theta
42
43 def loss(theta):
44     """Loss function for a design specification"""
45     x = fdm_iterative(theta, x0)
46     return jnp.mean(jnp.square(x - x_hat))
47
48 # Solve design task
49 theta_star = fdm_inverse(theta, mu=1.0e-1, loss=loss)
50 print(f"{theta_star}")

```

Fig. B.7. Python code to solve a shape matching task with the DFDM and JAX [137].

Appendix B. Implementation

We provide a minimalistic example of a DFDM implementation, and refer the reader to the code available at reference [137] for a more robust version that includes the adjoints of the linear solver and the URS developed in this work. We utilize JAX [136] in either case, a differentiable programming library written in Python that combines high-performance array computing with automatic differentiation and just-in-time (JIT) compilation for rapid execution on different hardware devices. In this example, we solve a shape matching task, similar to what we present in Section 3.1. The code for this task, shown in Fig. B.7, is organized into three main blocks.

In the first block, between lines 1 and 8, we import `jax` and its submodule `jax.numpy` for performing linear algebra operations. Then, we define the bar system and the numerical parameters for simulation. The function `create_graph()` generates the graph G encoding the system's topology, while `connectivities()` assembles the connectivity matrices C_u and C_s based on G . These matrices are referenced in the code as `c_free` and `c_fixed`, respectively. Next, the function `parameters_init()` estimates initial values for the FDM parameters θ : the force densities $\mathbf{q}$, the applied loads $\mathbf{p}$, and the support positions $\mathbf{s}$. The same function provides an initial

guess $\mathbf{x}_0$ of the geometry, which is only relevant for shape-dependent loads. In addition, we generate the target shape $\hat{\mathbf{x}}$ to match in line 8. The dimensions of all the arrays created in this first block are compatible and governed by the discretization of G .

In the second block, we implement the three main elements of the DFDM. The linear FDM, encapsulated in the function `fdm()`, computes a funicular geometry $\mathbf{x}$ for input parameters θ after solving a linear system (Eq. 23). We unpack the parameters into $\mathbf{q}$, $\mathbf{p}$, and $\mathbf{s}$ in line 12, and reshape $\mathbf{q}$ into the diagonal matrix $\mathbf{Q}$ in the same step. The stiffness matrix $\mathbf{K}$ and the load matrix $\mathbf{b}$ are calculated next as intermediate steps en route to obtaining $\mathbf{x}$ with a dense linear solver in line 21. Note the close similarity between the code and Eqs. 20–23 due to the use of the compact linear algebra primitives provided by `jax.numpy`. Shape-dependent loads are generated in line 18, depending on whether $\mathbf{x}_0$ is provided or not. If that is the case, the loads $\mathbf{p}$ are updated and turned into vertex loads by `loads_shape()`. This function generally encodes Eq. 28, but can be replaced with some other equation if the loads have a different type of dependency on the geometry.

Next, we implement the iterative FDM in the body of `fdm_iterative()` to arrive at an instance of $\mathbf{x}$ in equilibrium with the shape-dependent actions. The shape computation follows the URS, solving a sequence of linear equilibrium problems (see Eq. 32) until convergence to a fixed point. Here, we set the maximum number of URS steps to a hundred. The convergence is measured by Eq. 34 for a given tolerance value η , and is implemented in line 27 as `is_fixedpoint()`.

Lastly, `fdm_inverse()` performs gradient descent to minimize a task-specific loss function $\mathcal{L}$. The routine is loss-agnostic: one supplies a scalar-valued `loss()` implementing $\mathcal{L}$ that is JAX-compatible, and JAX's `jax.grad()` automatically generates another function that evaluates the gradient $\nabla_{\theta}\mathcal{L}(\theta)$ at an instance of θ using reverse-mode AD. No manual derivations are required. Backpropagation works through the control flow in the code (e.g., the loops and if statements). Each gradient descent iteration then evaluates the loss and its gradient, updates θ with the step size μ (Eq. 37), and stops if `is_optimal()` detects a small loss or gradient norm based on the optimization thresholds ϵ and κ . We arbitrarily cap the gradient descent iterations at five hundred in this example. For the shape matching task, our concrete version of `loss()` in line 43 operates by first running the iterative FDM to obtain a funicular geometry $\mathbf{x}$, and then measures the mean pairwise distance to the target $\hat{\mathbf{x}}$, which is then returned as the loss value.

The third and final block executes `fdm_inverse()`, tying all the preceding blocks together. We input an initial guess for the FDM parameters θ , the step size for gradient descent $\mu = 10^{-1}$, and the loss function $\mathcal{L}$ for shape matching. The loss function implementation `loss()` closes over the initial guess $\mathbf{x}_0$ and the target $\hat{\mathbf{x}}$ to simplify the code. Gradient descent iterates until the iteration budget is consumed or the stopping criteria are met. In the latter case, the algorithm returns optimal parameters θ^* that result in a funicular geometry that minimizes the loss. We print these parameters to the console in the last line of the code for inspection. The wall time performance of the code can be significantly improved by leveraging `jax.jit()` to compile the code into low-level machine code that is faster to execute, or by using a more powerful gradient-based optimizer. We apply both of these enhancements in our extended DFDM implementation [137].

References

- [1] J. Schlaich, Conceptual design of light structures, *J. Int. Assoc. Shell Spat. Struct.* 45 (3) (2004) 157–168.
- [2] S. Adriaenssens, P. Block, D. Veenendaal, C. Williams (Eds.), *Shell Structures for Architecture: Form Finding and Optimization*, Routledge/ Taylor & Francis Group, London ; New York, London ; New York, 2014.
- [3] C.J. Sun, S.R. Chen, G.Y. Xu, X.M. Liu, N. Yang, Global variation and uniformity of eggshell thickness for chicken eggs, *Poult. Sci.* 91 (10) (2012) 2718–2721. <https://doi.org/10.3382/ps.2012-02220>
- [4] M. Rippmann, M. Popescu, E. Augustynowicz, T. Méndez Echenagucia, C. Calvo Barentin, U. Frick, T. Van Mele, P. Block, The armadillo vault: computational design and digital fabrication of a freeform stone shell, in: *Advances in Architectural Geometry 2016*, vdf Hochschulverlag AG an der ETH Zürich, Zurich, Switzerland, 2016, pp. 344–363. <https://doi.org/10.3218/3778-4.23>
- [5] H. Engel, *Tragsysteme - Structure Systems*, Hatje Cantz Verlag, Ostfildern, 5th edition, Ostfildern, 2013.
- [6] K.-U. Bletzinger, R. Wüchner, F. Daoud, N. Camprubi, Computational methods for form finding and optimization of shells and membranes, *Comput. Methods Appl. Mech. Eng.* 194 (30) (2005) 3438–3452. <https://doi.org/10.1016/j.cma.2004.12.026>
- [7] M. Kilian, D. Pellis, J. Wallner, H. Pottmann, Material-minimizing forms and structures, *ACM Trans. Graph.* 36 (6) (2017) 1–12. <https://doi.org/10.1145/3130800.3130827>
- [8] F. Otto, B. Rasch, *Finding Form: Towards an Architecture of the Minimal*, Edition Axel Menges, Stuttgart, 3rd edition, Stuttgart, 1996.
- [9] J. Schlaich, K. Schafer, M. Jennewein, Toward a consistent design of structural concrete, *PCI J.* 32 (3) (1987) 74–150. <https://doi.org/10.15554/pcij.05011987.74.150>
- [10] P. Marti, *Theory of Structures: Fundamentals, Framed Structures, Plates and Shells*, Wiley Ernst and Sohn, Berlin, Berlin, 2013.
- [11] P. Varignon, *Nouvelle Mécanique Ou Statique: Dont Le Projet Fut Donn  En M. DC. LXXXVII, 2*, Jombert, 1725.
- [12] W.J.M. Rankine, XVII. Principle of the equilibrium of polyhedral frames, *The London Edinburgh Dublin Philosoph. Magaz. J. Sci.* 27 (180) (1864) 92. <https://doi.org/10.1080/14786446408643629>
- [13] J.C. Maxwell, On reciprocal figures, frames, and diagrams of forces, *Transac. R. Soc. Edinburgh* 26 (1) (1870) 1–40. <https://doi.org/10.1017/S0080456800026351>
- [14] A. Pucher, Die berechnung der dehnungsspannungen von rotationsschalen mit hilfe von spannungsfunktionen (1937). <https://doi.org/10.5169/SEALS-6163>
- [15] S.P. Timoshenko, S. Woinowsky-Krieger, *Theory of Plates and Shells*, McGraw-Hill Classic Textbook Reissue, McGraw-Hill, New York, 2 edition, New York, 1959.
- [16] Y.-C. Chiang, A. Borgart, A form-finding method for membrane shells with radial basis functions, *Eng. Struct.* 251 (2022) 113514. <https://doi.org/10.1016/j.engstruct.2021.113514>
- [17] M. Miki, Almost-analytical solution of pucher's equation in polar coordinates, *J. Int. Assoc. Shell Spat. Struct.* 66 (2) (2025) 105–112. <https://doi.org/10.20898/j.iaass.2025.004>
- [18] D. Veenendaal, P. Block, An overview and comparison of structural form finding methods for general networks, *Int. J. Solids Struct.* 49 (26) (2012) 3741–3753. <https://doi.org/10.1016/j.ijsolstr.2012.08.008>
- [19] J.H. Argyris, T. Angelopoulos, B. Bichat, A general method for the shape finding of lightweight tension structures, *Comput. Methods Appl. Mech. Eng.* 3 (1) (1974) 135–149. [https://doi.org/10.1016/0045-7825\(74\)90046-2](https://doi.org/10.1016/0045-7825(74)90046-2)
- [20] B. Tabarok, Z. Qin, Nonlinear analysis of tension structures, *Comput. Struct.* 45 (5) (1992) 973–984. [https://doi.org/10.1016/0045-7949\(92\)90056-6](https://doi.org/10.1016/0045-7949(92)90056-6)
- [21] M.R. Barnes, Form finding and analysis of tension structures by dynamic relaxation, *Int. J. Space Struct.* 14 (2) (1999) 89–104. <https://doi.org/10.1260/0266351991494722>

- [22] S. Adriaenssens, L. Ney, E. Bodarwe, C. Williams, Finding the form of an irregular meshed steel and glass shell based on construction constraints, *J. Archit. Eng.* 18 (3) (2012) 206–213. [https://doi.org/10.1061/\(ASCE\)AE.1943-5568.0000074](https://doi.org/10.1061/(ASCE)AE.1943-5568.0000074)
- [23] A. Kilian, J. Ochsendorf, Particle-spring systems for structural form finding, *J. Int. Assoc. Shell Spat. Struct.* 46 (2) (2005) 77–84.
- [24] T. Nouri Baranger, Form finding method of tensile fabric structures: revised geometric stiffness method, *J. Int. Assoc. Shell Spat. Struct.* 43 (1) (2002) 13–21.
- [25] R.M.O. Pauletti, P.M. Pimenta, The natural force density method for the shape finding of taut structures, *Comput. Methods Appl. Mech. Eng.* 197 (49) (2008) 4419–4428. <https://doi.org/10.1016/j.cma.2008.05.017>
- [26] P. Block, J. Ochsendorf, Thrust network analysis: a new methodology for three-dimensional equilibrium, *J. Int. Assoc. Shell Spat. Struct.* 48 (3) (2007) 167–173.
- [27] P.O. Ohlbrock, P. D'Acunzio, A computer-aided approach to equilibrium design based on graphic statics and combinatorial variations, *Comput.-Aided Des.* 121 (2020) 102802. <https://doi.org/10.1016/J.CAD.2019.102802>
- [28] K. Linkwitz, H.J. Schek, Einige bemerkungen zur berechnung von vorgespannten seilnetzkonstruktionen, *Ingenieur-Archiv* 40 (3) (1971) 145–158. <https://doi.org/10.1007/BF00532146>
- [29] H.J. Schek, The force density method for form finding and computation of general networks, *Comput. Methods Appl. Mech. Eng.* 3 (1) (1974) 115–134. [https://doi.org/10.1016/0045-7825\(74\)90045-0](https://doi.org/10.1016/0045-7825(74)90045-0)
- [30] L. Gründig, E. Moncrieff, P. Singer, D. Ströbel, A history of the principal developments and applications of the force density method in G 1970–1999, in: *Fourth International Colloquium on Computation of Shell and Spatial Structures*, Greece, 2000. https://www.technet-gmbh.com/fileadmin/user_upload/technet/Publikationen/Easy/Density2.pdf
- [31] D. Panozzo, P. Block, O. Sorkine-Hornung, Designing unreinforced masonry models, *ACM Trans. Graph. - SIGGRAPH* 32 (4) (2013) 91:1–91:12. <https://doi.org/10.1145/2461912.2461958>
- [32] F. Marmo, L. Rosati, Reformulation and extension of the thrust network analysis, *Computers & Structures* 182 (2017) 104–118. <https://doi.org/10.1016/j.compstruc.2016.11.016>
- [33] R. Maia Avelino, A. Iannuzzo, T. Van Mele, P. Block, Assessing the safety of vaulted masonry structures using thrust network analysis, *Computers & Structures* 257 (2021) 106647. <https://doi.org/10.1016/j.compstruc.2021.106647>
- [34] P. Cuvilliers, The Constrained Geometry of Structures: Optimization Methods for Inverse Form-Finding Design, Ph.D. thesis, Massachusetts Institute of Technology., 2020. <http://dspace.mit.edu/handle/1721.1/7582>
- [35] M. Miki, K. Kawaguchi, Extended force density method for form-finding of tension structures, *J. Int. Assoc. Shell Spat. Struct.* 51 (4) (2010) 13.
- [36] B. Descamps, R. Filomeno Coelho, L. Ney, Ph. Bouillard, Multicriteria optimization of lightweight bridge structures with a constrained force density method, *Comput. Struct.* 89 (3–4) (2011) 277–284. <https://doi.org/10.1016/j.compstruc.2010.11.010>
- [37] H. Tamai, Advanced application of the force density method in multidisciplinary design practice by incorporating with optimization using analytical derivatives, in: J.B. Obrebski, R. Tarczewski (Eds.), *Proceedings of the International Association for Shell and Spatial Structures (IASS) Symposium 2013*, Wroclaw, Poland, 2013, p. 9.
- [38] K. Takahashi, L. Ney, Advanced form finding by constraint projections for structural equilibrium with design objectives, in: *Proceedings of the IASS Symposium 2018*, Boston, USA, 2018, p. 8.
- [39] R. Hooke, *A Description of Helioscopes, and Some Other Instruments*, Royal Society, London, London, 1676.
- [40] J. Heyman, *Structural Analysis: A Historical Approach*, Cambridge University Press, Cambridge, online-aug edition, Cambridge, 1998. <https://doi.org/10.1017/CBO9780511529580>
- [41] P. Block, M. DeJong, J. Ochsendorf, As hangs the flexible line: equilibrium of masonry arches, *Nexus Netw. J.* 8 (2) (2006) 13–24. <https://doi.org/10.1007/s00004-006-0015-9>
- [42] K.-U. Bletzinger, E. Ramm, A general finite element approach to the form finding of tensile structures by the updated reference strategy, *Int. J. Space Struct.* 14 (2) (1999) 131–145. <https://doi.org/10.1260/0266351991494759>
- [43] T. Oberbichler, K.U. Bletzinger, CAD-Integrated Form-Finding of structural membranes using extended catmull-clark subdivision surfaces, *Comput.-Aided Des.* 151 (2022) 103360. <https://doi.org/10.1016/j.cad.2022.103360>
- [44] E. Ramm, K.-U. Bletzinger, R. Reitinger, Shape optimization of shell structures, *Revue Européenne des Éléments Finis* 2 (3) (1993) 377–398. <https://doi.org/10.1080/12506559.1993.10511083>
- [45] E. Turco, Tools for the numerical solution of inverse problems in structural mechanics: review and research perspectives, *Eur. J. Environ. Civil Eng.* 21 (5) (2017) 509–554. <https://doi.org/10.1080/19648189.2015.1134673>
- [46] A. Gallet, S. Rigby, T.N. Tallman, X. Kong, I. Hajirasouliha, A. Liew, D. Liu, L. Chen, A. Hauptmann, D. Smyl, Structural engineering from an inverse problems perspective, *Proc. R. Soc. A Mathem. Phys. Eng. Sci.* 478 (2257) (2022) 20210526. <https://doi.org/10.1098/rspa.2021.0526>
- [47] R. Oval, R. Pastrana, E.P.G. Bruun, V. Paris, S. Gomis Añiño, S. Adriaenssens, W. Al Asali, An integrated digital design and construction approach for falsework-minimal masonry vaults, *Structures* 63 (2024) 106428. <https://doi.org/10.1016/j.istruc.2024.106428>
- [48] E. Vouga, M. Höbinger, J. Wallner, H. Pottmann, Design of self-supporting surfaces, *ACM Trans. Graph.* 31 (4) (2012) 1–11. <https://doi.org/10.1145/2185520.2185583>
- [49] F. Marmo, D. Masi, D. Mase, L. Rosati, Thrust network analysis of masonry vaults, *Int. J. Masonry Res. Innov.* 4 (1/2) (2019) 64. <https://doi.org/10.1504/IJMRL.2019.096828>
- [50] W. Arnold, The structural engineer's responsibility in this climate emergency, *Struct. Eng.* (2020) 10–11. [https://www.istructe.org/journal/volumes/volume-98-\(2020\)/issue-6/the-structural-engineers-responsibility/](https://www.istructe.org/journal/volumes/volume-98-(2020)/issue-6/the-structural-engineers-responsibility/)
- [51] C.F. Dunant, M.P. Drewniok, J.J. Orr, J.M. Allwood, Good early stage design decisions can halve embodied CO2 and lower structural frames' cost, *Structures* 33 (2021) 343–354. <https://doi.org/10.1016/j.istruc.2021.04.033>
- [52] D.P. Bertsekas, *Nonlinear Programming*, Athena scientific, Belmont, Mass, 3rd ed edition, Belmont, Mass, 2016.
- [53] C. Cui, H. Ohmori, M. Sasaki, Computational morphogenesis of 3D structures by extended ESO method, *J. Int. Assoc. Shell Spat. Struct.* 44 (1) (2003) 51–61.
- [54] D. Wang, W.H. Zhang, J.S. Jiang, Truss shape optimization with multiple displacement constraints, *Comput. Methods Appl. Mech. Eng.* 191 (33) (2002) 3597–3612. [https://doi.org/10.1016/S0045-7825\(02\)00297-9](https://doi.org/10.1016/S0045-7825(02)00297-9)
- [55] C. Paul, H. Lipson, F.J.V. Cuevas, Evolutionary form-finding of tensegrity structures, in: *Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation, GECCO '05*, Association for Computing Machinery, New York, NY, USA, 2005, pp. 3–10. <https://doi.org/10.1145/1068009.1068011>
- [56] R. Kicingier, T. Arciszewski, K. DeJong, Evolutionary design of steel structures in tall buildings, *J. Comput. Civil Eng.* 19 (3) (2005) 223–238. [https://doi.org/10.1061/\(ASCE\)0887-3801\(2005\)19:3\(223\)](https://doi.org/10.1061/(ASCE)0887-3801(2005)19:3(223))
- [57] A. Pugnale, M. Sassone, Morphogenesis and structural optimization of shell structures with the aid of a genetic algorithm, *J. Int. Assoc. Shell Spat. Struct.* 48 (3) (2007) 6.
- [58] K. Koohestani, Form-finding of tensegrity structures via genetic algorithm, *Int. J. Solids Struct.* 49 (5) (2012) 739–747. <https://doi.org/10.1016/j.ijsolstr.2011.11.015>
- [59] T. Méndez-Echenagucia, *Computational Search in Architectural Design*, Ph.D. thesis, Politecnico di Torino, 2013.
- [60] C.T. Mueller, *Computational Exploration of the Structural Design Space*, Ph.D. thesis, Massachusetts Institute of Technology, 2014.
- [61] N.D. Lagaros, M. Papadarakakis, G. Kokossalakis, Structural optimization using evolutionary algorithms, *Comput. Struct.* 80 (7–8) (2002) 571–589. [https://doi.org/10.1016/S0045-7949\(02\)00027-5](https://doi.org/10.1016/S0045-7949(02)00027-5)
- [62] L. Mei, Q. Wang, Structural optimization in civil engineering: a literature review, *Buildings* 11 (2) (2021) 66. <https://doi.org/10.3390/buildings11020066>
- [63] S. Ruder, An Overview of Gradient Descent Optimization Algorithms, *arXiv:1609.04747* (2017). 1609.04747
- [64] W.F. Baker, Energy-based design of lateral systems, *Struct. Eng. Int.* 2 (2) (1992) 99–102. <https://doi.org/10.2749/101686692780615950>
- [65] C.-M. Chan, A.N. Sherbourne, D.E. Grierson, Stiffness optimization technique for 3D tall steel building frameworks under multiple lateral loadings, *Eng. Struct.* 16 (8) (1994) 570–576. [https://doi.org/10.1016/0141-0296\(94\)90042-6](https://doi.org/10.1016/0141-0296(94)90042-6)

- [66] C.K. Kim, H.S. Kim, J.S. Hwang, S.M. Hong, Stiffness-Based optimal design of tall steel frameworks subject to lateral loading, *Struct. Optimiz.* 15 (3) (1998) 180–186. <https://doi.org/10.1007/BF01203529>
- [67] J.Y. Zhang, M. Ohsaki, Adaptive force density method for form-finding problem of tensegrity structures, *Int. J. Solids Struct.* 43 (18) (2006) 5658–5673. <https://doi.org/10.1016/j.ijsolstr.2005.10.011>
- [68] M. Quagliaroli, P.G. Malerba, Flexible bridge decks suspended by cable nets. a constrained form finding approach, *Int. J. Solids Struct.* 50 (14) (2013) 2340–2352. <https://doi.org/10.1016/j.ijsolstr.2013.03.009>
- [69] U.T. Ringertz, Numerical methods for optimization of nonlinear shell structures, *Struct. Optimiz.* 4 (3-4) (1992) 193–198. <https://doi.org/10.1007/BF01742744>
- [70] A. Griewank, A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, SIAM, 2008.
- [71] G. Corliss, C. Faure, A. Griewank, L. Hascoet, U. Naumann, *Automatic Differentiation of Algorithms: From Simulation to Optimization*, Springer, New York, NY, New York, NY, 2013. <https://link.springer.com/book/10.1007/978-1-4613-0075-5>.
- [72] M. Blondel, V. Roulet, *The Elements of Differentiable Programming*, 2025, arXiv:2403.14606.
- [73] F.L. Bauer, Computational graphs and rounding error, *SIAM J. Numer. Anal.* 11 (1) (1974) 87–96. <https://doi.org/10.1137/0711010>
- [74] D. Oktay, N. McGreiv, J. Adul, A. Beatson, R.P. Adams, Randomized automatic differentiation, in: *International Conference on Learning Representations*, 2020, pp. 1–19. <https://openreview.net/forum?id=xpx9zj7CUIY>.
- [75] S. Linnainmaa, Taylor expansion of the accumulated rounding error, *BIT Numer. Mathem.* 16 (2) (1976) 146–160. <https://doi.org/10.1007/BF01931367>
- [76] D.E. Rumelhart, G.E. Hinton, R.J. Williams, Learning representations by back-propagating errors, *Nature* 323 (6088) (1986) 533–536. <https://doi.org/10.1038/323533a0>
- [77] R. Pastrana, P.O. Ohlbrock, T. Oberbichler, P. D’Acunto, S. Parascho, Constrained form-finding of tension–compression structures using automatic differentiation, *Comput.-Aided Des.* 155 (2023) 103435. <https://doi.org/10.1016/j.cad.2022.103435>
- [78] A.G. Baydin, B.A. Pearlmutter, A.A. Radul, J.M. Siskind, Automatic Differentiation in Machine Learning: A Survey, arXiv:1502.05767 (2018).
- [79] F.L. Bauer, Computational graphs and rounding error, *SIAM J. Numer. Anal.* 11 (1) (1974) 87–96. 2156433
- [80] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (7553) (2015) 436–444. <https://doi.org/10.1038/nature14539>
- [81] S.J.D. Prince, *Understanding Deep Learning*, The MIT Press, Cambridge, Massachusetts, 2023.
- [82] N. Thurey, P. Holl, M. Mueller, P. Schnell, F. Trost, K. Um, Physics-Based Deep Learning, 2022, <https://doi.org/10.48550/arXiv.2109.05237>
- [83] F. de Avila Belbute-Peres, K. Smith, K. Allen, J. Tenenbaum, J.Z. Kolter, End-to-end differentiable physics for learning and control, in: *Advances in Neural Information Processing Systems*, 31, Curran Associates, Inc., 2018. <https://papers.nips.cc/paper/2018/hash/842424a1d0595b76ec4fa03c46e8d755-Abstract.html>
- [84] J. Degraeve, M. Hermans, J. Dambre, F. Wyffels, A differentiable physics engine for deep learning in robotics, *Front. Neurorobot.* 13 (2019) 6. <https://doi.org/10.3389/fnbot.2019.00006>
- [85] Y. Hu, L. Anderson, T.-M. Li, Q. Sun, N. Carr, J. Ragan-Kelley, F. Durand, DiffTaichi: Differentiable Programming for Physical Simulation (2019). arXiv:1910.00935v3.
- [86] J.K. Murthy, M. Maklin, F. Golemo, V. Voleti, L. Petrini, M. Weiss, B. Considine, J. Parent-Lévesque, K. Xie, K. Erleben, L. Paull, F. Shkurti, D. Nowrouzezahrai, S. Fidler, gradSim: differentiable simulation for system identification and visuomotor control, in: *International Conference on Learning Representations*, 2020. https://openreview.net/forum?id=c_E8kFWfhp0
- [87] P. Holl, V. Koltun, N. Thurey, Learning to Control PDEs with Differentiable Physics (2020). arXiv:2001.07457v1.
- [88] S. Schoenholz, E.D. Cubuk, JAX MD: a framework for differentiable physics, in: *Advances in Neural Information Processing Systems*, 33, Curran Associates, Inc., 2020, pp. 11428–11441. <https://proceedings.neurips.cc/paper/2020/hash/83d3d4b6c9579515e1679aca8cbc8033-Abstract.html>
- [89] G. Bordiga, E. Medina, S. Jafarzadeh, C. Bösch, R.P. Adams, V. Tournat, K. Bertoldi, Automated discovery of reprogrammable nonlinear dynamic metamaterials, *Nat. Mater.* 23 (11) (2024) 1486–1494. <https://doi.org/10.1038/s41563-024-02008-6>
- [90] D. Oktay, M. Mirramezani, E. Medina, R.P. Adams, Neuromechanical autoencoders: learning to couple elastic and neural network nonlinearity, in: *International Conference on Learning Representations*, Kigali, Rwanda, 2023. https://openreview.net/pdf?id=QubsmJT_A0
- [91] G. Haase, U. Langer, E. Lindner, W. Mühlhuber, Optimal Sizing Using Automatic Differentiation, in: K.-H. Hoffmann, R.H.W. Hoppe, V. Schulz (Eds.), *Fast Solution of Discretized Optimization Problems*, ISNM International Series of Numerical Mathematics, Birkhäuser, Basel, 2001, pp. 120–138. https://doi.org/10.1007/978-3-0348-8233-0_10
- [92] A. Chandrasekhar, S. Sridhara, K. Suresh, AuTO: a framework for automatic differentiation in topology optimization, *Struct. Multidiscip. Optim.* 64 (6) (2021) 4355–4365. <https://doi.org/10.1007/s00158-021-03025-8>
- [93] S. Hoyer, J. Sohl-Dickstein, S. Greydanus, Neural Reparameterization Improves Structural Optimization (2019). arXiv:1909.04240v2.
- [94] T. Xue, S. Liao, Z. Gan, C. Park, X. Xie, W.K. Liu, J. Cao, JAX-FEM: a differentiable GPU-accelerated 3D finite element solver for automatic inverse design and mechanistic data science, *Comput. Phys. Commun.* 291 (2023) 108802. <https://doi.org/10.1016/j.cpc.2023.108802>
- [95] G. Wu, A framework for structural shape optimization based on automatic differentiation, the adjoint method and accelerated linear algebra, *Struct. Multidiscip. Optim.* 66 (7) (2023) 151. <https://doi.org/10.1007/s00158-023-03601-0>
- [96] G. Wu, JAX-SSO: Differentiable Finite Element Analysis Solver for Structural Optimization and Seamless Integration with Neural Networks, 2024, <https://doi.org/10.48550/arXiv.2407.20026>
- [97] K.J. Lee, Y. Huang, C.T. Mueller, A differentiable structural analysis framework for high-performance design optimization, *Structures* 78 (2025) 109292. <https://doi.org/10.1016/j.istruc.2025.109292>
- [98] G. Wu, R. Pastrana, S. Adriaenssens, M.E.M. Garlock, A numerical comparison between geometric stiffness- and elastic stiffness-Based structural shape optimization for gridshells, *Proceedings of IASS Annual Symposia* 2023 (22) (2023) 1–13.
- [99] K.M.M. Tam, T. Mele, P. Block, Trans-topological learning and optimisation of reticulated equilibrium shell structures with automatic differentiation and CW complexes message passing, in: *Proceedings of the IASS 2022 Symposium Affiliated with APCS 2022 Conference*, International Association for Shell and Spatial Structures (IASS) and Asian-Pacific Conference on Shell and Spatial Structures (APCS), Beijing, China, 2022.
- [100] K.M.M. Tam, Learning Discrete Equilibrium: Trans-topological Inverse Pattern and Force Design Using Machine Learning & Automatic Differentiation, Ph.D. thesis, ETH Zurich, 2023. <https://doi.org/10.3929/ETHZ-B-000661366>
- [101] A. Burke, K. Lee, J. Echelman, D. Feldman, M. Caitlin, FDMRemote: interactive inverse design of tensile structures with differentiable FDM, in: *Proceedings of the IASS Annual Symposium*, Melbourne, Australia 2023, 2023.
- [102] R. Pastrana, D. Oktay, R.P. Adams, S. Adriaenssens, JAX FDM: a differentiable solver for inverse form-finding, in: *Differentiable Almost Everything Workshop of the 40th International Conference on Machine Learning*, Hawaii, USA, 2023. <https://openreview.net/forum?id=Uu9OPgh24d>
- [103] K. Schwilizerhof, E. Ramm, Displacement dependent pressure loads in nonlinear finite element analyses, *Comput. Struct.* 18 (6) (1984) 1099–1114. [https://doi.org/10.1016/0045-7949\(84\)90154-8](https://doi.org/10.1016/0045-7949(84)90154-8)
- [104] A. Niewiarowski, S. Adriaenssens, R.M. Pauletti, Adjoint optimization of pressurized membrane structures using automatic differentiation tools, *Comput. Methods Appl. Mech. Eng.* 372 (2020) 113393. <https://doi.org/10.1016/j.cma.2020.113393>
- [105] G.I.N. Rozvany, C.M. Wang, On plane prager-structures—I, *Int. J. Mech. Sci.* 25 (7) (1983) 519–527. [https://doi.org/10.1016/0020-7403\(83\)90044-9](https://doi.org/10.1016/0020-7403(83)90044-9)
- [106] C. Tang, X. Sun, A. Gomes, J. Wallner, H. Pottmann, Form-finding with polyhedral meshes made simple, *ACM Trans. Graph.* 33 (4) (2014) 70:1–70:9. <https://doi.org/10.1145/2601097.2601213>
- [107] A. Liew, Constrained force density method optimisation for compression-only shell structures, *Structures* 28 (2020) 1845–1856. <https://doi.org/10.1016/j.istruc.2020.09.078>
- [108] Z. Wan, P.O. Ohlbrock, P. D’Acunto, Z. Cao, F. Fan, J. Schwartz, A form-finding approach for the conceptual design of air-supported structures using 3D graphic statics, *Comput. Struct.* 243 (2021) 106401. <https://doi.org/10.1016/j.compstruc.2020.106401>
- [109] T. Pagnacco, G. Masera, A. Weilandt, N. Baldassini, A framework for the optimisation of glazed gridshells in the early design stage under structural and energy-based criteria, *IOP Conference Series: Earth and Environmental Science* 296 (2019) 012057. <https://doi.org/10.1088/1755-1315/296/1/012057>

- [110] L.L. Beghini, J. Carrion, A. Beghini, A. Mazurek, W.F. Baker, Structural optimization using graphic statics, *Struct. Multidiscip. Optim.* 49 (3) (2014) 351–366. <https://doi.org/10.1007/s00158-013-1002-x>
- [111] P. D'Acunto, J.-P. Jasienski, P.O. Ohlbrock, C. Fivet, J. Schwartz, D. Zastavni, Vector-based 3D graphic statics: a framework for the design of spatial structures based on the relation between form and forces, *Int. J. Solids Struct.* 167 (2019) 58–70. <https://doi.org/10.1016/J.IJSOLSTR.2019.02.008>
- [112] K.-U. Bletzinger, IASS Workshop "Form Finding by the Force Density Method", 2025, <https://www.youtube.com/playlist?list=PLxHC49JkrQhmFQgx5FDADTNeUw3WC6-0>.
- [113] K. Linkwitz, Formfinding by the "direct approach" and pertinent strategies for the conceptual design of prestressed and hanging structures, *Int. J. Space Struct.* 14 (2) (1999) 73–87. <https://doi.org/10.1260/0266351991494713>
- [114] S. Liu, G. Trenkler, Hadamard, Khatri-Rao, kronecker and other matrix products, *Int. J. Inf. Syst. Sci.* 4 (1) (2008) 160–177. <https://zbmath.org/1159.15008>.
- [115] Y. Saad, *Iterative Methods for Sparse Linear Systems*, SIAM, Philadelphia, 2nd ed edition, Philadelphia, 2003.
- [116] J.-T. Chen, W.-S. Huang, J.-W. Lee, Y.-C. Tu, A self-regularized approach for deriving the free-free flexibility and stiffness matrices, *Comput. Struct.* 145 (2014) 12–22. <https://doi.org/10.1016/j.compstruc.2014.07.024>
- [117] M. Rippmann, *Funicular Shell Design: Geometric Approaches to Form Finding and Fabrication of Discrete Funicular Structures*, Ph.D. thesis, ETH Zurich, 2016. <https://doi.org/10.3929/ETHZ-A-010656780>
- [118] C. Du, Z. Wang, T. Van Mele, P. Block, Design of restrained pneumatic formwork – inverse form finding and materialisation for free form geometries, in: *Proceedings of the IASS 2024 Symposium*, Zurich, Switzerland, 2024. https://app.iass2024.org/files/IASS_2024_Paper_337.pdf.
- [119] M. Botsch, L. Kobbelt, M. Pauly, P. Alliez, B. Levy, *Polygon Mesh Processing*, A K Peters/CRC Press, 0 edition, 2010. <https://doi.org/10.1201/b10688>
- [120] T.V. Mele, et al., COMPAS: A framework for computational research in architecture and structures, <http://compas.dev>, (2017–2021). <https://doi.org/10.5281/zenodo.2594510>
- [121] M. AlQuraishi, End-to-end differentiable learning of protein structure, *Cell Syst.* 8 (4) (2019) 292–301.e3. <https://doi.org/10.1016/j.cels.2019.03.006>
- [122] C.P. Goodrich, E.M. King, S.S. Schoenholz, E.D. Cubuk, M.P. Brenner, Designing self-assembling kinetics with differentiable statistical physics models, *Proc. Natl. Acad. Sci.* 118 (10) (2021) e2024083118. <https://doi.org/10.1073/pnas.2024083118>
- [123] K.R. Allen, T. Lopez-Guevara, K. Stachenfeld, A. Sanchez-Gonzalez, P. Battaglia, J. Hamrick, T. Pfaff, Physical Design Using Differentiable Learned Simulators, (2022). <http://arxiv.org/abs/2202.00728>.
- [124] D. Veenendaal, J. Bakker, P. Block, Structural design of the flexibly formed, mesh-reinforced concrete sandwich shell roof of NEST HiLo, *J. Int. Assoc. Shell Spat. Struct.* 58 (1) (2017) 23–38. <https://doi.org/10.20898/j.ass.2017.191.847>
- [125] D. Veenendaal, K. Linkwitz, Nonlinear force density method: constraints of force and geometry, in: *Shell Structures for Architecture*, Routledge, London, 1st edition, 2014. <https://doi.org/10.4324/9781315849270>.
- [126] L. Lachauer, P. Block, Interactive equilibrium modelling, *Int. J. Space Struct.* 29 (1) (2014) 25–37. <https://doi.org/10.1260/0266-3511.29.1.25>
- [127] J. Nocedal, Updating quasi-Newton matrices with limited storage, *Math. Comput.* 35 (151) (1980) 773. <https://doi.org/10.1090/S0025-5718-1980-0572855-7>
- [128] D. Kraft, Algorithm 733: TOMP–Fortran modules for optimal control calculations, *ACM Trans. Math. Software* 20 (3) (1994) 262–281. <https://doi.org/10.1145/192115.192124>
- [129] C. Kanzow, N. Yamashita, M. Fukushima, Levenberg–Marquardt methods with strong local convergence properties for solving nonlinear equations with convex constraints, *J. Comput. Appl. Math.* 172 (2) (2004) 375–397. <https://doi.org/10.1016/j.cam.2004.02.013>
- [130] K. Takahashi, Advanced form-finding with design constraints and objectives through constraint projection, *Proceedings of IASS Annual Symposia 2020* (7) (2020) 1–12.
- [131] R.T. Haftka, Simultaneous analysis and design, *AIAA J.* 23 (7) (1985) 1099–1103. <https://doi.org/10.2514/3.9043>
- [132] P.G. Malerba, M. Patelli, M. Quagliaroli, An extended force density method for the form finding of cable systems with new forms, *Struct. Eng. Mechan.* 42 (2012) 191–210. <https://doi.org/10.12989/SEM.2012.42.2.191>
- [133] P. Cuvilliers, R. Danhaive, C. Mueller, Gradient-based optimization of closest-fit funicular structures, in: K. Kawaguchi, M. Ohsaki, T. Takeuchi (Eds.), *Proceedings of the IASS Annual Symposium 2016*, Tokyo, Japan, 2016, p. 10.
- [134] M. Ohsaki, K. Hayashi, Force density method for simultaneous optimization of geometry and topology of trusses, *Struct. Multidiscip. Optim.* 56 (5) (2017) 1157–1168. <https://doi.org/10.1007/s00158-017-1710-8>
- [135] A. Liew, D. Pagonakis, T. Van Mele, P. Block, Load-path optimisation of funicular networks, *Meccanica* 53 (1) (2018) 279–294. <https://doi.org/10.1007/s11012-017-0714-1>
- [136] J. Bradbury, R. Frostig, P. Hawkins, M.J. Johnson, C. Leary, D. Maclaurin, S. Wanderman-Milne, JAX: Composable Transformations of Python + NumPy Programs, 2018, <http://github.com/google/jax>.
- [137] R. Pastrana, D. Oktay, R.P. Adams, S. Adriaenssens, JAX FDM: Auto-differentiable and Hardware-Accelerated Force Density Method, 2023, <https://doi.org/10.5281/zenodo.7258292>
- [138] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: an imperative style, high-performance deep learning library, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. dAlché-Buc, E. Fox, R. Garnett (Eds.), *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019, pp. 8024–8035. <https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [139] J. Bezanson, A. Edelman, S. Karpinski, V.B. Shah, Julia: a fresh approach to numerical computing, *SIAM Rev.* 59 (1) (2017) 65–98. <https://doi.org/10.1137/14100671>
- [140] K.P. Murphy, *Probabilistic Machine Learning: Advanced Topics*, MIT Press, 2023. <http://probml.github.io/book2>.
- [141] S. Walter, Structured Higher-Order Algorithmic Differentiation in the Forward and Reverse Mode with Application in Optimum Experimental Design, 2012, <https://doi.org/10.18452/16514>
- [142] U. Naumann, *The Art of Differentiating Computer Programs: An Introduction to Algorithmic Differentiation*, Society for Industrial and Applied Mathematics, 2011. <https://doi.org/10.1137/1.9781611972078>
- [143] S.G. Krantz, H.R. Parks, *The Implicit Function Theorem: History, Theory, and Applications*, Springer, New York, NY, New York, NY, 2013. <https://doi.org/10.1007/978-1-4614-5981-1>
- [144] D. Scieur, Q. Bertrand, G. Gidel, F. Pedregosa, The curse of unrolling: rate of differentiating through optimization, in: *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, Curran Associates Inc., Red Hook, NY, USA, 2022.
- [145] S. Hochreiter, The vanishing gradient problem during learning recurrent neural nets and problem solutions, *Int. J. Uncert. Fuzziness Knowl. Based Syst.* 06 (02) (1998) 107–116. <https://doi.org/10.1142/S0218488598000094>
- [146] J.L. Nazareth, Conjugate gradient method, *WIREs Comput. Stat.* 1 (3) (2009) 348–353. <https://doi.org/10.1002/wics.13>
- [147] D.C. Liu, J. Nocedal, On the limited memory BFGS method for large scale optimization, *Math. Program.* 45 (1) (1989) 503–528. <https://doi.org/10.1007/BF01589116>
- [148] K.-U. Bletzinger, E. Ramm, Structural optimization and form finding of light weight structures, *Comput. Struct.* 79 (22) (2001) 2053–2062. [https://doi.org/10.1016/S0045-7949\(01\)00052-9](https://doi.org/10.1016/S0045-7949(01)00052-9)
- [149] M. Bruggi, A constrained force density method for the funicular analysis and design of arches, domes and vaults, *Int. J. Solids Struct.* 193–194 (2020) 251–269. <https://doi.org/10.1016/j.ijsolstr.2020.02.030>
- [150] M.J.D. Powell, An efficient method for finding the minimum of a function of several variables without calculating derivatives, *Comput. J.* 7 (2) (1964) 155–162. <https://doi.org/10.1093/comjnl/7.2.155>
- [151] Y. Xiang, D.Y. Sun, W. Fan, X.G. Gong, Generalized simulated annealing algorithm and its application to the Thomson model, *Phys. Lett. A* 233 (3) (1997) 216–220. [https://doi.org/10.1016/S0375-9601\(97\)00474-X](https://doi.org/10.1016/S0375-9601(97)00474-X)
- [152] P. Virtanen, R. Gommers, T.E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S.J. van der Walt, M. Brett, J. Wilson, K.J. Millman, N. Mayorov, A.R.J. Nelson, E. Jones, R. Kern, E. Larson, C.J. Carey, Í. Polat, Y. Feng, E.W. Moore, J. VanderPlas, D. Laxalde, J.

- Perktold, R. Cimrman, I. Henriksen, E.A. Quintero, C.R. Harris, A.M. Archibald, A.H. Ribeiro, F. Pedregosa, P. van Mulbregt, SciPy 1.0 Contributors, SciPy 1.0: fundamental algorithms for scientific computing in python, *Nat. Methods* 17 (2020) 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- [153] M. Streeter, L. Rhode-Barbarigos, S. Adriaenssens, Form finding and analysis of inflatable dams using dynamic relaxation, *Appl. Math. Comput.* 267 (2015) 742–749. <https://doi.org/10.1016/j.amc.2014.12.054>
- [154] J. Bonet, R.D. Wood, J. Mahaney, P. Heywood, Finite element analysis of air supported membrane structures, *Comput. Methods Appl. Mech. Eng.* 190 (5) (2000) 579–595. [https://doi.org/10.1016/S0045-7825\(99\)00428-4](https://doi.org/10.1016/S0045-7825(99)00428-4)
- [155] M. Skouras, B. Thomaszewski, B. Bickel, M. Gross, Computational design of rubber balloons, *Comput. Graph. Forum* 31 (2pt4) (2012) 835–844. <https://doi.org/10.1111/j.1467-8659.2012.03064.x>
- [156] M. Skouras, B. Thomaszewski, P. Kaufmann, A. Garg, B. Bickel, E. Grinspun, M. Gross, Designing inflatable structures, *ACM Trans. Graph.* 33 (4) (2014) 63:1–63:10. <https://doi.org/10.1145/2601097.2601166>
- [157] V. Fift, Statics, formfinding, and dynamics of air-supported membrane structures, number v. 5 in *Mechanics of surface structures*, M. Nijhoff ; Distributors for the United States and Canada, Kluwer Boston, The Hague ; Boston : Hingham, MA, The Hague ; Boston : Hingham, MA, 1983.
- [158] P.R.M. De Oliveira, Some issues on the design and analysis of pneumatic structures, *Int. J. Struct. Eng.* 1 (3/4) (2010) 217. <https://doi.org/10.1504/IJSTRUCTE.2010.033480>
- [159] R. Maia Avelino, A. Iannuzzo, T. Van Mele, P. Block, An energy-based strategy to find admissible thrust networks compatible with foundation settlements in masonry structures, *Mech. Res. Commun.* 125 (2022) 103978. <https://doi.org/10.1016/j.mechrescom.2022.103978>
- [160] S. Huerta, Arcos, bóvedas y cúpulas: geometría y equilibrio en el cálculo tradicional de estructuras de fábrica, Ph.D. thesis, Instituto Juan de Herrera, Madrid, 2004.
- [161] S. Huerta, The analysis of masonry architecture: a historical approach, *Archit. Sci. Rev.* 51 (4) (2008) 297–328. <https://doi.org/10.3763/asre.2008.5136>
- [162] P. Moit, *Theory of Structures: Fundamentals, Framed Structures, Plates and Shells*, Wiley, Ernst & Sohn, Berlin, Berlin, 2013.
- [163] J. Heyman, The stone skeleton, *Int. J. Solids Struct.* 2 (2) (1966) 249–279. [https://doi.org/10.1016/0020-7683\(66\)90018-7](https://doi.org/10.1016/0020-7683(66)90018-7)
- [164] L. Jurina, Developments and experimentations of the "reinforced arch method", in: *Structural Analysis of Historical Constructions: Anamnesis, Diagnosis, Therapy, Controls*, CRC Press, 2016.
- [165] L. Jurina, E.O. Radaelli, D. Coronelli, The reinforced arch method for the life of the ancient bridge of omegna, in: *Life-Cycle of Structures and Infrastructure Systems*, CRC Press, 2023.
- [166] L. Todisco, E. Stocks, J. León, H. Corres, Enhancing the structural performance of masonry structures by post-tensioning, *Nexus Netw. J.* 20 (3) (2018) 671–691. <https://doi.org/10.1007/s00004-018-0374-z>
- [167] D. Kraft, A Software Package for Sequential Quadratic Programming, Technical Report DFVLR-FB 88-28, DLR German Aerospace Center — Institute for Flight Mechanics, Köln, Germany, 1988.
- [168] R. Fugger, R. Maia Avelino, A. Iannuzzo, P. Block, G. De Felice, A new numerical limit analysis-based strategy to retrofit masonry curved structures with FRCC systems, in: *8th European Congress on Computational Methods in Applied Sciences and Engineering, CIMNE, 2022*. <https://doi.org/10.23967/eccomas.2022.020>
- [169] D. Pellicchia, F. Marmo, L. Rosati, Accounting for material strength in the thrust line analyses of unstrengthened and strengthened arches, *Compos. Struct.* 358 (2025) 118926. <https://doi.org/10.1016/j.compstruct.2025.118926>
- [170] H. Schober, Transparent Shells: Form Topology Structure, Wiley, 1 edition, 2015. <https://doi.org/10.1002/9783433605998>
- [171] C.J.K. Williams, The analytic and numerical definition of the geometry of the British museum great court roof, in: M. Burry, S. Datta, A. Dawson, A.J. Rollo (Eds.), *Mathematics & Design 2001*, Deakin University, 2001, pp. 434–440.
- [172] J. Knippers, T. Helbig, Recent developments in the design of glazed grid shells, *Int. J. Space Struct.* 24 (2) (2009) 111–126. <https://doi.org/10.1260/026635109789043205>
- [173] T. Helbig, L. Giampellegrini, M. Oppe, "Carioca Wave" – A free-form steel-and-glass canopy in Rio de Janeiro, Brazil, *Steel Constr.* 7 (4) (2014) 252–257. <https://doi.org/10.1002/stco.201420033>
- [174] A. Chadwick, C. Matthews, M.E. Villafane, C.J.K. Williams, PLATO – Gridshell design method for chadstone shopping mall roof, in: *IABSE Conference, Bath 2017: Creativity and Collaboration – Instilling Imagination and Innovation in Structural Design*, International Association for Bridge and Structural Engineering (IABSE), Bath, United Kingdom, 2017, pp. 159–160. <https://doi.org/10.2749/222137817821232621>
- [175] J. Schlaich, H. Schober, St. Justiz, Entwurf und Konstruktion der Bahnsteighalle des Lehrter Bahnhofs, *Stahlbau* 71 (12) (2002) 853–868. <https://doi.org/10.1002/stab.200203020>
- [176] E.M. Kido, Z. Cywiński, The steel-glass art of railway stations in Japan, *Stahlbau* 87 (6) (2018) 611–621. <https://doi.org/10.1002/stab.201820605>
- [177] F. Tahmasebinia, S. Jiang, S. Shirozwan, L. Mann, S.M.E. Sepasgozar, Exploring the integration of architectural design and advanced structural analysis for steel–glass structures: a comparative study of different case scenarios, *Buildings* 13 (6) (2023) 1369. <https://doi.org/10.3390/buildings13061369>
- [178] J. Sischka, B. Brown, E. Handel, G. Zenkner, Die überdachung des great court im british museum in London, *Stahlbau* 70 (7) (2001) 492–502. <https://doi.org/10.1002/stab.200101690>
- [179] R. Oval, R. Mesnil, T.V. Mele, O. Baverel, P. Block, Creasing the British museum: topology finding of crease patterns for shell structures, *J. Int. Assoc. Shell Spat. Struct.* 65 (1) (2024) 44–56. <https://doi.org/10.20898/j.iaass.2024.004>
- [180] R. Oval, M. Rippmann, R. Mesnil, T. Van Mele, O. Baverel, P. Block, Feature-based topology finding of patterns for shell structures, *Autom. Constr.* 103 (2019) 185–201. <https://doi.org/10.1016/j.autcon.2019.02.008>
- [181] A. Schifner, J. Balzer, Statics-sensitive layout of planar quadrilateral meshes, in: C. Ceccato, L. Hesselgren, M. Pauly, H. Pottmann, J. Wallner (Eds.), *Advances in Architectural Geometry 2010*, De Gruyter, 2017, pp. 221–236. <https://doi.org/10.1515/9783990433713-017>
- [182] R. Mesnil, C. Douthe, C. Richter, O. Baverel, Fabrication-aware shape parametrisation for the structural optimisation of shell structures, *Eng. Struct.* 176 (2018) 569–584. <https://doi.org/10.1016/j.engstruct.2018.09.026>
- [183] D. Pellis, H. Pottmann, Aligning principal stress and curvature directions, in: L. Hesselgren, A. Kilian, S. Malek, K.-G. Olsson, O. Sorkine-Hornung, C. Williams (Eds.), *Advances in Architectural Geometry*, Klein Publishing Ltd, 2018, pp. 34–53.
- [184] R. Bouillier, C. Douthe, L. Hauswirth, O. Baverel, Load-finding: a form-finding method for plane-faced funicular gridshells, *J. Int. Assoc. Shell Spat. Struct.* (2025). <https://doi.org/10.20898/j.iaass.2025.003>
- [185] W. Sobek, Ultra-Lightweight construction, *Int. J. Space Struct.* 31 (1) (2016) 74–80. <https://doi.org/10.1177/0266351116643246>
- [186] D. Maclaurin, D. Duvenaud, R. Adams, Gradient-based hyperparameter optimization through reversible learning, in: F. Bach, D. Blei (Eds.), *Proceedings of the 32nd International Conference on Machine Learning*, 37 of *Proceedings of Machine Learning Research*, PMLR, Lille, France, 2015, pp. 2113–2122. <https://proceedings.mlr.press/v37/maclaurin15.html>
- [187] A. Rajeswaran, C. Finn, S.M. Kakade, S. Levine, Meta-learning with implicit gradients, in: *Advances in Neural Information Processing Systems*, 32, 2019. [arxiv:1909.04630](https://arxiv.org/abs/1909.04630)
- [188] Q. Liu, M. Chu, N. Thurey, ConFIG: towards conflict-free training of physics informed neural networks, in: *The Thirteenth International Conference on Learning Representations*, 2024. [arXiv:2408.11104](https://arxiv.org/abs/2408.11104)
- [189] R. Pastrana, E. Medina, I.M. de Oliveira, S. Adriaenssens, R.P. Adams, Real-time design of architectural structures with differentiable mechanics and neural networks, in: *The 13th International Conference on Learning Representations (ICLR)*, 2025. <https://openreview.net/forum?id=Tpj66xwTq>
- [190] W.S. Hemp, The design of structures of least weight. h. l. cox. international series of monographs in aeronautics and astronautics, division 1, volume 8, pergamon press, oxford. 1965. 134 pp. diagrams. tables. £2, *Aeronaut. J.* 70 (662) (1966) 376. <https://doi.org/10.1017/S0368393100082511>
- [191] H.L. Cox, Optimum structures, W. S. hemp, Clarendon press. oxford 1973. 123 P. diagrams. £4.00, *Aeronaut. J.* 77 (750) (1973) 305. <https://doi.org/10.1017/S000192400004094X>
- [192] L.L. Stromberg, A. Beghini, W.F. Baker, G.H. Paulino, Topology optimization for braced frames: combining continuum and beam/column elements, *Eng. Struct.* 37 (2012) 106–124. <https://doi.org/10.1016/j.engstruct.2011.12.034>

- [193] T. Van Mele, P. Block, Algebraic graph statics, CAD Comput. Aided Design (2014). <https://doi.org/10.1016/j.cad.2014.04.004>
- [194] A. Liew, R. Avelino, V. Moosavi, T. Van Mele, P. Block, Optimising the load path of compression-only thrust networks through independent sets, Struct. Multidiscip. Optim. 60 (1) (2019) 231–244. <https://doi.org/10.1007/s00158-019-02214-w>
- [195] M. Smith, ABAQUS Standard user manual, version 6.12, dassault systèmes simulia corp, 2012,
- [196] NVIDIA Corporation, cusolver library, 2023, <https://developer.nvidia.com/cusolver>.
- [197] P.E. Farrell, D.A. Ham, S.W. Funke, M.E. Rognes, Automated derivation of the adjoint of high-level transient finite element programs, SIAM J. Scient. Comput. 35 (4) (2013) C369–C393. <https://doi.org/10.1137/120873558>
- [198] M. Blondel, Q. Berthet, M. Cuturi, R. Frostig, S. Hoyer, F. Llinares-López, F. Pedregosa, J.-P. Vert, Efficient and Modular Implicit Differentiation, 2022, <https://doi.org/10.48550/arXiv.2105.15183>